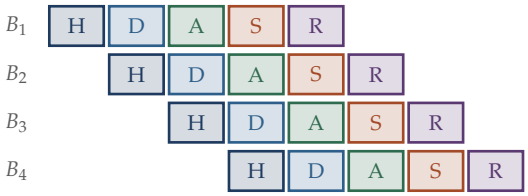
Informatik von Grund auf — Band 4

Die Anatomie der Geschwindigkeit

Eine vollständige Einführung in die Rechnerarchitektur



Von der Pipeline bis zur Speicherhierarchie
Jeder Schritt erklärt. Mit vollständig durchgerechneten Übungsaufgaben.

Informatik von Grund auf — Band 4
Die Anatomie der Geschwindigkeit · Rechnerarchitektur

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Informatik von Grund auf, Band 4: Die Anatomie der Geschwindigkeit*. Fassung 1.0.0.
`https://github.com/juergenmaier2026/informatik-von-grund-auf`

Satz: LaTeX mit Palatino. Fassung 1.0.0, Oktober 2026. 166 Seiten.

Stand der Forschung: August 2026

Inhaltsverzeichnis

Vorwort 1

I Der Datenpfad 9

1 Wie ein Prozessor aufgebaut ist 11

2 Der Befehlssatz als Vertrag 19

II Das Fließband 29

3 Fließbandverarbeitung 31

4 Konflikte im Fließband 39

5 Sprungvorhersage 47

6 Mehrere Befehle gleichzeitig 55

III Das Gedächtnis 65

7 Die Speicherhierarchie 67

8 Cachebewusst programmieren 77

9 Virtueller Speicher 87

IV Vielfalt und Parallelität 95

10 Vektorbefehle 97

11 Mehrere Kerne 105

12 Grafikprozessoren und Spezialbausteine 113

V Wo es hakt **121**

13 Energie als Grenze **123**

14 Wenn die Abstraktion undicht wird **131**

VI Aktueller Stand **141**

15 Wo die Architektur heute steht **143**

VII Anhang **149**

A Übersicht und Zahlen **151**
A.1 Die Zahlen, die man kennen sollte 151
A.2 Die Landkarte des Bandes 153
A.3 Glossar 153
A.4 Zeittafel 156

B Kommentierte Literatur **159**

Vorwort: Wo die Zeit hingeht

Die ersten drei Bände dieser Reihe haben Rechenzeit gezählt, indem sie Operationen gezählt haben. *Das ist eine Vereinfachung, und dieser Band handelt davon, was sie verschweigt.*

Zwei Programme, ein Ergebnis, Faktor sechzig

Die folgenden beiden Schleifen addieren die Einträge derselben Matrix. Sie führen gleich viele Additionen aus, greifen auf dieselben Speicherstellen zu und liefern dasselbe Ergebnis.

```
for i := 1 to n do          for j := 1 to n do
  for j := 1 to n             for i := 1 to n
do                          do
    s := s +                    s := s +
a[i][j]                     a[i][j]
```

Der einzige Unterschied ist die Reihenfolge der beiden Schleifen. Bei einer Matrix von 8192×8192 Gleitkommazahlen misst man auf einem gewöhnlichen Rechner ungefähr das hier:

Reihenfolge	Zeit	Verhältnis
zeilenweise	0,09 s	1
spaltenweise	5,4 s	**60**

> *Die Analyse aus Band 1 sieht keinen Unterschied. Beide Programme brauchen $\Theta(n^2)$ Operationen, und die Konstanten sind identisch.*

Der Faktor sechzig entsteht vollständig unterhalb der Ebene, auf der man normalerweise über Programme nachdenkt. Er kommt daher, dass die eine Fassung den Speicher in der Reihenfolge liest, in der er abgelegt ist, und die andere nicht.

Das Kostenmodell stimmt nicht mehr

Die klassische Analyse unterstellt, dass jeder Speicherzugriff gleich viel kostet. Diese Annahme war bis etwa 1985 vertretbar. Heute liegt zwischen dem schnellsten und dem langsamsten Zugriff ein Faktor von mehreren hundert.

Ort	Takte	maßstäblich
Register	0–1	eine Sekunde
L1-Cache	4	vier Sekunden
L2-Cache	14	fünfzehn Sekunden
L3-Cache	50	eine Minute
Hauptspeicher	250	**vier Minuten**
SSD	10^5	**ein Tag**

Die rechte Spalte rechnet alles so um, als dauerte ein Registerzugriff eine Sekunde. Wer in dieser Skala denkt, versteht sofort, warum ein Programm, das ständig in den Hauptspeicher greift, langsam ist, auch wenn es wenig rechnet.

Was dieser Band nachholt

Band 0 hat einen Rechner vorgestellt, der einen Befehl nach dem anderen ausführt: holen, dekodieren, ausführen, zurückschreiben. Ich habe dort ausdrücklich gesagt, dass reale Prozessoren das seit dreißig Jahren nicht mehr tun, und auf diesen Band verwiesen.

Hier ist die Liste dessen, was tatsächlich geschieht:

Fließband	Mehrere Befehle sind gleichzeitig in verschiedenen Bearbeitungsstufen (Kap. 3)
Sprungvorhersage	Die Maschine rät den Ausgang jeder Verzweigung, bevor sie ihn kennt (Kap. 5)
Umordnung	Befehle werden in anderer Reihenfolge ausgeführt, als sie im Programm stehen (Kap. 6)
Mehrfachausführung	Vier bis acht Befehle je Takt gleichzeitig
Zwischenspeicher	Drei bis vier Ebenen zwischen Prozessor und Speicher (Kap. 7)
Mehrere Kerne	Acht bis über hundert unabhängige Rechenwerke (Kap. 11)

> *Der Befehlssatz verschweigt das alles. Für ein Programm sieht die Maschine so aus, als arbeite sie einen Befehl nach dem anderen ab.*

Diese Fiktion aufrechtzuerhalten, während im Inneren etwas ganz anderes geschieht, ist die eigentliche Leistung der Rechnerarchitektur. Sie gelingt fast immer. Kapitel 14 handelt von den Fällen, in denen sie misslingt.

Was dieses Buch will

Erstens: Die Zahlen liefern.

Über Caches lässt sich viel Allgemeines sagen, und das meiste davon hilft nicht weiter. Was hilft, sind konkrete Größen: wie viele Bytes eine Cachezeile hat, wie viele Takte ein Fehlgriff kostet, wie groß die Assoziativität ist. Ich gebe diese Zahlen an, sage aber jedes Mal dazu, für welche Maschine sie gelten und wie schnell sie veralten.

Zweitens: Vom Programm her denken.

> *Dieser Band ist nicht dafür gedacht, dass Sie einen Prozessor bauen. Er ist dafür gedacht, dass Sie verstehen, warum Ihr Programm sich verhält, wie es sich verhält.*

Jedes Kapitel endet deshalb mit der Frage, was der behandelte Mechanismus für jemanden bedeutet, der Code schreibt. Manchmal lautet die Antwort: nichts. Auch das sage ich dann.

Drittens: Ehrlich über das Messen sein.

Kapitel 15 behandelt ausführlich, warum Leistungsmessungen so oft in die Irre führen. Eine Untersuchung von 2009 zeigte, dass allein die Länge der Umgebungsvariablen die gemessene Laufzeit eines Programms um mehrere Prozent verändern kann, weil sich die Ausrichtung des Stapels im Speicher verschiebt. Wer solche Effekte nicht kennt, misst regelmäßig Unsinn.

Drei Warnungen

Erstens: Die Zahlen altern schnell.
Taktraten, Cachegrößen und Strukturbreiten ändern sich alle paar Jahre. Ich beziehe mich auf den Stand von August 2026 und sage es dazu. Die Prinzipien dahinter sind seit den 1980er Jahren stabil.
Zweitens: Optimieren ohne Messen ist Zeitverschwendung.

> *Praktisch jede Vorhersage darüber, wo ein Programm seine Zeit verbringt, ist falsch. Das gilt auch für erfahrene Leute und auch für mich.*

Der einzige verlässliche Weg führt über Messwerkzeuge, und Kapitel 8 zeigt, welche es gibt und wie man sie liest.
Drittens: Die meisten Programme sind nicht rechenbegrenzt.
Ein Programm, das auf eine Datenbank oder auf das Netz wartet, wird von keiner der hier behandelten Techniken schneller. Der Stoff dieses Bandes zahlt sich in einem schmalen, aber wichtigen Bereich aus: bei numerischer Rechnung, bei Datenverarbeitung im großen Maßstab, bei Übersetzern, bei Bild- und Signalverarbeitung, bei Datenbankkernen.

Zum Aufbau

- **Teil I** (Kap. 1–2): Der Datenpfad eines einfachen Prozessors und die Rolle des Befehlssatzes als Schnittstelle.
- **Teil II** (Kap. 3–6): Das Fließband, seine Konflikte, die Sprungvorhersage und die Umordnung von Befehlen.
- **Teil III** (Kap. 7–9): Speicherhierarchie, cachebewusstes Programmieren und virtueller Speicher.
- **Teil IV** (Kap. 10–12): Vektorbefehle, Mehrkernsysteme mit ihren Speichermodellen, Grafikprozessoren und Spezialbausteine.
- **Teil V** (Kap. 13–15): Energie als begrenzende Größe, Sicherheitslücken in der Mikroarchitektur, und der Stand im August 2026.

Die Wand, die 1995 vorhergesagt wurde

Im März 1995 erschien in der Zeitschrift *Computer Architecture News* ein vierseitiger Aufsatz von William Wulf und Sally McKee. Er trug den Titel „Hitting the Memory Wall: Implications of the Obvious“.
Die beiden rechneten etwas vor, das damals jeder wusste und niemand zu Ende gedacht hatte. Prozessoren wurden jährlich um etwa 50 Prozent schneller, Speicher um etwa 7 Prozent. Beide Wachstumsraten waren exponentiell, aber verschieden steil.

Jahr	Takte je Speicherzugriff	Bemerkung
1985	≈ 1	kein Cache nötig
1995	≈ 20	Caches werden wichtig
2005	≈ 200	
2026	≈ 250	Stagnation seit 2005

Ihre Schlussfolgerung war unbequem. Wenn die Trefferrate des Caches auch nur ein wenig unter hundert Prozent liegt, so wird die durchschnittliche Zugriffszeit irgendwann von den Fehlgriffen beherrscht, gleichgültig wie schnell der Prozessor ist.

> *Sie schrieben, das Ende sei absehbar, und man solle jetzt darüber nachdenken statt in fünf Jahren. Der Aufsatz schließt mit dem Satz, sie hofften, sich zu irren.*

Sie irrten sich nicht in der Sache, aber im Zeitpunkt. Die Wand kam später als befürchtet, weil die Prozessoren ab 2005 aufhörten, schneller zu werden (Kapitel 13). Das Verhältnis stagnierte, weil sich beide Seiten gleich langsam bewegten.
Die praktische Folge sieht man in jedem heutigen Prozessor. Etwa die Hälfte der Chipfläche ist Cache, nicht Rechenwerk. Man baut hauptsächlich Speicher und legt ein paar Rechenwerke daneben.

Sechzig Jahre Rechnerarchitektur

1945	von Neumann	gespeichertes Programm
1961	Atlas	**virtueller Speicher, Seitenwechsel**
1962	Atlas	erste Sprungvorhersage-Ansätze
1964	IBM System/360	**Befehlssatz als stabile Schnittstelle**
1965	IBM	erster Cache (System/360 Modell 85)
1965	Moore	Verdopplungsgesetz
1967	**Tomasulo**	**dynamische Umordnung, Registerumbenennung**
1967	Amdahl	Grenze der Parallelisierung
1974	Dennard	Skalierungsgesetz
1976	Cray-1	Vektorrechner
1980	**Patterson, Ditzel**	**RISC**
1981	Stanford MIPS	Fließband ohne Verriegelung
1985	Intel 386	Seitenverwaltung im Massenmarkt
1991	Yeh, Patt	zweistufige Sprungvorhersage
1993	Pentium	superskalar im Massenmarkt
1995	**Wulf, McKee**	**„Memory Wall“**
1995	Pentium Pro	Umordnung im Massenmarkt
1996	MMX	erste SIMD-Befehle für PCs

2001	POWER4	erster Zweikernchip
2005	—	**Taktrate stagniert; Wende zu Mehrkern**
2006	Berkeley View	sieben Muster paralleler Rechnung
2007	CUDA	Grafikprozessoren allgemein programmierbar
2008	Hill, Marty	Amdahl im Mehrkernzeitalter
2011	Esmaeilzadeh u. a.	**Dark Silicon**
2011	RISC-V	offener Befehlssatz
2016	Google	TPU im Rechenzentrum
2018	**Spectre, Meltdown**	**Lücken in der Mikroarchitektur**
2019	Hennessy, Patterson	„A New Golden Age“
2020	Apple M1	ARM verdrängt x86 im Arbeitsplatzrechner
2026	—	Chiplets, Spezialbausteine, Speicher nah am Rechenwerk

Zwei Zeilen sind für diesen Band besonders wichtig.
1967, Tomasulo. Robert Tomasulo entwarf für das IBM System/360 Modell 91 einen Mechanismus, der Gleitkommabefehle in beliebiger Reihenfolge ausführt und die Abhängigkeiten dabei selbst auflöst. Der Chip verkaufte sich schlecht, und die Idee verschwand für fast dreißig Jahre. Seit dem Pentium Pro von 1995 steckt sie in jedem Hochleistungsprozessor.
2005. In diesem Jahr hörte etwas auf, das seit den 1970er Jahren gegolten hatte. Die Taktraten stiegen nicht mehr, weil die Chips sonst zu heiß geworden wären. Statt schnellerer Prozessoren gab es von da an mehr davon, und die Last, damit umzugehen, wanderte zu den Programmierern.

Es gibt keinen Grund, warum irgendjemand einen Computer zu Hause haben wollte.

— Ken Olsen, Gründer von DEC, 1977.
Er meinte damit Rechner zur Haussteuerung, nicht Personalcomputer. Das Zitat wird seit fünfzig Jahren falsch verstanden und ist trotzdem lehrreich: Vorhersagen über Rechnerarchitektur altern schnell.

Part I

Der Datenpfad

1 Wie ein Prozessor aufgebaut ist

Die fünf Stufen

Band 0 hat den Befehlszyklus in vier Schritten beschrieben. Für alles Weitere ist eine feinere Einteilung nützlich, und sie hat sich seit den 1980er Jahren als Standard durchgesetzt.

H	**Holen**	Befehl aus dem Speicher lesen, Befehlszähler erhöhen
D	**Dekodieren**	Befehl auseinandernehmen, Registerinhalte lesen
A	**Ausführen**	Rechenwerk arbeiten lassen, Adresse berechnen
S	**Speicher**	bei Lade- und Schreibbefehlen zugreifen
R	**Rückschreiben**	Ergebnis ins Zielregister



ein Befehl, fünf Takte

Nicht jeder Befehl braucht alle fünf Stufen. Eine Addition zwischen Registern hat in der Speicherstufe nichts zu tun, ein Sprungbefehl schreibt nichts zurück. Man lässt die Befehle trotzdem alle Stufen durchlaufen, weil das die Steuerung erheblich vereinfacht.

Die Zahl fünf ist eine Konvention

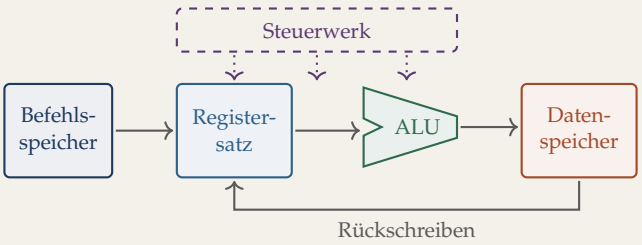
Sie stammt vom MIPS-Prozessor der frühen 1980er Jahre und hat sich in der Lehre gehalten, weil sie übersichtlich ist. Reale Prozessoren haben andere Zahlen.

Prozessor	Stufen	Jahr
MIPS R2000	5	1985
Pentium	5	1993
Pentium III	10	1999
Pentium 4 (Prescott)	**31**	2004
Core 2	14	2006
heutige x86-Kerne	15–20	2026

Die dritte Spalte erzählt eine Geschichte. Der Pentium 4 wurde bewusst auf eine sehr lange Pipeline ausgelegt, um hohe Taktraten zu erreichen. Das funktionierte auch, bis die Verlustleistung dem ein Ende setzte. Seither sind die Pipelines wieder kürzer geworden. Kapitel 3 erklärt, warum lange Pipelines teuer sind.

Der Datenpfad

Der Datenpfad ist die Menge aller Bausteine, durch die Daten fließen. Das Steuerwerk sagt ihnen, was sie tun sollen.



Zwei getrennte Speicher für Befehle und Daten sind hier nur eingezeichnet, damit das Bild einfach bleibt. Reale Rechner haben einen gemeinsamen Speicher, aber getrennte Caches der ersten Ebene.

Man nennt das die **Harvard-Architektur**, wenn die Trennung durchgängig ist, und **modifizierte Harvard-Architektur**, wenn sie nur bei den Caches besteht. Der zweite Fall ist heute der Normalfall. Er verbindet den Vorteil der Trennung, nämlich zwei gleichzeitige Zugriffe je Takt, mit dem Vorteil des gemeinsamen Speichers, nämlich der Möglichkeit, Programme zu laden und zu verändern.

Warum es überhaupt einen Registersatz gibt

Man könnte alle Operanden direkt aus dem Speicher holen. Frühe Rechner haben genau das getan, und einige Befehlssätze erlauben es bis heute.

Ort	Zugriff	Anzahl
Register	unter einem Takt	16 bis 32
L1-Cache	vier Takte	einige tausend Wörter
Hauptspeicher	250 Takte	Milliarden

Register sind schnell, weil es wenige gibt. Ein Registersatz mit 32 Einträgen lässt sich als reine Schaltlogik bauen, die in einem Bruchteil eines Takts antwortet. Bei tausend Einträgen ginge das nicht mehr, weil allein die Auswahllogik zu tief würde.

> *Die Zahl 32 ist ein Kompromiss. Mehr Register bedeuten weniger Speicherzugriffe, aber längere Registernummern in den Befehlen und mehr Aufwand beim Umschalten zwischen Programmen.*

ARM64 hat 31 allgemeine Register, RISC-V ebenfalls 31 plus ein festverdrahtetes Nullregister, x86-64 nur 16. Die letzte Zahl ist ein Erbe aus dem Jahr 1978 und einer der Gründe, warum x86-Code mehr Speicherzugriffe braucht als ARM-Code.

Aufgabe 1.1.

(a) Verfolgen Sie einen Befehl vollständig durch den Datenpfad und bestimmen Sie die Taktzeit.

(b) Vergleichen Sie Einzeltakt-, Mehrtakt- und Fließbandausführung rechnerisch.

(c) Untersuchen Sie, wie die Zahl der Register die Laufzeit beeinflusst.

Lösung Schritt für Schritt.

(a) Ein Befehl durch den Datenpfad.
Wir verfolgen `LW r5, 8(r6)`, also „lade das Wort an der Adresse $r6 + 8$ nach $r5$“.

Stufe	Was geschieht	Ergebnis
H	Befehlsspeicher an Adresse `PC` lesen; `PC := PC+4`	32-Bit-Befehlswort
D	Opcode auswerten; $r6$ aus dem Registersatz lesen; Konstante 8 vorzeichenrichtig erweitern	Wert von $r6$, Zahl 8
A	ALU addiert	Adresse $r6 + 8$
S	Datenspeicher an dieser Adresse lesen	geladenes Wort
R	Wort nach $r5$ schreiben	

Die Taktzeit im Einzeltaktentwurf.
Bei einer Maschine, die jeden Befehl in genau einem Takt abarbeitet, muss der Takt so lang sein wie der längste Weg durch den Datenpfad. Mit typischen Verzögerungen:

Baustein	Verzögerung	bei `LW` benutzt
Befehlsspeicher	200 ps	ja
Registersatz lesen	100 ps	ja
ALU	200 ps	ja
Datenspeicher	200 ps	ja
Registersatz schreiben	100 ps	ja
Summe	**800 ps**	

`LW` ist der längste Befehl, also bestimmt er die Taktzeit:

$$T_{\text{Takt}} = 800\ \text{ps} \Longrightarrow f = 1{,}25\ \text{GHz}.$$

Der Einzeltaktentwurf verschwendet Zeit

Eine Addition zwischen Registern braucht nur 600 ps, ein Sprung sogar nur 500. Beide bekommen trotzdem 800, weil der Takt für alle gleich lang ist.

Befehl	Bedarf	bekommt	Verlust
`LW`	800 ps	800 ps	0
`SW`	700 ps	800 ps	100
`ADD`	600 ps	800 ps	200
`BEQ`	500 ps	800 ps	300

Bei einer üblichen Befehlsmischung mit etwa 25 Prozent Ladebefehlen und 45 Prozent Rechenbefehlen liegt der mittlere Bedarf bei ungefähr 675 ps. Der Einzeltaktentwurf verschenkt also rund ein Sechstel der Zeit.

> *Genau daraus entstand der Mehrtaktentwurf: Man teilt den Befehl in Stufen auf und gibt jedem Befehl nur so viele Takte, wie er braucht.*

(b) Drei Entwürfe im Vergleich.
Wir rechnen mit einer Befehlsmischung, wie sie für Programme dieser Art typisch ist:

Art	Anteil	Takte (mehrt.)	Beitrag
Laden	25 %	5	1,25
Speichern	10 %	4	0,40
Rechnen	45 %	4	1,80
Verzweigen	20 %	3	0,60
		im Mittel	**4,05**

Entwurf 1 – Einzeltakt.

$$T_{\text{Takt}} = 800\ \text{ps}, \qquad \text{CPI} = 1 \Longrightarrow \text{Zeit je Befehl} = 800\ \text{ps}.$$

Entwurf 2 – Mehrtakt.
Jede Stufe bekommt einen eigenen Takt, und der Takt richtet sich nach der längsten Einzelstufe, also nach 200 ps.

$$T_{\text{Takt}} = 200\ \text{ps}, \qquad \text{CPI} = 4{,}05 \Longrightarrow 810\ \text{ps}.$$

Das ist kein Fortschritt. Der Mehrtaktentwurf gewinnt beim Takt, was er bei der Befehlsdauer verliert.

Sein Vorteil liegt woanders: Er braucht weniger Hardware, weil dieselbe ALU in verschiedenen Takten für verschiedene Zwecke benutzt werden kann. Bis in die 1980er Jahre war das ausschlaggebend, weil Transistoren teuer waren.

Entwurf 3 – Fließband.
Man behält die fünf Stufen bei, lässt aber fünf Befehle gleichzeitig arbeiten, jeden in einer anderen Stufe.

$$T_{\text{Takt}} = 200\ \text{ps} + 50\ \text{ps (Register zwischen den Stufen)} = 250\ \text{ps},$$

$$\text{CPI} \to 1 \text{ im eingeschwungenen Zustand} \Longrightarrow 250\ \text{ps}.$$

Entwurf	Takt	CPI	je Befehl
Einzeltakt	800 ps	1,00	800 ps
Mehrtakt	200 ps	4,05	810 ps
Fließband	250 ps	$\approx 1{,}0$	**250 ps**

Der Faktor beträgt hier 3,2, nicht 5. Zwei Gründe dafür: Die Zwischenregister kosten 50 ps je Takt, und die Stufen sind nicht gleich lang. Kapitel 3 führt das aus.

(c) Der Einfluss der Registerzahl.
Der Übersetzer muss die Werte, die ein Programm gerade braucht, auf die vorhandenen Register verteilen. Reichen sie nicht, so muss er Werte in den Speicher auslagern und später zurückholen. Man nennt das **Spilling**.

Schritt 1 – Wie oft das vorkommt.
Messungen an übersetztem C-Code ergeben ungefähr das folgende Bild:

Register	Spill-Anteil	Bemerkung
8	$\approx 12\,\%$	x86 (32 Bit)
16	$\approx 4\,\%$	x86-64
32	$\approx 1\,\%$	ARM64, RISC-V
64	$\approx 0{,}7\,\%$	kaum noch Gewinn

Schritt 2 – Was das kostet.
Ein ausgelagerter Wert liegt fast immer im L1-Cache, kostet also etwa vier Takte statt null. Bei einem Anteil von 4 Prozent zusätzlicher Speicherzugriffe:

$$\text{Mehraufwand} \approx 0{,}04 \cdot 4 = 0{,}16 \text{ Takte je Befehl.}$$

Bei einem CPI von etwa 1 sind das rund 16 Prozent. Der Sprung von 8 auf 16 Register bringt entsprechend mehr.
Schritt 3 – Warum man trotzdem nicht 256 Register baut.

Befehlslänge	Die Registernummer braucht $\log_2 R$ Bit, und jeder Befehl enthält bis zu drei davon. Bei 256 Registern sind das 24 Bit allein für die Nummern
Zugriffszeit	Der Auswahlbaum wird tiefer; ab etwa 128 Einträgen passt der Zugriff nicht mehr in einen Takt
Kontextwechsel	Beim Umschalten zwischen Programmen müssen alle Register gerettet werden
Abnehmender Ertrag	Über 32 hinaus findet der Übersetzer kaum noch Werte, die er dauerhaft halten will

Die vierte Zeile ist die entscheidende. Ein typischer Programmabschnitt hält gleichzeitig zwischen fünf und fünfzehn Werte am Leben. Alles darüber steht ungenutzt herum.

Was moderne Prozessoren stattdessen tun

Es gibt einen Ausweg, der beide Vorteile verbindet: viele physische Register bei wenigen sichtbaren.
Ein heutiger x86-Kern hat 16 architektonisch sichtbare Ganzzahlregister und intern zwischen 180 und 300 physische. Die Zuordnung wechselt bei jedem Befehl. Man nennt das **Registerumbenennung**, und Kapitel 6 entwickelt sie ausführlich.

> *Der Zweck ist allerdings ein anderer als das Vermeiden von Auslagerungen. Es geht darum, Scheinabhängigkeiten aufzulösen, die entstehen, wenn zwei unabhängige Rechnungen zufällig dasselbe Register benutzen.*

Der Befehlssatz bleibt dabei unverändert. Das ist ein Musterbeispiel dafür, wie eine Schnittstelle stabil bleiben kann, während sich darunter alles ändert (Band 0, Kapitel 12).

Patterson und Hennessy[19] entwickeln den Datenpfad Schritt für Schritt und in RISC-V; Harris und Harris[7] tun dasselbe mit mehr Digitaltechnik; Nisan und Schocken[18] lassen den Leser alles selbst bauen.

2 Der Befehlssatz als Vertrag

Was zugesichert wird und was nicht

Der Befehlssatz ist die einzige Schicht dieses Bandes, die sich nicht ändern darf. Alles darunter wird alle paar Jahre neu gebaut, alles darüber verlässt sich darauf.

Zugesichert	Welche Befehle es gibt, was sie bewirken, wie viele Register sichtbar sind, wie Speicher adressiert wird
Nicht zugesichert	Wie lange etwas dauert, in welcher Reihenfolge intern gerechnet wird, wie viele Rechenwerke es gibt, wie die Caches aussehen

Die zweite Zeile ist der Grund, warum derselbe Maschinencode auf einem Prozessor von 1995 und einem von 2026 läuft, obwohl beide innen nichts gemeinsam haben.

Sie ist zugleich der Grund für die Sicherheitslücken aus Kapitel 14. Was nicht zugesichert ist, wird auch nicht geprüft, und Angreifer können es beobachten.

Die Erfindung der stabilen Schnittstelle

Vor 1964 war jeder Rechner ein eigenes Gerät mit eigenem Befehlssatz. Wer von einem IBM 7090 auf ein neueres Modell wechselte, musste seine Programme neu schreiben.
IBM kündigte am 7. April 1964 die System/360 an. Das Besondere war nicht ein einzelner Rechner, sondern eine *Familie*: sechs Modelle mit stark unterschiedlicher Leistung und Preis, aber demselben Befehlssatz.

> *Ein Programm, das auf dem kleinsten Modell lief, lief unverändert auf dem größten. Innen waren die Maschinen völlig verschieden gebaut, teilweise mit Mikroprogrammen, teilweise fest verdrahtet.*

Das Vorhaben kostete IBM etwa fünf Milliarden Dollar, mehr als das Manhattan-Projekt und ein Vielfaches des damaligen Jahresumsatzes. THOMAS WATSON JUNIOR nannte es später die riskanteste Entscheidung seines Lebens.
Sie ging auf. Die 360 wurde die erfolgreichste Rechnerfamilie ihrer Zeit, und ihr Befehlssatz existiert in erweiterter Form bis heute in den z-Systemen. Code aus dem Jahr 1964 läuft darauf noch immer.

> *Die dabei erfundene Denkfigur, nämlich die Trennung von Architektur und Implementierung, ist wichtiger geworden als die Maschine selbst. Der Begriff* ***computer architecture*** *in seiner heutigen Bedeutung stammt aus diesem Projekt.*

GENE AMDAHL, GERRIT BLAAUW und FREDERICK BROOKS beschrieben 1964 in einem Aufsatz, was sie unter Architektur verstanden: die Beschreibung der Maschine, wie sie ein Programmierer sieht, im Unterschied zu der, wie sie ein Ingenieur baut.

RISC und CISC, dreißig Jahre später

Band 0 hat den Gegensatz eingeführt. Hier die Fortsetzung.

	x86-64	ARM64 / RISC-V
Befehlslänge	1–15 Byte	4 Byte (fest)
Sichtbare Register	16	31
Speicherzugriff	in fast jedem Befehl	nur Laden/Speichern
Dekodierung	aufwendig	einfach
Codedichte	gut	etwa 10–20 % größer

Die letzte Zeile war lange das Hauptargument für CISC. Dichterer Code passt besser in den Befehlscache. Seit dieser groß genug ist, wiegt das Argument weniger.

Wie x86 heute funktioniert.

Ein moderner x86-Kern übersetzt jeden Befehl beim Dekodieren in eine oder mehrere **Mikrooperationen**, die intern RISC-artig aussehen.

x86-Befehl	Mikrooperationen	Bemerkung
`ADD rax, rbx`	1	
`ADD rax, [rbx]`	2	Laden + Addieren
`ADD [rbx], rax`	3	Laden, Addieren, Speichern
`PUSH rax`	2	
`DIV rbx`	≈ 36	

> *Der Aufwand für diese Übersetzung ist der eigentliche Preis von x86. Er kostet Chipfläche, Verlustleistung und eine oder zwei Pipelinestufen.*

Um ihn zu mildern, haben x86-Kerne seit 2008 einen **Mikrooperationscache**, der die bereits übersetzten Formen aufbewahrt. Bei Schleifen entfällt die Dekodierung dann ganz.

Warum die Frage RISC gegen CISC heute anders gestellt wird

Die interessanten Unterschiede liegen inzwischen woanders.

Dekodierbreite	Bei fester Befehlslänge kann man acht Befehle gleichzeitig dekodieren, weil man weiß, wo sie anfangen. Bei variabler Länge ist das viel schwerer
Speichermodell	x86 gibt stärkere Garantien über die Reihenfolge von Speicherzugriffen als ARM (Kap. 11)
Lizenz	x86 gehört zwei Firmen, ARM wird lizenziert, RISC-V ist frei

Die erste Zeile ist ein handfester technischer Vorteil von ARM, und Apples M-Prozessoren haben ihn ab 2020 ausgespielt. Ihre Dekodierbreite von acht war mit x86 zu dieser Zeit nicht erreichbar.

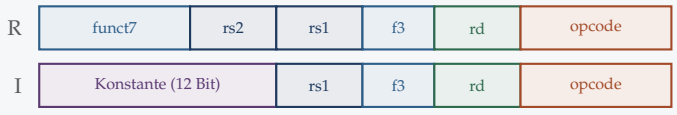
Die dritte Zeile erklärt den Aufstieg von RISC-V. Ein Befehlssatz ist eine Schnittstelle, und Schnittstellen ohne Eigentümer haben sich in der Informatik regelmäßig durchgesetzt.

Aufgabe 2.1.

(a) Untersuchen Sie die Befehlskodierung von RISC-V und rechnen Sie die Sprungweiten aus.
(b) Vergleichen Sie die Codegröße derselben Funktion in x86-64 und ARM64.
(c) Diskutieren Sie, welche Zusicherungen ein Befehlssatz über Zeit machen sollte.

Lösung Schritt für Schritt.

(a) Kodierung in RISC-V.
RISC-V benutzt sechs Befehlsformate mit einheitlicher Länge von 32 Bit. Zwei davon:



Auffällig ist, dass `rs1`, `rs2` und `rd` in beiden Formaten an denselben Bitpositionen stehen. Das ist Absicht: Der Registersatz kann ausgelesen werden, bevor der Befehl vollständig dekodiert ist.

Dieselbe Überlegung erklärt, warum die Konstanten in RISC-V so eigenartig zerstückelt sind. Das Vorzeichenbit steht in jedem Format an Bit 31, damit die Vorzeichenerweiterung ohne Dekodierung beginnen kann.

Die Sprungweiten.
Ein bedingter Sprung (Format B) hat 12 Bit für die Weite, und weil Befehle an geraden Adressen liegen, wird das unterste Bit weggelassen:

$$\text{Bereich} = \pm 2^{12}\ \text{Byte} = \pm 4096\ \text{Byte} = \pm 1024\ \text{Befehle}.$$

Ein unbedingter Sprung (Format J) hat 20 Bit:

$$\pm 2^{20}\ \text{Byte} = \pm 1\ \text{MiB}.$$

Was bei größeren Weiten geschieht.
Der Assembler löst das mit zwei Befehlen auf:

`AUIPC t0, hi20`	obere 20 Bit relativ zum Befehlszähler
`JALR ra, t0, lo12`	untere 12 Bit dazu, dann springen

$$\text{Reichweite} = \pm 2^{31}\ \text{Byte} = \pm 2\ \text{GiB}.$$

Zwei Befehle statt einem, und das nur bei sehr weiten Sprüngen. Der Übersetzer weiß meist, welcher Fall vorliegt, und der Binder korrigiert notfalls nach.

Ein Zahlenbeispiel.
Der Sprung steht an Adresse $(\texttt{1000})_{16}$, das Ziel an $(\texttt{0F80})_{16}$.

$$\text{Weite} = (\texttt{0F80})_{16} - (\texttt{1000})_{16} = 3968 - 4096 = -128.$$

Das passt bequem in 12 Bit mit Vorzeichen, denn der Bereich reicht von -4096 bis $+4094$. ✓

(b) Codegröße im Vergleich.
Wir übersetzen dieselbe Funktion:

```
int summe(int *a, int n) { int s=0; for (int i=0;i<n;i++)
                  s+=a[i]; return s; }
```

ARM64.

Befehl	Bytes	Kommentar
`mov w8, wzr`	4	$s := 0$
`cmp w1, #0`	4	
`b.le ende`	4	
`mov x9, xzr`	4	$i := 0$
`schleife:`		
`ldr w10, [x0, x9, lsl #2]`	4	`a[i]`
`add w8, w8, w10`	4	
`add x9, x9, #1`	4	
`cmp x9, x1`	4	
`b.lt schleife`	4	
`ende: mov w0, w8`	4	
`ret`	4	
11 Befehle	**44**	

Bemerkenswert ist die fünfte Zeile. Der Befehl `ldr w10, [x0, x9, lsl #2]` berechnet die Adresse $x0 + 4 \cdot x9$ und lädt in einem Zug. ARM hat also durchaus zusammengesetzte Adressierungsarten.

x86-64.

Befehl	Bytes	Kommentar
`xor eax, eax`	2	$s := 0$
`test esi, esi`	2	
`jle ende`	2	
`xor ecx, ecx`	2	$i := 0$
`schleife:`		
`add eax, [rdi+rcx*4]`	3	**Laden und Addieren**
`inc rcx`	3	
`cmp rcx, rsi`	3	
`jl schleife`	2	
`ende: ret`	1	
9 Befehle	**20**	

Der Vergleich.

	Befehle	Bytes	Schleifenrumpf
ARM64	11	44	5 Befehle, 20 B
x86-64	9	20	4 Befehle, 11 B
Verhältnis	1,22	**2,2**	

Der x86-Code ist hier deutlich kürzer, weil `add eax, [rdi+rcx*4]` zwei Dinge in drei Bytes erledigt. Bei anderen Funktionen fällt der Vergleich weniger deutlich aus; über größere Programme gemittelt liegt der Unterschied bei etwa 10 bis 20 Prozent.

Kürzerer Code heißt nicht schnellerer Code

Der x86-Befehl `add eax, [rdi+rcx*4]` zerfällt intern in zwei Mikrooperationen. Gezählt in Mikrooperationen sind beide Schleifenrümpfe gleich lang.

	Befehle	Mikrooperationen
ARM64	5	5
x86-64	4	5

Die Codegröße wirkt sich vor allem auf den Befehlscache aus. Bei kleinen Schleifen, die ohnehin hineinpassen, spielt sie keine Rolle. Bei großen Programmen mit vielen selten benutzten Pfaden schon.

> *ARM hat auf dieses Problem mit dem Thumb-Befehlssatz reagiert, der 16-Bit-Kodierungen für die häufigsten Befehle anbietet. RISC-V macht dasselbe mit der Erweiterung „C“ und erreicht damit ungefähr die Codedichte von x86.*

(c) Welche Zusicherungen über Zeit sinnvoll wären.
Ein Befehlssatz sagt heute nichts über Ausführungszeiten. Das war lange unproblematisch und ist es seit 2018 nicht mehr.
Das Problem.

> *Kryptographische Programme müssen unabhängig von geheimen Daten immer gleich lange laufen. Sonst kann ein Angreifer aus der Laufzeit auf den Schlüssel schließen.*

Es gibt aber keine Zusicherung, dass etwa eine Multiplikation für alle Operanden gleich lange braucht. Auf manchen Prozessoren ist sie bei kleinen Zahlen schneller.

Ganzzahlmultiplikation	Auf einigen ARM-Kernen datenabhängig
Division	Fast überall datenabhängig
Gleitkomma	Subnormale Zahlen sind auf x86 oft 100-mal langsamer
Speicherzugriff	Immer datenabhängig über den Cache

Die Antwort der Hersteller.
Seit etwa 2020 gibt es Betriebsarten, die genau das zusichern:

Name	Hersteller	Wirkung
`DIT`	ARM	datenunabhängige Zeit für viele Befehle
`DOITM`	Intel	ähnlich
Zkt-Erweiterung	RISC-V	Liste konstantzeitiger Befehle

> *Das ist eine bemerkenswerte Entwicklung: Zum ersten Mal seit 1964 wird der Befehlssatz um Aussagen über* Zeit *erweitert. Die Schnittstelle, die genau das immer verschwiegen hat, muss darüber jetzt reden.*

Der Grund liegt darin, dass Sicherheit sich nicht auf eine Ebene beschränken lässt. Wer garantieren will, dass ein Programm nichts verrät, braucht Garantien von jeder Schicht, auf der etwas beobachtbar ist. Kapitel 14 führt das aus. Patterson und Hennessy[19] behandeln RISC-V vollständig; die offizielle Beschreibung[25] ist frei verfügbar und lesbar; Fog[6] misst seit Jahrzehnten die tatsächlichen Befehlszeiten aller gängigen Prozessoren und stellt die Tabellen frei zur Verfügung.

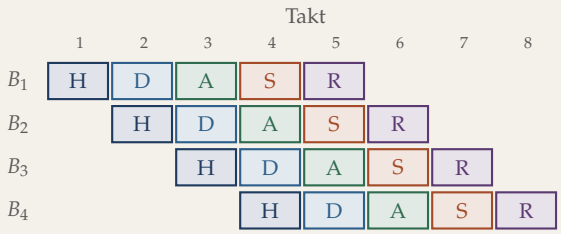
Part II

Das Fließband

3 Fließbandverarbeitung

Das Prinzip

Ein Fließband erhöht nicht die Geschwindigkeit, mit der ein einzelner Befehl fertig wird. Es erhöht die Zahl der Befehle, die je Zeiteinheit fertig werden.



Nach dem Anlaufen verlässt in jedem Takt ein Befehl das Fließband. Die Zeit für einen einzelnen Befehl bleibt bei fünf Takten, oder wird sogar etwas länger, weil zwischen den Stufen Register liegen, die Zeit kosten.

	Latenz	**Durchsatz**
ohne Fließband	800 ps	1,25 Mrd./s
mit Fließband	1250 ps	4 **Mrd./s**

> *Der einzelne Befehl wird also langsamer, und trotzdem ist die Maschine dreimal schneller. Das ist die zentrale Unterscheidung dieses Kapitels.*

Man sieht dasselbe an jeder Waschküche. Wer eine Ladung Wäsche waschen, trocknen und falten muss, braucht dafür immer gleich lange. Wer vier Ladungen hat und die Geräte überlappend benutzt, ist deutlich schneller fertig.

Die Formel für die Beschleunigung

Sei T die Gesamtverzögerung des ungetakteten Datenpfads, k die Zahl der Stufen und r die Verzögerung eines Zwischenregisters.

$$T_{\text{Takt}} = \frac{T}{k} + r \qquad \text{(bei gleich langen Stufen).}$$

Für n Befehle braucht das Fließband $k + n - 1$ Takte, also

$$T_{\text{ges}} = (k + n - 1)\left(\frac{T}{k} + r\right).$$

Für große n überwiegt der erste Summand nicht mehr, und die Beschleunigung gegenüber dem Einzeltaktentwurf beträgt

$$S = \frac{T}{\frac{T}{k} + r} = \frac{k}{1 + \frac{kr}{T}}.$$

k	T_{Takt}	S	**Wirkungsgrad**
1	800 ps	1,0	100 %
5	210 ps	3,8	76 %
10	130 ps	6,2	62 %
20	90 ps	8,9	44 %
40	70 ps	11,4	29 %

(gerechnet mit $T = 800$ ps und $r = 10$ ps)

> *Die Registerverzögerung r setzt der Sache eine Grenze. Für $k \to \infty$ strebt die Beschleunigung gegen $T/r = 80$, aber der Weg dorthin wird immer teurer.*

In der Praxis liegt die Grenze viel früher, und zwar wegen der Konflikte aus Kapitel 4.

Warum der Pentium 4 scheiterte

Intel entwarf um 2000 eine Architektur, die konsequent auf hohe Taktraten ausgelegt war. Der Name „NetBurst“ war Programm: viele kurze Stufen, hohe Frequenz.

Kern	Stufen	Takt	Jahr
Pentium III	10	1,0 GHz	2000
Willamette	20	1,5 GHz	2000
Prescott	**31**	3,8 GHz	2004
Core 2	14	2,9 GHz	2006

Die Rechnung ging zunächst auf. Bei gleicher Fertigungstechnik erreichte der Prescott fast die doppelte Taktrate des Vorgängers. Zwei Dinge machten den Gewinn zunichte.

Strafe bei Fehlvorhersage	Bei 31 Stufen müssen nach einem falsch geratenen Sprung rund 30 Takte Arbeit verworfen werden (Kap. 5)
Verlustleistung	Sie wächst überproportional mit der Taktrate; der Prescott gab über 100 Watt ab und ließ sich nicht mehr kühlen

Intel stellte das Projekt 2005 ein. Ein geplantes Modell mit 4 GHz wurde abgesagt, und die Nachfolgearchitektur Core stammte von der viel kürzeren Pipeline des Pentium III ab.

> *Der Core 2 von 2006 hatte weniger Takt als der Pentium 4 von 2004 und war trotzdem erheblich schneller. Das war für die Öffentlichkeit schwer zu verstehen, weil die Gigahertzzahl jahrelang als Maß für Leistung gegolten hatte.*

Seither ist die Pipelinetiefe bei etwa 15 bis 20 Stufen stehengeblieben. Kapitel 13 behandelt die physikalische Seite dieses Endes.

Aufgabe 3.1.

(a) Rechnen Sie Durchsatz und Latenz für verschiedene Pipelinetiefen aus.
(b) Bestimmen Sie die optimale Stufenzahl bei gegebener Fehlvorhersagestrafe.
(c) Untersuchen Sie, wie ungleich lange Stufen die Rechnung verändern.

Lösung Schritt für Schritt.

(a) Durchsatz und Latenz.
Gegeben: $T = 800$ ps ungetaktete Verzögerung, $r = 20$ ps je Zwischenregister.
Schritt 1 – Die Taktzeit.

k	T/k	T_{Takt}	f
1	800	800 ps	1,25 GHz
2	400	420 ps	2,38 GHz
5	160	180 ps	5,56 GHz
10	80	100 ps	10,0 GHz
20	40	60 ps	16,7 GHz

Schritt 2 – Latenz eines einzelnen Befehls.

$$L = k \cdot T_{\text{Takt}} = k\left(\frac{T}{k} + r\right) = T + k\,r.$$

k	L	**Zuwachs**
1	800 ps	—
5	900 ps	+12,5 %
10	1000 ps	+25 %
20	1200 ps	+50 %

Die Latenz wächst linear mit der Stufenzahl, der Durchsatz aber nicht. Genau darin liegt der Kompromiss.

Schritt 3 – Wann die Latenz zählt.
Bei einer langen Rechnung ohne Abhängigkeiten spielt sie keine Rolle. Bei einer Kette abhängiger Operationen bestimmt sie alles.

Kette von n abhängigen Additionen: $n \cdot L$ statt $n \cdot T_{\text{Takt}}$.

Das ist der Grund, warum Programme, die eine lange Abhängigkeitskette bilden, auf tief gestaffelten Prozessoren schlecht laufen. Ein Beispiel dafür ist die Summation einer Zahlenreihe in eine einzige Variable. Kapitel 10 zeigt, wie man solche Ketten aufbricht.

(b) Die optimale Stufenzahl.
Wir berücksichtigen jetzt, dass ein Teil der Befehle Sprünge sind und dass eine Fehlvorhersage die Pipeline leert.
Das Modell.

b	Anteil der Sprungbefehle, hier 0,20
m	Anteil der Fehlvorhersagen, hier 0,05
Strafe	etwa k Takte, denn die halbe bis ganze Pipeline muss geleert werden

$$\text{CPI} = 1 + b \cdot m \cdot k.$$

$$\text{Zeit je Befehl} = \text{CPI} \cdot T_{\text{Takt}} = \big(1 + b\,m\,k\big)\Big(\frac{T}{k} + r\Big).$$

Minimieren.
Wir setzen $c := b\,m = 0{,}01$ und leiten nach k ab:

$$f(k) = \Big(1 + ck\Big)\Big(\frac{T}{k} + r\Big) = \frac{T}{k} + r + cT + c\,r\,k.$$

$$f'(k) = -\frac{T}{k^2} + c\,r = 0 \Longrightarrow k_{\text{opt}} = \sqrt{\frac{T}{c\,r}}.$$

Einsetzen.

$$k_{\text{opt}} = \sqrt{\frac{800}{0{,}01 \cdot 20}} = \sqrt{4000} = 63{,}2.$$

Das Modell empfiehlt also 63 Stufen, was offenkundig unsinnig ist. Reale Prozessoren haben 15 bis 20. Das Modell muss zu grob sein.

Was fehlt.

Verlustleistung	Sie wächst etwa mit f^3, wenn man die nötige Spannungserhöhung mitrechnet. Bei 63 Stufen wäre der Chip nicht kühlbar
Datenkonflikte	Auch ohne Sprünge kosten Abhängigkeiten Wartetakte, und zwar mehr bei tieferer Pipeline (Kap. 4)
Cachefehlgriffe	Sie kosten in Takten gerechnet mehr, wenn der Takt kürzer ist
Ungleiche Stufen	Man kann den Datenpfad nicht beliebig fein teilen; siehe Teil (c)

Nimmt man die Verlustleistung dazu, verschiebt sich das Optimum auf etwa 10 bis 20 Stufen. Eine Untersuchung von HARTSTEIN und PUZAK kam 2002 auf ungefähr $k = 25$ für reine Leistung und auf $k \approx 8$ bis 10, wenn man Leistung je Watt optimiert.

Die tatsächliche Entwicklung folgte genau dieser Rechnung. Solange Leistung zählte, wurden die Pipelines tiefer. Als ab 2004 die Energie zählte, wurden sie wieder kürzer.

(c) Ungleich lange Stufen.
Bisher haben wir angenommen, der Datenpfad ließe sich in k gleich lange Stücke schneiden. Das stimmt nicht.
Die tatsächlichen Verzögerungen aus Kapitel 1:

Stufe	Dauer	Anteil
Holen	200 ps	25 %
Dekodieren	100 ps	12,5 %
Ausführen	200 ps	25 %
Speicher	200 ps	25 %
Rückschreiben	100 ps	12,5 %

Der Takt richtet sich nach der längsten Stufe.

$$T_{\text{Takt}} = 200 + 20 = 220 \text{ ps}, \qquad S = \frac{800}{220} = 3{,}64.$$

Bei gleich langen Stufen wären es

$$T_{\text{Takt}} = 160 + 20 = 180 \text{ ps}, \qquad S = 4{,}44.$$

Die Ungleichheit kostet also etwa 18 Prozent. Man verliert nicht nur die Differenz, sondern zahlt sie in jedem Takt für jede Stufe.

Was man dagegen tut.

Aufteilen	Die langen Stufen werden weiter zerschnitten. Aus „Speicher“ werden zwei oder drei Stufen
Zusammenlegen	Kurze Stufen werden verschmolzen. Dekodieren und Registerlesen liegen oft in einer Stufe
Ungleiche Wege	Verschiedene Befehlsarten durchlaufen verschieden viele Stufen

Die dritte Zeile ist bei Gleitkommabefehlen der Normalfall:

Operation	Latenz	Durchsatz
Ganzzahl-Addition	1 Takt	1/Takt
Ganzzahl-Multiplikation	3 Takte	1/Takt
Gleitkomma-Addition	4 Takte	2/Takt
Gleitkomma-Multipl.	4 Takte	2/Takt
Gleitkomma-Division	14 Takte	1 je 4 Takte
Quadratwurzel	18 Takte	1 je 6 Takte

(Größenordnungen für einen x86-Kern der Jahre um 2025)

Latenz und Durchsatz sind verschiedene Zahlen

Die letzten beiden Zeilen der Tabelle zeigen, dass die Division nicht vollständig im Fließband läuft. Man kann alle vier Takte eine neue starten, aber nicht jeden Takt.

> *Für die Praxis heißt das: Eine Schleife mit vielen unabhängigen Divisionen ist durch den Durchsatz begrenzt, eine Kette abhängiger Divisionen durch die Latenz.*

Ein Zahlenbeispiel. Hundert unabhängige Divisionen brauchen

$$14 + 99 \cdot 4 = 410 \text{ Takte},$$

hundert abhängige dagegen

$$100 \cdot 14 = 1400 \text{ Takte}.$$

Derselbe Code, dreifacher Unterschied, allein durch die Abhängigkeitsstruktur. AGNER FOG misst diese Zahlen seit über zwanzig Jahren für alle gängigen Prozessoren und stellt die Tabellen frei zur Verfügung[6]. Sie sind bei ernsthafter Optimierung unentbehrlich, weil die Hersteller sie oft nicht vollständig dokumentieren.

Patterson und Hennessy[19] entwickeln das Fließband ausführlich; Hennessy und Patterson[8] behandeln die quantitative Seite; Bryant und O'Hallaron[3] zeigen, wie sich das im Programm bemerkbar macht.

4 Konflikte im Fließband

Drei Arten von Ärger

Das Fließband aus Kapitel 3 funktioniert nur, solange die Befehle nichts miteinander zu tun haben. Sobald sie einander brauchen, geht es nicht mehr glatt.

Art	Ursache	Abhilfe
Struktur	Zwei Befehle wollen dieselbe Hardware	mehr Hardware
Daten	Ein Befehl braucht das Ergebnis eines früheren	Weiterleitung, Warten
Steuerung	Nach einem Sprung ist unklar, welcher Befehl folgt	Vorhersage (Kap. 5)

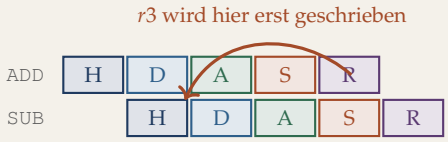
Strukturkonflikte sind die harmloseste Sorte, weil man sie durch Bauen beseitigen kann. Getrennte Caches für Befehle und Daten lösen den häufigsten Fall, nämlich dass in einem Takt sowohl ein Befehl geholt als auch ein Datum geladen werden soll.

Datenkonflikte und Weiterleitung

Betrachten Sie diese beiden Befehle:

```
ADD r3, r1, r2
SUB r5, r3, r4
```

Der zweite braucht *r*3, das der erste erst in Stufe R schreibt. Ohne Gegenmaßnahmen sieht das so aus:



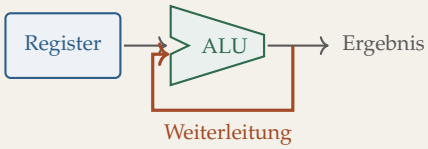
Der `SUB` liest in seiner Dekodierstufe, also im Takt 3. Geschrieben wird aber erst im Takt 5. Zwei Takte fehlen.

Die naive Lösung: warten.

Man hält den zweiten Befehl zwei Takte an und schiebt Leerbefehle ins Fließband. Das kostet, wie die Rechnung in Aufgabe (a) zeigt, sehr viel.

Die bessere Lösung: weiterleiten.

Das Ergebnis der Addition liegt schon am Ende von Takt 3 am Ausgang der ALU vor. Man muss es nur nicht erst ins Register schreiben und wieder herauslesen, sondern direkt zurückführen.



*Man nennt das **Forwarding** oder **Bypassing**. Der Wert nimmt eine Abkürzung und steht schon im nächsten Takt zur Verfügung.*

Damit lassen sich alle Konflikte zwischen Rechenbefehlen vollständig beseitigen. Ein Fall bleibt übrig.

Der Ladebefehl lässt sich nicht wegleiten

```
LW r3, 0(r1)
ADD r5, r3, r4
```

Der geladene Wert steht erst am Ende der Speicherstufe fest, also im Takt 4. Der `ADD` braucht ihn aber schon zu Beginn seiner Ausführungsstufe, ebenfalls im Takt 4. Die Weiterleitung müsste rückwärts in der Zeit laufen.

> *Ein Wartetakt ist hier unvermeidlich. Man nennt das die **Ladeverzögerung**, und sie ist der Grund, warum Übersetzer versuchen, nach einem Ladebefehl einen unabhängigen Befehl einzuschieben.*

Frühe RISC-Prozessoren machten daraus eine Eigenschaft des Befehlssatzes. Der MIPS von 1985 führte den Befehl nach einem Ladebefehl *immer* aus, auch wenn der Wert noch nicht da war. Der Übersetzer musste dort etwas Sinnvolles hinschreiben oder einen Leerbefehl. Das hieß **load delay slot**.

> *Der Name MIPS stand ursprünglich für „Microprocessor without Interlocked Pipeline Stages“, also für einen Prozessor, der solche Konflikte gar nicht erst in Hardware behandelt.*

Man hat das später aufgegeben, weil es den Befehlssatz an eine bestimmte Pipelinetiefe bindet. Sobald eine neue Generation eine andere Zahl von Stufen hat, stimmt der Code nicht mehr. Das ist genau die Art von Zusicherung, die eine Schnittstelle nicht machen sollte (Kapitel 2).

Aufgabe 4.1.

(a) Bestimmen Sie den CPI-Verlust durch Datenkonflikte mit und ohne Weiterleitung.
(b) Ordnen Sie eine Befehlsfolge so um, dass keine Wartetakte entstehen.
(c) Untersuchen Sie Steuerkonflikte und die Kosten einer Fehlvorhersage.

Lösung Schritt für Schritt.

(a) Der CPI-Verlust.
Schritt 1 – Die Häufigkeit.
In übersetztem Code hängt ungefähr die Hälfte aller Befehle vom unmittelbaren Vorgänger ab, weil Rechnungen typischerweise als Ketten geschrieben werden.

Abstand	Anteil	Wartetakte ohne Weiterl.
direkter Vorgänger	$\approx 45\,\%$	2
zwei Befehle vorher	$\approx 15\,\%$	1
drei oder weiter	$\approx 10\,\%$	0
keine Abhängigkeit	$\approx 30\,\%$	0

Schritt 2 – Ohne Weiterleitung.

$$\mathrm{CPI} = 1 + 0{,}45 \cdot 2 + 0{,}15 \cdot 1 = 1 + 0{,}90 + 0{,}15 = 2{,}05.$$

Die Maschine ist also nur halb so schnell wie erhofft. Von der Beschleunigung durch das Fließband bleibt weniger als die Hälfte übrig.

Schritt 3 – Mit Weiterleitung.
Alle Konflikte zwischen Rechenbefehlen verschwinden. Übrig bleibt nur der Ladefall.

Größe	Wert	Quelle
Anteil Ladebefehle	25 %	Befehlsmischung
davon direkt benutzt	$\approx 50\,\%$	Messung
Strafe	1 Takt	
Beitrag	**0,125**	$0{,}25 \cdot 0{,}5 \cdot 1$

$$\mathrm{CPI} = 1 + 0{,}125 = 1{,}125.$$

Der Vergleich.

Ausführung	CPI	relative Leistung
ohne Weiterleitung	2,05	1,00
mit Weiterleitung	1,125	**1,82**
ideal	1,00	2,05

Die Weiterleitung kostet ein paar Multiplexer und ein wenig Steuerlogik und bringt fast die Verdopplung. Sie gehört zu den lohnendsten Bauteilen überhaupt und steckt in jedem Prozessor seit den 1960er Jahren.

(b) Befehle umordnen.
Gegeben ist der übersetzte Code für `a = b + c; d = e - f;`

```
1    LW t1, 0(b)
2    LW t2, 0(c)
3    ADD t3, t1, t2     wartet auf 2
4    SW t3, 0(a)
5    LW t4, 0(e)
6    LW t5, 0(f)
7    SUB t6, t4, t5     wartet auf 6
8    SW t6, 0(d)
```

Die Wartetakte im Original.
Befehl 3 folgt unmittelbar auf 2 und braucht dessen Ergebnis. Das ist der Ladefall, also ein Wartetakt. Dasselbe bei 7 nach 6.

$$\text{Takte} = 8 + 2 = 10 \quad \text{(Wartetakte)}, \qquad \text{CPI} = 1{,}25.$$

Die Umordnung.
Man zieht einen unabhängigen Befehl in die Lücke.

```
1    LW t1, 0(b)
2    LW t2, 0(c)
3    LW t4, 0(e)        vorgezogen, füllt die Lücke
4    ADD t3, t1, t2     kein Warten mehr
5    LW t5, 0(f)
6    SW t3, 0(a)        füllt die zweite Lücke
7    SUB t6, t4, t5     kein Warten mehr
8    SW t6, 0(d)
```

$$\text{Takte} = 8, \qquad \text{CPI} = 1{,}00.$$

*Dieselben acht Befehle, dieselben Ergebnisse, zwanzig Prozent schneller. Der Übersetzer macht das automatisch; man nennt es **Befehlsanordnung** oder **scheduling**.*

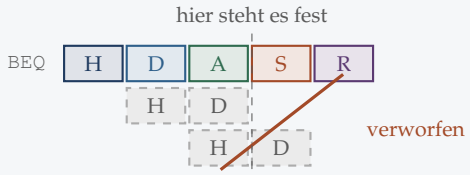
Warum das nicht immer geht.

Zu wenig Befehle	In einem kurzen Grundblock gibt es keine unabhängigen Befehle zum Vorziehen
Zeigermehrdeutigkeit	Der Übersetzer weiß oft nicht, ob `a` und `e` auf dieselbe Speicherstelle zeigen. Dann darf er nichts vertauschen
Registermangel	Vorziehen bedeutet, einen Wert länger zu halten, und dafür braucht man ein Register

Die zweite Zeile ist in C ein ernstes Problem und der Grund für das Schlüsselwort `restrict`. Damit sichert der Programmierer zu, dass zwei Zeiger nicht auf dasselbe zeigen, und der Übersetzer darf umordnen.

> *In Fortran ist diese Zusicherung Teil der Sprache. Das ist einer der Gründe, warum numerischer Fortran-Code über Jahrzehnte schneller war als der entsprechende C-Code.*

(c) Steuerkonflikte.
Das Problem.
Bei einem bedingten Sprung steht erst in der Ausführungsstufe fest, ob gesprungen wird. Bis dahin hat die Maschine schon zwei oder drei weitere Befehle geholt.



Schritt 1 – Die Kosten ohne Vorhersage.
Bei fünf Stufen und Auflösung in Stufe A sind zwei Befehle zu verwerfen.

$$\mathrm{CPI} = 1 + b \cdot 2 = 1 + 0{,}20 \cdot 2 = 1{,}40.$$

Bei einer Pipeline von 20 Stufen, bei der die Auflösung in Stufe 15 erfolgt:

$$\mathrm{CPI} = 1 + 0{,}20 \cdot 14 = 3{,}80.$$

> *Ohne Sprungvorhersage wäre eine tiefe Pipeline vollkommen sinnlos. Sie ist die Voraussetzung dafür, dass sich Fließbandverarbeitung überhaupt lohnt.*

Schritt 2 – Die einfachen Gegenmaßnahmen.

Verfahren	Trefferquote	Bemerkung
immer „nicht springen“	$\approx 40\,\%$	einfachste Lösung
immer „springen“	$\approx 60\,\%$	Schleifen dominieren
rückwärts springen, vorwärts nicht	$\approx 65\,\%$	statische Heuristik
Zweibitzähler	$\approx 90\,\%$	Kap. 5
moderne Verfahren	$> 98\,\%$	Kap. 5

Die dritte Zeile beruht auf einer einfachen Beobachtung: Ein Rückwärtssprung ist meist das Ende einer Schleife und wird deshalb oft genommen. Ein Vorwärtssprung überspringt meist einen Sonderfall und wird selten genommen.

Schritt 3 – Was 98 Prozent bedeuten.

$$\mathrm{CPI} = 1 + 0{,}20 \cdot 0{,}02 \cdot 14 = 1 + 0{,}056 = 1{,}056.$$

Trefferquote	CPI	Leistung
0 %	3,80	1,00
60 %	2,12	1,79
90 %	1,28	2,97
98 %	1,056	**3,60**
99,5 %	1,014	3,75

Die letzten Prozent sind die teuersten und die wichtigsten

Von 90 auf 98 Prozent gewinnt man 21 Prozent Leistung. Von 98 auf 99,5 nur noch 4 Prozent.

> *Trotzdem investiert man erheblichen Aufwand in diese letzten Prozente, weil bei tiefen Pipelines und breiter Ausführung jede Fehlvorhersage sehr viel Arbeit vernichtet.*

Bei einem Prozessor, der vier Befehle je Takt ausführt und 15 Takte bis zur Auflösung braucht, kostet eine Fehlvorhersage bis zu 60 verworfene Befehle. Kapitel 5 zeigt, mit welchen Mitteln man auf 98 Prozent kommt, und Kapitel 14, welche Sicherheitslücke daraus entstanden ist.

Patterson und Hennessy[19] behandeln Weiterleitung und Konflikte im Detail; Hennessy und Patterson[8] geben die Messwerte; Fog[6] dokumentiert, welche Weiterleitungen auf welchem Prozessor tatsächlich vorhanden sind.

5 Sprungvorhersage

Was vorherzusagen ist

Bei jedem Sprungbefehl muss die Maschine zwei Dinge raten, und sie muss beides tun, bevor sie den Befehl überhaupt dekodiert hat.

Frage	Vorhersagewerkzeug	Schwierigkeit
Wird gesprungen?	Richtungsvorhersage	**das eigentliche Problem**
Wohin?	Sprungzielpuffer	meist leicht

Die zweite Frage ist einfacher, weil das Ziel bei den meisten Sprüngen fest im Befehl steht. Man merkt es sich beim ersten Durchlauf in einer kleinen Tabelle, dem **Branch Target Buffer**, und schlägt es beim nächsten Mal anhand der Befehlsadresse nach.
Schwierig sind nur **indirekte** Sprünge, bei denen das Ziel in einem Register steht. Das kommt bei Rücksprüngen aus Unterprogrammen vor, bei Sprungtabellen und bei virtuellen Methodenaufrufen.

> *Für Rücksprünge gibt es eine eigene Struktur, den **Return Address Stack**. Beim Aufruf legt die Maschine die Rücksprungadresse dort ab, beim Rücksprung nimmt sie sie herunter. Die Trefferquote liegt bei über 99 Prozent, solange die Schachtelung nicht tiefer wird als der Stapel, der typischerweise 16 bis 32 Einträge hat.*

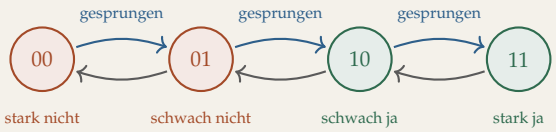
Vom Einbitzähler zum Zweibitzähler

Der einfachste Ansatz. Man merkt sich für jede Sprungadresse ein Bit: Wurde beim letzten Mal gesprungen? Man sagt dasselbe wieder vorher.
Bei einer Schleife mit n Durchläufen irrt dieses Verfahren zweimal:

Durchlauf	tatsächlich	Vorhersage
1	springt	nicht (falsch)
$2 \ldots n-1$	springt	springt ✓
n	springt nicht	springt (falsch)

Bei einer geschachtelten Schleife wiederholt sich das bei jedem Durchlauf der äußeren. Eine innere Schleife mit 10 Durchläufen, hundertmal ausgeführt, ergibt 200 Fehlvorhersagen bei 1000 Sprüngen, also 80 Prozent Trefferquote.

Der Zweibitzähler.
Man gibt der Vorhersage ein Beharrungsvermögen. Ein Zähler zählt von 0 bis 3 und wird bei jedem genommenen Sprung erhöht, bei jedem nicht genommenen verringert. Vorhergesagt wird „springt“, wenn der Zähler bei 2 oder 3 steht.



Bei derselben geschachtelten Schleife irrt der Zähler nur noch einmal je Durchlauf der äußeren Schleife, nämlich beim Verlassen der inneren. Beim Wiedereintritt steht er noch auf „schwach ja“ und sagt richtig vorher.

Verfahren	Trefferquote	Speicher
statisch	$\approx 65\,\%$	keiner
Einbitzähler	$\approx 80\,\%$	1 Bit je Eintrag
Zweibitzähler	$\approx 90\,\%$	2 Bit je Eintrag

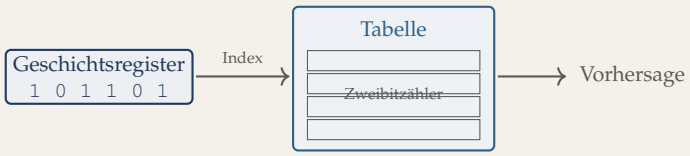
Korrelierende Vorhersage

Der Zweibitzähler betrachtet jeden Sprung für sich. Damit übersieht er, dass Sprünge oft voneinander abhängen.

```
if (a == 0) ...     /* Sprung 1 */
if (b == 0) ...     /* Sprung 2 */
if (a == 0 && b == 0) ...     /* Sprung
3 */
```

Der Ausgang des dritten Sprungs folgt zwingend aus den ersten beiden. Ein Vorhersager, der sich die jüngste Sprunggeschichte merkt, kann ihn mit hundert Prozent Genauigkeit voraussagen.

TSE-YU YEH und YALE PATT beschrieben 1991 ein zweistufiges Verfahren, das genau das ausnutzt.



Das Geschichtsregister enthält die Ausgänge der letzten h Sprünge. Zusammen mit der Sprungadresse bildet es den Index in eine Tabelle von Zweibitzählern.

h	**Tabellengröße**	**Trefferquote**
0	2^k	$\approx 90\,\%$
4	$16 \cdot 2^k$	$\approx 93\,\%$
8	$256 \cdot 2^k$	$\approx 95\,\%$
12	$4096 \cdot 2^k$	$\approx 96\,\%$

Die Tabelle wächst exponentiell mit der Geschichtslänge

Bei einer Geschichte von 20 Sprüngen bräuchte man eine Million Einträge je Sprungadresse. Das ist nicht baubar.

> *Zugleich wäre eine lange Geschichte oft nützlich, weil manche Sprünge von weit zurückliegenden Entscheidungen abhängen. Die Aufgabe lautet also, lange Geschichten mit wenig Speicher zu nutzen.*

Die Antwort darauf ist der **TAGE**-Vorhersager von André Seznec (2006). Er führt mehrere Tabellen mit *verschieden langen* Geschichten und benutzt jeweils diejenige, die für den fraglichen Sprung schon eine verlässliche Auskunft hat.

Tabelle	Geschichtslänge	Zweck
T_0	0	Grundvorhersage
T_1	5	kurze Muster
T_2	15	
T_3	44	
T_4	130	**sehr lange Abhängigkeiten**

Jeder Eintrag trägt eine Markierung, mit der sich prüfen lässt, ob er wirklich zu diesem Sprung und dieser Geschichte gehört. Genommen wird die Auskunft der Tabelle mit der *längsten* passenden Geschichte.

> *TAGE erreicht auf Standardmessreihen über 98 Prozent und ist seit etwa 2015 in fast allen Hochleistungsprozessoren die Grundlage. Die Hersteller nennen ihre Varianten meist nicht beim Namen, aber die Struktur ist erkennbar.*

Seznec hat mit Varianten dieses Verfahrens seit 2004 praktisch jeden internationalen Vorhersagewettbewerb gewonnen.

Aufgabe 5.1.

(a) Verfolgen Sie einen Zweibitzähler durch eine geschachtelte Schleife.
(b) Rechnen Sie die Kosten der Fehlvorhersage für einen breiten Prozessor aus.
(c) Untersuchen Sie, was ein Programmierer beeinflussen kann und was nicht.

Lösung Schritt für Schritt.

(a) Der Zweibitzähler in einer Schleife.
Gegeben ist eine innere Schleife mit vier Durchläufen, die dreimal ausgeführt wird.
Der Sprungverlauf. Mit `J` für „gesprungen“ und `N` für „nicht gesprungen“:

$$\underbrace{\texttt{J J J N}}_{\text{1. Ausf.}}\ \underbrace{\texttt{J J J N}}_{\text{2.}}\ \underbrace{\texttt{J J J N}}_{\text{3.}}$$

Der Einbitzähler.

Schritt	Zustand	Vorhersage	tatsächl.	Treffer?
1	N	N	J	nein
2	J	J	J	ja
3	J	J	J	ja
4	J	J	N	nein
5	N	N	J	nein
6–7	J	J	J	ja
8	J	J	N	nein

Fehler $=$ 6 von 12 $\Longrightarrow$ Trefferquote 50 %.

Der Zweibitzähler.
Startzustand sei „schwach nicht“ $=$ 01.

Schritt	Zähler	Vorhersage	tatsächl.	Treffer?
1	01	N	J	nein
2	10	J	J	ja
3	11	J	J	ja
4	11	J	N	nein
5	10	J	J	ja
6	11	J	J	ja
7	11	J	J	ja
8	11	J	N	nein
9	10	J	J	ja
10–11	11	J	J	ja
12	11	J	N	nein

$$\text{Fehler} = 4 \text{ von } 12 \Longrightarrow \text{Trefferquote } 67\,\%.$$

Der Zweibitzähler irrt nur noch beim Verlassen der Schleife, nicht mehr beim Wiedereintritt. Er hat aus dem einmaligen Ausreißer gelernt, dass die Ausnahme eine Ausnahme ist.

Und ein korrelierender Vorhersager?
Mit einer Geschichte von drei Sprüngen sieht der Vorhersager das Muster `JJJ` und weiß, dass danach `N` kommt.

$$\text{Trefferquote nach der Einlernphase: } 100\,\%.$$

*Schleifen mit fester, kurzer Durchlaufzahl werden von modernen Vorhersagern vollständig erfasst. Manche Prozessoren haben dafür sogar eine eigene Struktur, den **Loop Predictor**, der die Durchlaufzahl zählt.*

(b) Die Kosten bei einem breiten Prozessor.
Die Größen.

Größe	Wert	Bemerkung
Breite	6 Befehle/Takt	heutiger Hochleistungskern
Tiefe bis Auflösung	17 Takte	
Sprunganteil	20 %	
Trefferquote	97 %	

Schritt 1 – Wie viel Arbeit eine Fehlvorhersage vernichtet.

$$6 \cdot 17 = 102 \text{ Befehlsplätze.}$$

In der Praxis ist die Pipeline nicht ganz gefüllt, sagen wir zu 60 Prozent. Dann sind es etwa 60 Befehle.
Schritt 2 – Der CPI-Beitrag.

$$\Delta\text{CPI} = 0{,}20 \cdot 0{,}03 \cdot 17 = 0{,}102.$$

Bei einem Grund-CPI von 0,25 (also vier Befehlen je Takt im Mittel):

$$\text{CPI} = 0{,}25 + 0{,}102 = 0{,}352.$$

$$\text{Leistungsverlust} = 1 - \frac{0{,}25}{0{,}352} = 29\,\%.$$

Drei Prozent Fehlvorhersagen kosten also fast dreißig Prozent Leistung. Diese Verstärkung erklärt, warum in die Vorhersage so viel Aufwand fließt.

Schritt 3 – Die Verbesserung auf 99 Prozent.

$$\Delta\mathrm{CPI} = 0{,}20 \cdot 0{,}01 \cdot 17 = 0{,}034, \qquad \mathrm{CPI} = 0{,}284.$$

$$\text{Gewinn gegenüber 97 Prozent} = \frac{0{,}352}{0{,}284} - 1 = 24\,\%.$$

Zwei Prozentpunkte bessere Vorhersage bringen ein Viertel mehr Leistung. Ein Vorhersager darf entsprechend groß sein; in heutigen Kernen belegt er zwischen 30 und 100 Kilobyte.

(c) Was ein Programmierer tun kann.
Wirksam.

Sprung vermeiden	Bedingte Zuweisung statt Verzweigung, wo möglich
Sortieren	Wenn Daten sortiert sind, wird eine datenabhängige Bedingung vorhersagbar
Schleifen abrollen	Weniger Sprünge insgesamt
Seltene Fälle auslagern	Den Ausnahmepfad in eine eigene Funktion, damit der Hauptpfad geradeaus läuft

Ein Beispiel zur zweiten Zeile.
Die folgende Schleife summiert alle Werte über einer Schwelle:

```
for (i=0;i<n;i++) if (a[i] >= 128) s += a[i];
```

Bei zufälligen Daten ist die Bedingung nicht vorhersagbar, die Trefferquote liegt bei 50 Prozent. Sortiert man das Feld vorher, so ist sie fast immer vorhersagbar.

Daten	Zeit	Verhältnis
unsortiert	11,5 s	1
sortiert	1,9 s	**6,1**

Dieselbe Schleife, dieselben Werte, nur in anderer Reihenfolge. Der Faktor sechs entsteht vollständig durch die Sprungvorhersage. Das Sortieren selbst kostet natürlich Zeit, aber wenn man die Schleife oft ausführt, lohnt es sich.

Die sprunglose Fassung.
Man kann die Bedingung auch ohne Sprung ausdrücken:

```
s += a[i] * (a[i] >= 128);
```

oder mit einem bedingten Bewegungsbefehl, den der Übersetzer meist selbst einsetzt. Dann spielt die Sortierung keine Rolle mehr, und die Zeit liegt in beiden Fällen bei etwa 2,5 Sekunden.

Sprunglos ist nicht immer besser

Eine bedingte Zuweisung führt *beide* Zweige aus und wählt hinterher. Sie kostet also immer die volle Arbeit.

Gut vorhersagbar	Der Sprung ist billiger, weil der nicht genommene Zweig gar nicht ausgeführt wird
Schlecht vorhersagbar	Die bedingte Zuweisung ist billiger, weil es keine Fehlvorhersage geben kann

Die Grenze liegt ungefähr dort, wo die Fehlvorhersagequote das Verhältnis der Zweigkosten erreicht. Bei gleich teuren Zweigen und einer Strafe von 17 Takten lohnt sich die sprunglose Fassung ab etwa 10 Prozent Fehlvorhersagen.

> *Übersetzer versuchen, das selbst zu entscheiden, und liegen dabei oft daneben, weil sie die Datenverteilung nicht kennen. Wer es besser weiß, kann mit profilgeführter Übersetzung nachhelfen: Man lässt das Programm einmal mit echten Daten laufen und übersetzt dann erneut.*

Nicht wirksam.
Alle Versuche, dem Vorhersager direkt etwas mitzuteilen, sind heute wirkungslos. Intel führte 2000 Präfixe ein, mit denen man einen Sprung als „wahrscheinlich genommen“ kennzeichnen konnte. Sie werden seit dem Pentium 4 ignoriert, weil der eingebaute Vorhersager praktisch immer besser rät als der Programmierer. Yeh und Patt[28] beschreiben das zweistufige Verfahren; Seznec[20] hat TAGE über zwanzig Jahre entwickelt und die Arbeiten frei verfügbar gemacht; Hennessy und Patterson[8] geben die Messwerte für reale Programme.

6 Mehrere Befehle gleichzeitig

Zwei Wege, mehr zu tun

Ein Fließband erreicht bestenfalls einen Befehl je Takt. Um darüber hinauszukommen, gibt es zwei Möglichkeiten.

	VLIW	Superskalar
Wer entscheidet	der Übersetzer	**die Hardware**
Befehlsformat	lange Wörter mit festen Plätzen	gewöhnlich
Aufwand im Chip	gering	**erheblich**
Kompatibilität	bricht bei neuer Breite	bleibt
Beispiele	Itanium, TI-Signalprozessoren	alles Übrige

Der VLIW-Ansatz sieht aufgeräumter aus und ist in der Praxis fast überall gescheitert. Der Grund liegt darin, dass der Übersetzer die Ausführungszeiten nicht kennt. Ob ein Ladebefehl vier oder dreihundert Takte braucht, hängt vom Cache ab, und das steht erst zur Laufzeit fest.

Der Itanium und eine Milliarde Dollar

Intel und HP begannen 1994 gemeinsam die Entwicklung einer neuen 64-Bit-Architektur. Sie hieß IA-64 und beruhte auf einer verfeinerten VLIW-Idee, für die Intel den Begriff **EPIC** prägte.
Der Gedanke war überzeugend. Wenn der Übersetzer die Parallelität ausdrücklich in den Code schreibt, kann die Hardware einfach bleiben und dafür breiter und schneller werden. Die aufwendige Umordnungslogik superskalarer Prozessoren entfiele.

> *Fachleute nannten das Vorhaben „Itanic“. Der erste Chip erschien 2001, vier Jahre später als geplant, und war bei gleicher Fertigungstechnik langsamer als ein gewöhnlicher x86-Prozessor.*

Die Gründe wurden im Rückblick deutlich. Der Übersetzer musste Entscheidungen treffen, für die ihm die nötigen Angaben fehlten. Er wusste nicht, welche Ladebefehle im Cache treffen würden, wie sich Sprünge verhalten würden und ob zwei Zeiger auf dieselbe Speicherstelle zeigen.

> *Ein superskalarer Prozessor weiß das alles zur Laufzeit und entscheidet entsprechend. Diese Information kann kein Übersetzer beschaffen, so gut er auch programmiert ist.*

Der Itanium wurde 2021 endgültig eingestellt. Die Kosten für Intel werden auf mehrere Milliarden Dollar geschätzt. Als Nebenfolge setzte sich AMDs Erweiterung von x86 auf 64 Bit durch, weil Intel keine Alternative anbieten wollte. Die Architektur, die heute in jedem PC steckt, heißt deshalb auch „AMD64“.

Die Lehre daraus formulierten Hennessy und Patterson so: Man soll dem Übersetzer nichts aufbürden, was er ohne Kenntnis der Laufzeit nicht entscheiden kann.

Warum Umordnung nötig ist

Betrachten Sie diese Folge:

1	`LW r1, 0(r10)`	Cachefehlgriff, 200 Takte
2	`ADD r2, r1, r3`	braucht $r1$
3	`MUL r5, r6, r7`	unabhängig
4	`SUB r8, r6, r9`	unabhängig

Eine Maschine, die die Reihenfolge einhält, steht 200 Takte still, obwohl die Befehle 3 und 4 sofort ausgeführt werden könnten. Eine Maschine, die umordnen darf, erledigt sie in der Wartezeit.

> *Das ist der eigentliche Zweck der Umordnung. Es geht weniger darum, mehr Befehle je Takt zu schaffen, als darum, die Wartezeit auf den Speicher zu überbrücken.*

Drei Arten von Abhängigkeit.

Name	Muster	Bemerkung
echt (RAW)	schreiben, dann lesen	unvermeidbar
Gegen (WAR)	lesen, dann schreiben	durch Umbenennung lösbar
Ausgabe (WAW)	zweimal schreiben	ebenso

Nur die erste ist eine echte Abhängigkeit des Rechenwegs. Die beiden anderen entstehen allein daraus, dass zwei unabhängige Rechnungen zufällig dasselbe Register benutzen.

`ADD r1, r2, r3`	
`SUB r4, r1, r5`	echt: braucht $r1$
`MUL r1, r6, r7`	Gegen- und Ausgabeabhängigkeit

Der dritte Befehl hat mit den ersten beiden inhaltlich nichts zu tun. Er darf nur nicht vorher schreiben, weil sonst der zweite den falschen Wert läse.

Registerumbenennung

Die Lösung besteht darin, den Registernamen aus dem Befehl von der tatsächlichen Speicherstelle zu trennen.

vorher	nach Umbenennung
`ADD r1, r2, r3`	`ADD p12, p7, p8`
`SUB r4, r1, r5`	`SUB p13, p12, p9`
`MUL r1, r6, r7`	`MUL p14, p10, p11`

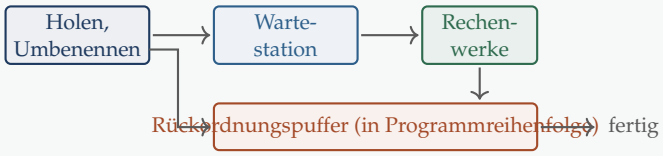
Der dritte Befehl schreibt jetzt nach $p14$ statt nach $p12$. Die Gegenabhängigkeit ist verschwunden, und er darf beliebig früh ausgeführt werden.

Eine Tabelle merkt sich, welches physische Register gerade zu welchem architektonischen gehört. Bei jedem schreibenden Befehl wird ein freies physisches Register genommen und der Eintrag aktualisiert.

Prozessor	sichtbar	physisch
Pentium Pro (1995)	8	40
Core 2 (2006)	16	96
Skylake (2015)	16	180
heutige Kerne	16–32	250–400

Der Rückordnungspuffer

Wenn Befehle in beliebiger Reihenfolge ausgeführt werden, muss ihre *Wirkung* trotzdem in Programmreihenfolge sichtbar werden. Sonst funktionierten weder Unterbrechungen noch Ausnahmen noch die Fehlerbehandlung.
Dafür sorgt der **Rückordnungspuffer** (*reorder buffer*). Er ist eine Warteschlange, in die die Befehle in Programmreihenfolge eintreten und die sie in derselben Reihenfolge verlassen.



Ein Befehl gilt erst dann als vollzogen, wenn er den Puffer verlässt, und das geschieht erst, wenn alle vor ihm ihn verlassen haben.

> *Damit lässt sich der Zustand jederzeit auf einen sauberen Punkt zurücksetzen. Bei einer falschen Sprungvorhersage verwirft man alle Einträge hinter dem Sprung, und die Maschine sieht aus, als hätte sie den falschen Pfad nie betreten.*

Fast. Kapitel 14 handelt von dem, was dabei doch zurückbleibt.

Aufgabe 6.1.

(a) Führen Sie Tomasulos Verfahren an einem Beispiel durch.
(b) Bestimmen Sie, wie groß das Befehlsfenster sein muss, um einen Cachefehlgriff zu überbrücken.
(c) Untersuchen Sie die Grenzen der Befehlsparallelität.

Lösung Schritt für Schritt.

(a) Tomasulos Verfahren.
ROBERT TOMASULO entwarf 1967 für das IBM System/360 Modell 91 einen Mechanismus, der ohne zentrale Steuerung auskommt. Jedes Rechenwerk hat eigene Warteplätze, die **Reservierungsstationen**.
Der Ablauf in drei Schritten.

Ausgeben	Der Befehl kommt in eine freie Reservierungsstation. Operanden, die schon bekannt sind, werden mitgenommen; für die anderen merkt man sich, welche Station sie liefern wird
Ausführen	Sobald alle Operanden da sind, rechnet das Werk
Verbreiten	Das Ergebnis wird über einen gemeinsamen Bus an alle wartenden Stationen geschickt

Das Beispiel.

1	`MUL f2, f4, f6`	Latenz 4 Takte
2	`ADD f8, f2, f10`	braucht Ergebnis von 1
3	`MUL f2, f12, f14`	schreibt wieder nach $f2$
4	`SUB f16, f18, f20`	unabhängig

Takt 1 – Ausgabe von Befehl 1.

Station	Befehl	Operand 1	Operand 2
`MUL1`	`MUL`	Wert $f4$	Wert $f6$

Das Register $f2$ wird mit dem Vermerk „kommt von `MUL1`“ versehen.
Takt 2 – Ausgabe von Befehl 2.

Station	Befehl	Operand 1	Operand 2
`MUL1`	`MUL`	Wert $f4$	Wert $f6$
`ADD1`	`ADD`	wartet auf `MUL1`	Wert $f10$

Takt 3 – Ausgabe von Befehl 3.
Hier geschieht das Entscheidende. Befehl 3 schreibt ebenfalls nach $f2$.

Station	Befehl	Operand 1	Operand 2
MUL1	MUL	Wert $f4$	Wert $f6$
ADD1	ADD	wartet auf MUL1	Wert $f10$
MUL2	MUL	Wert $f12$	Wert $f14$

Der Vermerk bei $f2$ wird auf MUL2 umgestellt.

> *ADD1 wartet aber weiterhin auf MUL1, nicht auf $f2$. Der Verweis zeigt auf die* Station*, nicht auf das Register. Damit ist die Ausgabeabhängigkeit vollständig aufgelöst.*

Genau das ist Tomasulos Kerngedanke: Man ersetzt Registernamen durch Verweise auf die Stelle, die den Wert erzeugen wird. Die Umbenennung geschieht dabei nebenbei.

Takt 5 – MUL1 ist fertig.

Das Ergebnis geht über den gemeinsamen Bus. ADD1 erkennt seinen Verweis, übernimmt den Wert und ist bereit. Das Register $f2$ wird *nicht* beschrieben, denn dort steht inzwischen der Vermerk auf MUL2.

Die Bilanz.

Befehl	fertig in Takt	bei strenger Reihenfolge
1 (MUL)	5	5
2 (ADD)	8	8
3 (MUL)	7	12
4 (SUB)	6	14

> *Die Befehle 3 und 4 sind vor dem Befehl 2 fertig, obwohl sie später im Programm stehen. Das Ergebnis ist trotzdem korrekt, weil die echte Abhängigkeit zwischen 1 und 2 eingehalten wird.*

Warum die Idee dreißig Jahre lag

Das Modell 91 wurde nur etwa zwanzigmal gebaut. Es war teuer, hatte keine präzisen Ausnahmen und kam zu einer Zeit, als die Speicherzugriffe noch nicht das Hauptproblem waren.

> *Tomasulos Verfahren lohnt sich erst, wenn es lange Wartezeiten zu überbrücken gibt. In den 1960er Jahren waren Speicherzugriffe nur wenige Takte lang, und die Umordnung brachte kaum etwas.*

Mit der Speicherlücke der 1990er Jahre änderte sich das. Der Pentium Pro von 1995 war der erste Massenprozessor mit Umordnung, und der fehlende Teil, nämlich die präzisen Ausnahmen, wurde durch den Rückordnungspuffer ergänzt, den SMITH und PLESZKUN 1985 beschrieben hatten.

Tomasulo erhielt 1997 den Eckert-Mauchly-Preis, dreißig Jahre nach seiner Arbeit.

(b) Wie groß muss das Befehlsfenster sein?
Die Frage.
Um einen Fehlgriff auf den Hauptspeicher zu überbrücken, muss die Maschine genug unabhängige Arbeit finden. Wie viele Befehle muss sie dafür überblicken?
Die Rechnung.

Größe	Wert	Quelle
Fehlgriffkosten	250 Takte	Hauptspeicher
Ausführungsbreite	4 Befehle/Takt	
nötige Befehle	**1000**	Produkt

Tausend Befehle müssten gleichzeitig im Fenster sein, damit die Maschine während des Wartens vollständig ausgelastet bliebe. Reale Fenster haben 200 bis 600 Einträge.

Prozessor	Fenster	Jahr
Pentium Pro	40	1995
Core 2	96	2006
Skylake	224	2015
Apple M1	≈ 630	2020
heutige Kerne	500–700	2026

Warum man nicht einfach vergrößert.
Der Aufwand wächst überproportional. Die Auswahllogik muss in jedem Takt aus allen wartenden Befehlen die bereiten heraussuchen, und die Zahl der zu prüfenden Paare wächst quadratisch mit der Fenstergröße.

> *Apple hat mit dem M1 gezeigt, dass sehr große Fenster machbar sind, wenn man genug Chipfläche und eine einfache Dekodierung hat. Der Vorsprung gegenüber x86 lag 2020 wesentlich hier.*

(c) Die Grenzen der Befehlsparallelität.
Wie viel wäre theoretisch drin?
Untersuchungen mit unbeschränkten Ressourcen, also unendlich vielen Registern, perfekter Sprungvorhersage und unbeschränktem Fenster, ergeben Werte zwischen 20 und mehreren hundert Befehlen je Takt.

Annahme	IPC	Bemerkung
alles ideal	50–150	je nach Programm
Fenster 2000	≈ 15	
+ reale Sprungvorhersage	≈ 6	
+ reale Fenstergröße	≈ 4	
tatsächlich gemessen	**1,5–3**	

> *Der größte Einzelverlust entsteht bei der Sprungvorhersage. Jede Fehlvorhersage begrenzt das Fenster faktisch auf die Befehle bis zum nächsten Sprung, und das sind im Mittel nur fünf bis sieben.*

Die praktische Folge.
Die erreichbare Befehlsparallelität stagniert seit etwa 2010. Von 1995 bis 2010 stieg der IPC von etwa 1 auf 2; seither auf etwa 3. Weitere Verbesserungen kosten überproportional viel Fläche und Energie.

Konsequenz 1	Man baut mehr Kerne statt breiterer Kerne (Kap. 11)
Konsequenz 2	Man baut Spezialbausteine für Aufgaben mit viel Datenparallelität (Kap. 12)
Konsequenz 3	Man verlagert Arbeit in Vektorbefehle, wo die Parallelität ausdrücklich im Code steht (Kap. 10)

Alle drei Wege haben denselben Grund: Die Hardware findet allein nicht genug Parallelität, also muss sie ihr mitgeteilt werden. Das ist bemerkenswert nah an der Idee, an der der Itanium gescheitert ist, nur auf einer anderen Ebene.

Der Unterschied liegt darin, dass Datenparallelität sich gut ausdrücken lässt und Befehlsparallelität nicht. Ein Programmierer weiß, dass tausend Vektorelemente unabhängig sind. Er weiß nicht, ob zwei aufeinanderfolgende Befehle voneinander abhängen, wenn Zeiger im Spiel sind. Tomasulos Originalarbeit[24] ist zwölf Seiten lang und erstaunlich klar; Hennessy und Patterson[8] behandeln die Umordnung ausführlich; Fog[6] beschreibt die Mikroarchitekturen realer Prozessoren in einem eigenen Handbuch.

Part III

Das Gedächtnis

7 Die Speicherhierarchie

Warum es Caches gibt

Man hätte gern viel Speicher und schnellen Speicher. Beides zugleich ist nicht zu haben, weil die Zugriffszeit mit der Größe wächst. Ein Speicher mit n Zellen braucht mindestens $\log n$ Ebenen an Auswahllogik, und die Leitungen werden länger.

Ebene	Größe	Zugriff	Kosten je GB
Register	1 KB	0 Takte	—
L1	32–64 KB	4 Takte	sehr hoch
L2	0,5–2 MB	14 Takte	
L3	8–96 MB	50 Takte	
DRAM	8–256 GB	250 Takte	wenige Euro
SSD	1–8 TB	10^5 Takte	Cent

(Größenordnungen für einen Arbeitsplatzrechner, Stand 2026)

Der Ausweg besteht darin, mehrere Speicher zu staffeln und darauf zu wetten, dass die meisten Zugriffe die kleinen und schnellen treffen. Diese Wette geht auf, und zwar aus zwei Gründen.

Zeitliche Lokalität	Was gerade benutzt wurde, wird bald wieder benutzt. Schleifenvariablen, Schleifenkörper, Stapelrahmen
Räumliche Lokalität	Was neben dem gerade Benutzten liegt, wird bald benutzt. Felder, Strukturen, aufeinanderfolgende Befehle

> *Beide Eigenschaften sind keine Naturgesetze, sondern empirische Beobachtungen über die Art, wie Menschen Programme schreiben. Ein Programm, das rein zufällig auf den Speicher zugreift, hat keine Lokalität, und für dieses Programm nützt die ganze Hierarchie nichts.*

Genau das war der Befund aus dem Vorwort. Die spaltenweise durchlaufene Matrix hat zeitliche, aber keine räumliche Lokalität, und deshalb kostet sie das Sechzigfache.

Wie ein Cache aufgebaut ist

Ein Cache speichert nicht einzelne Bytes, sondern **Zeilen** von meist 64 Byte. Wer ein Byte anfordert, bekommt die ganze Zeile geholt.



Versatz	Welches Byte innerhalb der Zeile. Bei 64 Byte sind das 6 Bit
Index	Welche Menge von Plätzen in Frage kommt
Marke	Wird mitgespeichert und beim Zugriff verglichen, um zu prüfen, ob es die richtige Zeile ist

Assoziativität.
Die Frage lautet, wie viele Plätze für eine bestimmte Adresse in Frage kommen.

Art	**Plätze je Adresse**	**Aufwand**
direkt abgebildet	1	ein Vergleich
n-fach mengenassoziativ	n	n Vergleiche
voll assoziativ	alle	alle Vergleiche

Ein direkt abgebildeter Cache ist schnell und billig, hat aber ein Problem: Zwei häufig benutzte Adressen, die denselben Index ergeben, verdrängen einander ständig. Man nennt das **Konfliktfehlgriffe**.

Heutige L1-Caches sind acht- bis zwölffach assoziativ, L2 und L3 bis sechzehnfach. Damit sind Konfliktfehlgriffe selten geworden, aber nicht verschwunden.

Die drei Arten von Fehlgriffen

MARK HILL schlug 1989 eine Einteilung vor, die sich durchgesetzt hat, weil sie sagt, was man dagegen tun kann.

Art	Ursache	Abhilfe
Kalt	erster Zugriff überhaupt	längere Zeilen, Vorabholen
Kapazität	Arbeitsmenge größer als der Cache	**Programm umbauen**
Konflikt	ungünstige Adressverteilung	mehr Assoziativität, Versatz

Man bestimmt die Anteile, indem man dasselbe Programm dreimal simuliert: einmal mit unendlichem Cache (dann bleiben nur kalte Fehlgriffe), einmal mit voll assoziativem Cache der richtigen Größe (dann kommen die Kapazitätsfehlgriffe dazu), einmal mit dem echten Cache.

Programm	kalt	Kapazität	Konflikt
Matrixmultipl.	2 %	**95** %	3 %
Datenbankanfrage	30 %	60 %	10 %
Übersetzer	45 %	40 %	15 %

Die erste Zeile sagt, dass bei der Matrixmultiplikation kein größerer Cache und keine bessere Assoziativität helfen. Man muss das Programm ändern, und Kapitel 8 zeigt wie.

Die mittlere Zugriffszeit

$$t_{\text{mittel}} = t_{\text{Treffer}} + p_{\text{Fehl}} \cdot t_{\text{Strafe}}.$$

Diese Formel sieht harmlos aus und hat unangenehme Folgen, weil t_{Strafe} so groß ist.

p_{Fehl}	t_{mittel}	**Verlangsamung**
0 %	4,0 Takte	1,0
1 %	6,5 Takte	1,6
5 %	16,5 Takte	4,1
10 %	29,0 Takte	7,3
50 %	129 Takte	32

(mit $t_{Treffer} = 4$ und $t_{Strafe} = 250$)

> *Fünf Prozent Fehlgriffe vervierfachen die mittlere Zugriffszeit. Bei einem Programm, das viel auf den Speicher zugreift, ist das der beherrschende Faktor, und alles andere in diesem Band spielt daneben keine Rolle.*

Bei mehreren Ebenen setzt man die Formel geschachtelt an:

$$t_{\text{mittel}} = t_{L1} + p_{L1}\big(t_{L2} + p_{L2}(t_{L3} + p_{L3} \cdot t_{\text{DRAM}})\big).$$

Mit den üblichen Werten $p_{L1} = 0{,}05$, $p_{L2} = 0{,}4$, $p_{L3} = 0{,}3$:

$$t = 4 + 0{,}05\big(14 + 0{,}4(50 + 0{,}3 \cdot 250)\big) = 4 + 0{,}05 \cdot 64 = 7{,}2 \text{ Takte}.$$

> *Die Prozentzahlen der tieferen Ebenen sind auf die dort ankommenden Zugriffe bezogen, nicht auf alle. Man nennt das die **lokale** Fehlgriffrate. Die **globale** Rate für L3 wäre hier $0{,}05 \cdot 0{,}4 \cdot 0{,}3 = 0{,}6$ Prozent.*

Die Unterscheidung ist wichtig, weil eine hohe lokale Rate bei einer tiefen Ebene harmlos sein kann. Wenn nur ein Prozent aller Zugriffe überhaupt bis L3 vordringen, dann ist eine lokale Fehlgriffrate von 50 Prozent dort völlig unbedenklich.

Aufgabe 7.1.

(a) Rechnen Sie Cachegrößen, Marken und Speicheraufwand vollständig aus.

(b) Untersuchen Sie Konfliktfehlgriffe an einem konkreten Beispiel.

(c) Analysieren Sie das Vorabholen und seine Grenzen.

Lösung Schritt für Schritt.

(a) Ein Cache im Detail.
Gegeben: 32 KB Datencache, 64 Byte je Zeile, achtfach mengenassoziativ, 48 Bit physische Adressen.
Schritt 1 – Die Zahl der Zeilen.

$$\frac{32\,768}{64} = 512 \text{ Zeilen.}$$

Schritt 2 – Die Zahl der Mengen.

$$\frac{512}{8} = 64 \text{ Mengen.}$$

Schritt 3 – Die Aufteilung der Adresse.

Feld	Bits	Rechnung
Versatz	6	$\log_2 64$
Index	6	$\log_2 64$
Marke	36	$48 - 6 - 6$

Schritt 4 – Der Verwaltungsaufwand.
Jede Zeile braucht außer den Daten noch die Marke, ein Gültigkeitsbit und ein Schmutzbit.

$$512 \cdot (36 + 1 + 1) = 512 \cdot 38 = 19\,456 \text{ Bit} = 2432 \text{ Byte.}$$

$$\frac{2432}{32\,768} = 7{,}4\,\%.$$

Ein Cache braucht also gut sieben Prozent Zusatzspeicher für die Verwaltung. Bei kürzeren Zeilen wäre es mehr, bei längeren weniger.

Schritt 5 – Warum die Zeilen nicht länger sind.

Zeile	Verwaltung	räuml. Lok.	Verschwendung
16 B	23 %	mäßig	gering
64 B	7,4 %	gut	mäßig
256 B	2,2 %	sehr gut	hoch

Lange Zeilen nutzen die räumliche Lokalität besser, holen aber bei wahlfreiem Zugriff viel Unnötiges. Bei 256 Byte und einem einzelnen gebrauchten Wort wären 97 Prozent des geholten Datenverkehrs verschwendet.

> *Die Zahl 64 hat sich seit den 1990er Jahren als Kompromiss gehalten und ist heute bei praktisch allen Prozessoren gleich. Apples M-Reihe benutzt allerdings 128 Byte, was beim Portieren von Code gelegentlich auffällt.*

(b) Konfliktfehlgriffe.
Der Fall. Ein direkt abgebildeter Cache von 4 KB mit 64-Byte-Zeilen hat 64 Mengen. Zwei Felder sollen elementweise addiert werden:

```
for (i=0;i<n;i++) c[i] = a[i] + b[i];
```

Die drei Felder liegen bei den Adressen $(10000)_{16}$, $(11000)_{16}$ und $(12000)_{16}$.
Schritt 1 – Die Indizes.
Der Index ergibt sich aus den Bits 6 bis 11, also aus $(\text{Adresse}/64) \bmod 64$.

Feld	Adresse	Adr. / 64	Index
`a[0]`	$(10000)_{16}$	1024	**0**
`b[0]`	$(11000)_{16}$	1088	**0**
`c[0]`	$(12000)_{16}$	1152	**0**

> *Alle drei Felder bilden auf dieselbe Menge ab. Bei einem direkt abgebildeten Cache gibt es dort nur einen Platz, also verdrängen sie sich in jedem Schleifendurchlauf gegenseitig.*

Schritt 2 – Der Ablauf.

Zugriff	Zustand vorher	Ergebnis
`a[0]`	leer	Fehlgriff, lädt *a*
`b[0]`	*a* drin	Fehlgriff, verdrängt *a*
`c[0]`	*b* drin	Fehlgriff, verdrängt *b*
`a[1]`	*c* drin	Fehlgriff, obwohl schon geladen

$$p_{\text{Fehl}} = 100\,\%.$$

Schritt 3 – Was ohne Konflikt zu erwarten wäre.
Eine Zeile fasst 16 Gleitkommazahlen zu vier Byte. Bei sequentiellem Durchlauf trifft man also 15 von 16 Zugriffen.

$$p_{\text{Fehl}} = \frac{1}{16} = 6{,}25\,\%.$$

Fall	p_{Fehl}	t_{mittel}
ohne Konflikt	6,25 %	19,6 Takte
mit Konflikt	100 %	254 Takte
Faktor		**13**

Schritt 4 – Drei Abhilfen.

Assoziativität	Bei zweifacher Assoziativität passen zwei der drei Felder hinein, bei vierfacher alle drei
Versatz einbauen	Die Felder um eine Zeile gegeneinander verschieben, etwa durch Auffüllen auf $n + 16$ Elemente
Felder verschmelzen	Statt drei Feldern ein Feld von Strukturen, sodass `a[i]`, `b[i]` und `c[i]` benachbart liegen

Die zweite Zeile ist ein bekannter Kunstgriff bei numerischem Code. Man deklariert Felder absichtlich etwas größer als nötig, damit die Zeilenlänge nicht die Zweierpotenz trifft.

Besonders unangenehm sind Feldgrößen, die genau Zweierpotenzen sind, weil dann Zeilen einer Matrix systematisch kollidieren. Ein Feld von 1024×1024 ist typischerweise langsamer als eines von 1025×1025, obwohl es kleiner ist.

(c) Vorabholen.
Die Idee. Wenn die Maschine erraten kann, welche Zeile als Nächstes gebraucht wird, kann sie den Ladevorgang früh beginnen und die Wartezeit verstecken.
Was die Hardware selbst erkennt.

Muster	erkannt?	Bemerkung
`a[i]`, `a[i+1]`, …	ja	einfachster Fall
Schrittweite k	ja	„stride prefetcher“
zwei Ströme parallel	ja	meist 8–32 Ströme
verkettete Liste	nein	Adresse steht in den Daten
Baum, Hashtabelle	nein	

Die letzten beiden Zeilen sind der eigentliche Grund, warum verkettete Listen in der Praxis so viel schlechter abschneiden, als die Analyse aus Band 1 erwarten lässt. Bei einem Feld holt der Prozessor voraus, bei einer Liste kann er es nicht.

Ein Zahlenbeispiel.

Struktur	Zeit je Element	Bemerkung
Feld, sequentiell	≈ 1 Takt	Vorabholen greift
Feld, Schrittweite 64 B	≈ 15 Takte	Vorabholen greift teilweise
Feld, zufällige Reihenfolge	≈ 250 Takte	kein Vorabholen
verkettete Liste	≈ 250 Takte	jeder Schritt ein Fehlgriff

Vorabholen von Hand.
Die meisten Übersetzer bieten dafür eine Anweisung, etwa `__builtin_prefetch` in GCC. Bei einer Liste, die man mehrfach durchläuft, kann man beim Bearbeiten eines Knotens den übernächsten anfordern.

```
while (p) {
  if (p->next)
__builtin_prefetch(p->next->next);
  bearbeite(p);
  p = p->next; }
```

Vorabholen von Hand geht meist schief

Es gibt drei Arten, es falsch zu machen.

Zu früh	Die Zeile wird verdrängt, bevor sie gebraucht wird. Doppelter Aufwand ohne Nutzen
Zu spät	Der Ladevorgang ist noch nicht fertig, wenn der Wert gebraucht wird. Kein Gewinn
Zu viel	Man verdrängt nützliche Daten und erzeugt Fehlgriffe, die es sonst nicht gäbe

Der richtige Abstand hängt von der Speicherlatenz und der Bearbeitungsdauer je Element ab und muss gemessen werden. Zahlen zwischen zwei und zwölf Elementen Vorlauf sind üblich.

Hinzu kommt, dass die eingebaute Erkennung meist schon gut arbeitet. Ein Vorabholbefehl kann sie sogar stören, weil er das Zugriffsmuster verändert, das sie beobachtet.

Die Regel lautet deshalb: erst messen, dann einbauen, dann wieder messen. Und im Zweifel weglassen.

Drepper[4] ist die ausführlichste frei verfügbare Darstellung zum Thema und trotz des Alters von 2007 in den Grundzügen gültig; Jacob, Ng und Wang[12] behandeln DRAM und Caches auf Bauteilebene; Hennessy und Patterson[8] liefern die Messwerte.

8 Cachebewusst programmieren

Was tatsächlich hilft

Aus Kapitel 7 folgt eine überschaubare Liste von Maßnahmen. Sie sind nach dem zu erwartenden Gewinn geordnet.

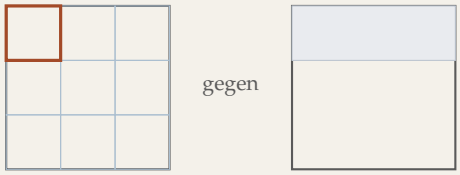
Maßnahme	Gewinn	Aufwand
Zugriffsreihenfolge anpassen	**bis 60-fach**	gering
Blockbildung	2–10-fach	mittel
Datenstruktur umbauen	2–5-fach	hoch
Feldgrößen versetzen	bis 10-fach	sehr gering
Vorabholen von Hand	1–2-fach	hoch, riskant

> *Die erste Zeile kostet fast nichts und bringt am meisten. Wer nur eine Sache aus diesem Band mitnimmt, sollte es diese sein: durchlaufe Daten in der Reihenfolge, in der sie im Speicher liegen.*

Das klingt trivial und wird ständig verletzt, weil die natürliche mathematische Formulierung oft die falsche Reihenfolge nahelegt. Bei einer Matrixmultiplikation etwa durchläuft man die zweite Matrix nach Spalten, wenn man die Definition wörtlich umsetzt.

Blockbildung

Der Kerngedanke: Man zerlegt die Arbeit in Stücke, die vollständig in den Cache passen, und erledigt jedes Stück ganz, bevor man das nächste beginnt.



Bei der Matrixmultiplikation lautet die Frage, ob die Arbeitsmenge für ein Zwischenergebnis in den Cache passt. Ohne Blockbildung braucht man drei ganze Zeilen oder Spalten, mit Blockbildung nur drei Blöcke.

Die Blockgröße wählt man so, dass drei Blöcke zusammen in den Cache passen:

$$3B^2 \cdot s \leq C \Longrightarrow B \leq \sqrt{\frac{C}{3s}},$$

wobei s die Elementgröße ist und C die Cachegröße.

Ebene	**Größe**	$s = 8$ B	**übliche Wahl**
L1	32 KB	$B \leq 37$	32
L2	1 MB	$B \leq 209$	128–256
L3	32 MB	$B \leq 1180$	selten genutzt

In der Praxis blockt man auf mehreren Ebenen zugleich, also etwa mit $B_1 = 32$ innerhalb von $B_2 = 256$. Bibliotheken wie OpenBLAS tun genau das und erreichen damit über 90 Prozent der theoretischen Rechenleistung.

Cache-oblivious: blockfrei blocken

MATTEO FRIGO, CHARLES LEISERSON und Mitarbeiter zeigten 1999, dass man die Blockgröße gar nicht kennen muss.
Der Gedanke: Man zerlegt die Aufgabe rekursiv in Hälften und hört auf, wenn sie trivial klein ist. Irgendwann sind die Teilprobleme so klein, dass sie in den Cache passen, und zwar in *jede* Ebene, sobald sie klein genug sind.

> *Ein solches Verfahren nutzt L1, L2, L3 und den Hauptspeicher gleichzeitig gut, ohne eine einzige dieser Größen zu kennen. Es passt sich automatisch an jede Maschine an.*

	cachebewusst	**cache-oblivious**
kennt Cachegröße	ja	**nein**
optimal auf einer Ebene	ja	ja
optimal auf allen Ebenen	schwierig	**automatisch**
Portierbarkeit	schlecht	gut
Spitzenleistung	etwas höher	etwa 10 % darunter

Die letzte Zeile erklärt, warum Hochleistungsbibliotheken trotzdem von Hand abgestimmt werden. Wo zehn Prozent zählen, nimmt man die Mühe in Kauf.

Die Bibliothek FFTW für schnelle Fouriertransformationen geht einen dritten Weg: Sie probiert beim ersten Aufruf verschiedene Zerlegungen durch und merkt sich die schnellste für die vorliegende Maschine.

Aufgabe 8.1.

(a) Rechnen Sie die Fehlgriffe bei der Matrixmultiplikation mit und ohne Blockbildung aus.
(b) Vergleichen Sie zwei Anordnungen derselben Daten im Speicher.
(c) Beschreiben Sie, wie man misst, wo ein Programm seine Zeit verliert.

Lösung Schritt für Schritt.

(a) Matrixmultiplikation.
Die naive Fassung.

```
for i, for j, for k:  c[i][j] += a[i][k] * b[k][j];
```

Schritt 1 – Die Zugriffsmuster.

Feld	durchlaufen nach	Lokalität
`a[i][k]`	Zeilen (k innen)	gut
`b[k][j]`	**Spalten** (k innen)	schlecht
`c[i][j]`	fest in der Schleife	sehr gut

Schritt 2 – Die Fehlgriffe zählen.
Sei n die Matrixgröße, L die Zahl der Elemente je Cachezeile (bei 64 Byte und 8-Byte-Zahlen also $L = 8$).

Feld	Fehlgriffe	Begründung
`a`	n^3 / L	sequentiell, jede L-te trifft
`b`	$\mathbf{n^3}$	**jeder Zugriff ein Fehlgriff**
`c`	n^2 / L	vernachlässigbar

$$M_{\text{naiv}} \approx n^3\left(1 + \frac{1}{L}\right) = 1{,}125\, n^3.$$

Bei $n = 1024$ sind das etwa 1,2 Milliarden Fehlgriffe. Bei 250 Takten je Fehlgriff wären das $3 \cdot 10^{11}$ Takte, also über eine Minute allein für das Warten.

Schritt 3 – Mit Blockbildung.

```
for ii, for jj, for kk:  for i in Block, for j, for k:  ...
```

Innerhalb eines Blocks passen alle drei Teilmatrizen in den Cache. Man lädt sie einmal und rechnet dann B^3 Multiplikationen darauf.

Größe	Wert	Rechnung
Blöcke insgesamt	$(n/B)^3$	
Ladeaufwand je Block	$3B^2/L$	drei Teilmatrizen
Fehlgriffe	$\mathbf{3n^3/(BL)}$	Produkt

Schritt 4 – Der Vergleich.

$$\frac{M_{\text{naiv}}}{M_{\text{Block}}} = \frac{1{,}125\,n^3}{3n^3/(BL)} = \frac{1{,}125\,BL}{3} = 0{,}375\,BL.$$

Mit $B = 32$ und $L = 8$:

$$\text{Faktor} = 0{,}375 \cdot 32 \cdot 8 = 96.$$

Fassung	Fehlgriffe	gemessene Zeit
naiv	$1{,}2 \cdot 10^9$	8,5 s
k-Schleife vertauscht	$2{,}7 \cdot 10^8$	1,9 s
geblockt ($B = 32$)	$1{,}3 \cdot 10^7$	0,55 s
Bibliothek (OpenBLAS)	$\approx 10^7$	0,08 s

($n = 1024$, doppelte Genauigkeit, ein Kern)

> *Die zweite Zeile ist bemerkenswert: Das bloße Vertauschen der beiden inneren Schleifen bringt schon Faktor 4,5, weil dann beide Felder zeilenweise durchlaufen werden. Es ist eine Änderung von zwei Zeilen.*

Die letzte Zeile erreicht ihre weiteren sieben Faktoren durch Vektorbefehle (Kapitel 10) und sorgfältig von Hand geschriebene innere Schleifen.

(b) Zwei Anordnungen derselben Daten.
Gegeben sind Angaben über eine Million Teilchen: Ort, Geschwindigkeit, Masse, Ladung, Kennung.
Anordnung 1: Feld von Strukturen.

```
struct Teilchen { double x,y,z; double
vx,vy,vz;
  double m, q; int id; };
struct Teilchen p[N];
```

Größe je Teilchen: $8 \cdot 8 + 4 = 68$ Byte, aufgefüllt auf 72.
Anordnung 2: Struktur von Feldern.

```
double x[N], y[N], z[N];
double vx[N], vy[N], vz[N];
double m[N], q[N]; int id[N];
```

Der Vergleich bei einer Ortsaktualisierung.

```
for (i=0;i<N;i++) { x[i] += vx[i]*dt; y[i] += vy[i]*dt; z[i]
                        += vz[i]*dt; }
```

Gebraucht werden 6 der 9 Werte je Teilchen, also 48 von 72 Byte.

Anordnung	**geholt je Teilchen**	**davon genutzt**
Strukturenfeld	72 B	48 B (67 %)
Feldstruktur	48 B	48 B (**100** %)

Bandbreitenersparnis $= 33\,\%$.

Hinzu kommt ein zweiter Vorteil, der oft schwerer wiegt: Die Feldstruktur lässt sich vektorisieren. Acht aufeinanderfolgende `x`-Werte liegen zusammen und passen in ein Vektorregister.

Anordnung	**Zeit**	**Bemerkung**
Strukturenfeld	12,4 ms	nicht vektorisierbar
Feldstruktur	2,1 ms	vektorisiert

Wann das Strukturenfeld besser ist

Die Umkehrung gilt, sobald man alle Felder eines Teilchens gleichzeitig braucht und die Teilchen in wahlfreier Reihenfolge anfasst.

Feldstruktur besser	Wenige Felder, alle Elemente, sequentiell. Typisch für Simulationen
Strukturenfeld besser	Alle Felder, wenige Elemente, wahlfrei. Typisch für Datenbanken und Objektgeflechte

Bei wahlfreiem Zugriff auf ein Teilchen holt die Feldstruktur neun verschiedene Cachezeilen, das Strukturenfeld nur eine oder zwei. Der Vorteil kehrt sich vollständig um.

Genau diese Abwägung steckt hinter der Unterscheidung zwischen zeilen- und spaltenorientierten Datenbanken. Spaltenorientierung gewinnt bei Auswertungen über wenige Spalten und viele Zeilen, Zeilenorientierung bei Zugriffen auf einzelne Datensätze (Band 10).

(c) Messen statt raten.
Die Werkzeuge.

Werkzeug	Art	Was es zeigt
`perf`	Hardwarezähler	Fehlgriffe, Fehlvorhersagen, IPC
`cachegrind`	Simulation	exakte Fehlgriffzahlen, langsam
`VTune`, `uProf`	Hersteller	sehr detailliert
`likwid`	Zähler	gut für Messreihen

Die Kennzahlen, die man ansieht.

Größe	guter Wert	was ein schlechter bedeutet
IPC	> 2	unter 1: irgendetwas blockiert
L1-Fehlgriffe	$< 5\,\%$	Zugriffsmuster prüfen
L3-Fehlgriffe	$< 1\,\%$	Arbeitsmenge zu groß
Sprungfehler	$< 2\,\%$	unvorhersagbare Bedingung
TLB-Fehlgriffe	$< 0{,}1\,\%$	große Seiten erwägen (Kap. 9)

Eine typische Sitzung.

```
perf stat -e
cycles,instructions,cache-misses, \
  L1-dcache-load-misses,branch-misses
./programm
```

Daraus liest man den IPC ab und sieht, ob eine der Fehlerraten aus dem Rahmen fällt. Anschließend lokalisiert man mit `perf record` und `perf report` die verantwortliche Stelle im Code.

Warum Messungen so oft lügen

TODD MYTKOWICZ und Mitarbeiter veröffentlichten 2009 einen Aufsatz mit dem Titel „Producing Wrong Data Without Doing Anything Obviously Wrong!“.
Sie zeigten, dass zwei Einflüsse, an die niemand denkt, die gemessene Laufzeit um mehrere Prozent verändern können.

Umgebungsvariablen	Ihre Gesamtlänge verschiebt den Stapelanfang und damit die Cacheausrichtung aller lokalen Variablen
Verknüpfungs-reihenfolge	Die Reihenfolge der Objektdateien verschiebt den Code und damit die Ausrichtung im Befehlscache

In ihren Messungen erzeugten diese Effekte Unterschiede von bis zu 33 Prozent. Das ist mehr als die Verbesserung, die viele veröffentlichte Optimierungen behaupten.

Ihre Empfehlung lautet, über eine Verteilung zufälliger Anordnungen zu mitteln, statt eine einzelne Konfiguration zu messen. Das tut fast niemand.

Die praktische Regel für den Alltag: Wer eine Verbesserung von unter fünf Prozent misst, hat vermutlich nichts gemessen.

Drepper[4] und Bryant und O’Hallaron[3] zeigen die Programmiererseite ausführlich; Mytkowicz und Mitautoren[18] sollte lesen, wer Messungen veröffentlicht; Fog[6] beschreibt die Optimierung bis auf Befehlsebene.

9 Virtueller Speicher

Drei Aufgaben in einem Mechanismus

Der virtuelle Speicher löst drei Probleme, die zunächst nichts miteinander zu tun haben.

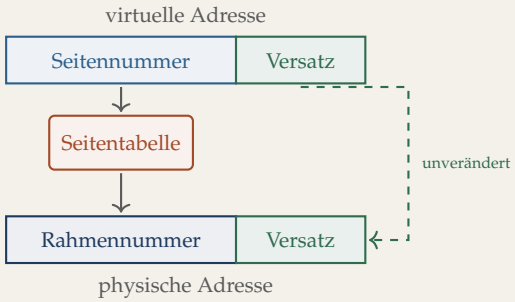
Schutz	Ein Programm kann nicht auf den Speicher eines anderen zugreifen
Umlagerung	Jedes Programm sieht seinen Speicher ab Adresse null; der Binder muss keine festen Adressen vergeben
Überzeichnung	Programme dürfen zusammen mehr Speicher belegen, als vorhanden ist; Selten benutztes wandert auf die Platte

> *Die dritte Aufgabe war die ursprüngliche und ist heute die unwichtigste. Der Atlas-Rechner von 1961 hatte 16 KB Kernspeicher und eine Trommel von 96 KB, und der virtuelle Speicher verbarg diesen Unterschied vor dem Programmierer.*

Heute hat ein Arbeitsplatzrechner genug Hauptspeicher, und Auslagerung ist ein Notbehelf, den man eher vermeidet. Die beiden ersten Aufgaben sind geblieben und heute die wichtigeren.

Seiten und Seitentabellen

Der Adressraum wird in **Seiten** gleicher Größe zerlegt, üblicherweise 4 KB. Der physische Speicher wird in ebenso große **Rahmen** zerlegt. Eine Tabelle sagt, welche Seite in welchem Rahmen liegt.



Der Versatz bleibt unverändert, weil Seiten und Rahmen gleich groß sind. Übersetzt wird nur die obere Hälfte der Adresse.

Das Größenproblem.
Bei 48 Bit Adressraum und 4-KB-Seiten gibt es

$$2^{48-12} = 2^{36} = 6{,}9 \cdot 10^{10} \text{ Seiten.}$$

Eine einstufige Tabelle mit 8 Byte je Eintrag wäre 550 GB groß, und zwar für jeden Prozess.

> *Der Ausweg besteht darin, die Tabelle selbst zu staffeln und nur die Teile anzulegen, die gebraucht werden. Ein typischer Prozess benutzt nur einen winzigen Bruchteil seines Adressraums.*



Bei x86-64 mit vier Ebenen zerlegt man die 48 Bit in $9+9+9+9+12$. Jede Tabelle hat $2^9 = 512$ Einträge zu 8 Byte, passt also genau in eine Seite von 4 KB.

> *Dass eine Tabellenebene genau in eine Seite passt, ist kein Zufall, sondern der Grund für die Wahl von neun Bit. Die Tabellen können damit selbst ausgelagert und mit demselben Mechanismus verwaltet werden.*

Seit 2019 gibt es eine fünfte Ebene für 57 Bit Adressraum, die aber nur bei sehr großen Serversystemen eingeschaltet wird.

Der TLB

Vier Tabellenebenen bedeuten vier Speicherzugriffe für jede Adressübersetzung, und die kämen zu jedem eigentlichen Zugriff hinzu. Das wäre untragbar.
Deshalb gibt es einen kleinen, sehr schnellen Zwischenspeicher für Übersetzungen, den **Translation Lookaside Buffer**.

Ebene	Einträge	Zugriff	abgedeckt
L1-TLB (Daten)	64–96	1 Takt	256–384 KB
L2-TLB	1536–3072	7 Takte	6–12 MB
Tabellenweg	—	≈ 100 Takte	

Die letzte Spalte ist die wichtige Zahl. Ein L2-TLB mit 2048 Einträgen deckt bei 4-KB-Seiten nur 8 MB ab. Wer über einen größeren Datenbereich wahlfrei zugreift, erzeugt bei fast jedem Zugriff einen TLB-Fehlgriff.

Das ist ein eigenes Problem, unabhängig vom Datencache. Ein Programm kann perfekte Cachetrefferquoten haben und trotzdem am TLB scheitern.

Große Seiten

Die Abhilfe besteht darin, größere Seiten zu benutzen.

Seitengröße	Ebenen	abgedeckt bei 2048 Einträgen
4 KB	4	8 MB
2 MB	3	4 GB
1 GB	2	2 TB

Der Sprung von 4 KB auf 2 MB vergrößert die Reichweite um den Faktor 500 und spart zugleich eine Tabellenebene.

Anwendung	Gewinn	Bemerkung
Datenbankpuffer	10–30 %	regelmäßig gemessen
Numerische Rechnung	5–15 %	
Sprachlaufzeitsysteme	5–20 %	Java, .NET
gewöhnliche Programme	≈ 0	Arbeitsmenge klein genug

> *Der Preis ist innere Verschnitt. Wer 100 KB braucht und eine 2-MB-Seite bekommt, verschwendet 95 Prozent. Deshalb schaltet man große Seiten gezielt für einzelne Bereiche ein und nicht pauschal.*

Linux bietet dafür zwei Wege: `hugetlbfs` mit ausdrücklicher Reservierung und **Transparent Huge Pages**, wo der Kern selbst entscheidet. Die zweite Variante hat bei Datenbanken einen schlechten Ruf, weil das Zusammenlegen von Seiten im Hintergrund unvorhersehbare Aussetzer erzeugt.

Aufgabe 9.1.

(a) Rechnen Sie eine vierstufige Adressübersetzung vollständig durch.

(b) Bestimmen Sie die Kosten von TLB-Fehlgriffen bei einem konkreten Programm.

(c) Untersuchen Sie, wie Speicherzuteilung und Cacheverhalten zusammenhängen.

Lösung Schritt für Schritt.

(a) Eine Übersetzung Schritt für Schritt.
Gegeben: virtuelle Adresse $(\texttt{00007F3C 52A1 B008})_{16}$, x86-64 mit vier Ebenen.
Schritt 1 – Die Adresse zerlegen.
Die unteren 48 Bit sind $(\texttt{7F3C52A1B008})_{16}$, binär:

$$(0111\,1111\,0011\,1100\,0101\,0010\,1010\,0001\,1011\,0000\,0000\,1000)_2.$$

Aufgeteilt in $9+9+9+9+12$:

Feld	Bits	Binär	Dezimal
L4-Index	47–39	011111110	254
L3-Index	38–30	011110001	241
L2-Index	29–21	010101000	168
L1-Index	20–12	011011000	216
Versatz	11–0	000000001000	8

Schritt 2 – Der Weg durch die Tabellen.

Ebene	Adresse des Eintrags	Ergebnis
Start	$\texttt{CR3} = (\texttt{1A000})_{16}$	Wurzel der Tabelle
L4	$(\texttt{1A000})_{16} + 254 \cdot 8$	Rahmen $(\texttt{2B000})_{16}$
L3	$(\texttt{2B000})_{16} + 241 \cdot 8$	Rahmen $(\texttt{3C000})_{16}$
L2	$(\texttt{3C000})_{16} + 168 \cdot 8$	Rahmen $(\texttt{4D000})_{16}$
L1	$(\texttt{4D000})_{16} + 216 \cdot 8$	Rahmen $(\texttt{7E000})_{16}$
Ergebnis	$(\texttt{7E000})_{16} + 8$	$= (\texttt{7E008})_{16}$

Schritt 3 – Die Kosten.

Fall	Takte	Bemerkung
TLB-Treffer (L1)	0–1	im Zugriff enthalten
TLB-Treffer (L2)	7	
Tabellenweg, alles im Cache	≈ 20	vier L2-Zugriffe
Tabellenweg, nichts im Cache	≈ 1000	vier DRAM-Zugriffe
Seitenfehler	$\approx 10^6$	Betriebssystem, Platte

Die vorletzte Zeile ist der unangenehme Fall. Vier abhängige Speicherzugriffe, die alle fehlgreifen, kosten mehr als tausend Takte, und zwar bevor der eigentliche Zugriff überhaupt beginnt.

Deshalb legen Prozessoren die Tabelleneinträge im normalen Datencache ab und haben zusätzlich eigene kleine Zwischenspeicher für die oberen Ebenen.

(b) TLB-Fehlgriffe messen.
Der Fall. Eine Hashtabelle mit 100 Millionen Einträgen zu je 16 Byte, also 1,6 GB, wird zufällig durchsucht.
Schritt 1 – Die TLB-Reichweite.

$$2048 \text{ Einträge} \cdot 4 \text{ KB} = 8 \text{ MB} \ll 1{,}6 \text{ GB}.$$

$$p_{\text{TLB-Fehl}} \approx 1 - \frac{8}{1600} = 99{,}5\,\%.$$

Schritt 2 – Die Kosten je Zugriff.

Anteil	Takte	Summe
Tabellenweg	≈ 150	$0{,}995 \cdot 150 = 149$
eigentlicher Zugriff	250	250
gesamt		**399**

Die Adressübersetzung kostet also etwa 60 Prozent zusätzlich, obwohl der Datenzugriff selbst schon teuer ist. Das ist ein Aufschlag, den man leicht übersieht, weil er in keiner Cachestatistik auftaucht.

Schritt 3 – Mit großen Seiten.

$$2048 \cdot 2\,\text{MB} = 4\,\text{GB} > 1{,}6\,\text{GB}.$$

$$p_{\text{TLB-Fehl}} \approx 0 \Longrightarrow \text{Kosten} = 250\ \text{Takte}.$$

$$\text{Gewinn} = \frac{399}{250} = 1{,}60.$$

Konfiguration	Zeit	relativ
4-KB-Seiten	8,2 s	1,00
2-MB-Seiten	5,3 s	0,65

Ein Drittel der Laufzeit gespart, durch eine Einstellung, die keine Codeänderung verlangt. Datenbanken schalten große Seiten deshalb standardmäßig ein.

(c) Speicherzuteilung und Cache.
Das Problem der Ausrichtung.
Der Zuteiler `malloc` liefert Zeiger, die auf 16 Byte ausgerichtet sind. Für Cachezeilen von 64 Byte ist das zu wenig.

Eine Struktur von 48 Byte kann so über zwei Cachezeilen verteilt liegen. Jeder Zugriff kostet dann zwei Zeilen statt einer.

Ausrichtung	Zeilen je Zugriff	Wirkung
zufällig	1,75 im Mittel	75 % Mehraufwand
auf 64 B	1,0	

Man erzwingt die Ausrichtung mit `aligned_alloc` oder mit einem Attribut an der Strukturdefinition.
Das Problem des falschen Teilens.
Bei mehreren Kernen entsteht ein zweites Problem. Wenn zwei Kerne verschiedene Variablen bearbeiten, die zufällig in derselben Cachezeile liegen, muss die Zeile ständig zwischen ihnen wandern.

```
struct { long zaehler[8]; } gemeinsam;
/* schlecht */
/* Kern k erhöht gemeinsam.zaehler[k]
*/
```

Acht `long` sind 64 Byte und liegen damit in einer Zeile. Jeder Kern schreibt in dieselbe Zeile, obwohl die Werte unabhängig sind.

Anordnung	Zeit	Bemerkung
gemeinsame Zeile	4,1 s	falsches Teilen
je Zähler eine Zeile	0,12 s	64 B Abstand

Faktor 34, allein durch das Einfügen von Füllbytes zwischen den Zählern. Kapitel 11 erklärt den Mechanismus dahinter.

Woran man falsches Teilen erkennt

Es ist eine der tückischsten Bremsen, weil im Code nichts Verdächtiges steht. Die Variablen sind unabhängig, es gibt keine Sperren, und trotzdem skaliert das Programm nicht.

Symptom	Mehr Kerne machen das Programm langsamer statt schneller
Messgröße	Hoher Anteil an Kohärenzverkehr; unter Linux etwa `perf c2c`
Abhilfe	Auffüllen auf Zeilengröße, oder Zwischenergebnisse lokal halten und erst am Ende zusammenführen

Die zweite Abhilfe ist die bessere, weil sie den Verkehr ganz vermeidet statt ihn nur zu entzerren. Jeder Faden zählt in einer lokalen Variablen und addiert am Schluss einmal auf.

Bryant und O’Hallaron[3] behandeln den virtuellen Speicher aus Programmierersicht; Jacob, Ng und Wang[12] gehen auf die Bauteilebene; Smith und Nair[21] behandeln die Virtualisierung, die auf demselben Mechanismus beruht.

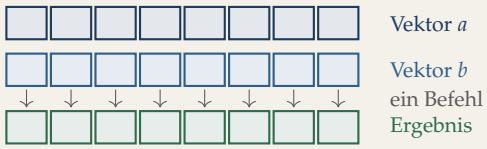
Part IV

Vielfalt und Parallelität

10 Vektorbefehle

Ein Befehl, viele Daten

Kapitel 6 hat gezeigt, dass die Hardware allein kaum mehr als drei Befehle je Takt aus einem gewöhnlichen Programm herausholt. Wenn die Parallelität dagegen im Code steht, geht deutlich mehr.



Erweiterung	Breite	Jahr	floats je Reg.
MMX	64 Bit	1996	— (nur ganzzahlig)
SSE	128 Bit	1999	4
AVX	256 Bit	2011	8
AVX-512	512 Bit	2016	16
ARM NEON	128 Bit	2005	4
ARM SVE	**variabel**	2016	4–64
RISC-V V	**variabel**	2021	

Die letzten beiden Zeilen sind ein bemerkenswerter Bruch mit der bisherigen Praxis. Bei SVE und der RISC-V-Vektorerweiterung steht die Registerbreite nicht im Befehl. Derselbe Code läuft auf Maschinen mit 128 und mit 2048 Bit breiten Registern.

Das ist genau die Lehre aus dem Itanium (Kapitel 6), auf eine andere Ebene übertragen: Man schreibt in den Code, *dass* etwas parallel ist, aber nicht, *wie breit* die Maschine ist.

Wie man an Vektorcode kommt

Weg	Aufwand	Ergebnis
Bibliothek benutzen	keiner	**meist optimal**
Übersetzer automatisch	gering	wechselhaft
Übersetzerhinweise	mittel	oft gut
Intrinsics	hoch	sehr gut
Assembler	sehr hoch	Spitze

Die erste Zeile ist die wichtigste und wird am häufigsten übersehen. Wer Matrizen multipliziert, sollte eine BLAS-Bibliothek benutzen und nicht selbst vektorisieren.
Warum die automatische Vektorisierung so oft scheitert.

Zeigermehrdeutigkeit	Der Übersetzer weiß nicht, ob sich Ein- und Ausgabefeld überlappen
Datenabhängigkeit	`a[i] = a[i-1] + 1` lässt sich nicht vektorisieren
Kontrollfluss	Verzweigungen im Schleifenkörper erschweren es
Unbekannte Länge	Was tun, wenn n kein Vielfaches der Vektorbreite ist
Gleitkommaordnung	Umordnen ändert das Ergebnis; ohne Erlaubnis darf der Übersetzer nicht

Die letzte Zeile ist die unangenehmste, weil sie viele numerische Schleifen betrifft. Eine Summation lässt sich nur vektorisieren, wenn man die Addition umordnen darf, und die Gleitkommaaddition ist nicht assoziativ (Band 0, Kapitel 3).

Man erlaubt es mit `-ffast-math` oder gezielter mit einem Pragma. Beides sollte man bewusst tun und nicht pauschal.

Wie der Übersetzer die Überlappung umgeht

Bei einer Schleife wie

```
void f(float *a, float *b, int n) { for (i=0;i<n;i++)
                    a[i] += b[i]; }
```

kann der Übersetzer nicht wissen, ob `a` und `b` auf denselben Speicher zeigen. Er hilft sich mit einer Laufzeitprüfung. Der erzeugte Code enthält beide Fassungen und wählt zu Beginn:

```
if (a+n <= b || b+n <= a)
  vektorisierte Schleife
else
  gewöhnliche Schleife
```

Das kostet Codegröße und eine Verzweigung, funktioniert aber. Mit `restrict` entfällt die Prüfung, weil der Programmierer zusichert, dass keine Überlappung vorliegt.

> *Bei kurzen Schleifen kann der Prüfaufwand den Gewinn auffressen. Der Übersetzer schätzt deshalb ab, ob sich das lohnt, und liegt dabei manchmal daneben.*

Aufgabe 10.1.

(a) Vektorisieren Sie eine Summation und behandeln Sie die Assoziativitätsfrage.
(b) Untersuchen Sie, warum AVX-512 den Takt senkt.
(c) Vergleichen Sie Vektorbefehle mit mehreren Kernen.

Lösung Schritt für Schritt.

(a) Eine Summation vektorisieren.
Die Ausgangsschleife.

```
float s = 0; for (i=0;i<n;i++) s += a[i];
```

Schritt 1 – Warum das nicht direkt geht.
Jeder Durchlauf braucht das Ergebnis des vorigen. Es entsteht eine Abhängigkeitskette der Länge n.

Größe	Wert	Folge
Latenz einer Addition	4 Takte	
Kettenlänge	n	
Gesamtzeit	**4n Takte**	unabhängig von der Breite

Der Prozessor könnte zwei Additionen je Takt starten, aber es gibt nur eine, die bereit ist.

Schritt 2 – Die Kette aufbrechen.
Man führt mehrere Teilsummen und addiert sie am Ende zusammen.

```
float s0=0,s1=0,s2=0,s3=0;
for (i=0;i+3<n;i+=4) {
  s0 += a[i];  s1 += a[i+1];
  s2 += a[i+2];  s3 += a[i+3]; }
s = (s0+s1) + (s2+s3);
```

Jetzt gibt es vier unabhängige Ketten der Länge $n/4$.

$$T = 4 \cdot \frac{n}{4} = n \text{ Takte}, \qquad \text{Faktor 4.}$$

Schritt 3 – Mit Vektorbefehlen.
Mit AVX (8 floats je Register) und vier Teilsummen zu je einem Vektorregister:

```
__m256 v0=_mm256_setzero_ps(), v1=...,
v2=..., v3=...;
for (i=0;i+31<n;i+=32) {
  v0 = _mm256_add_ps(v0,
_mm256_loadu_ps(a+i));
  v1 = _mm256_add_ps(v1,
_mm256_loadu_ps(a+i+8));
  v2 = _mm256_add_ps(v2,
_mm256_loadu_ps(a+i+16));
  v3 = _mm256_add_ps(v3,
_mm256_loadu_ps(a+i+24)); }
```

$$T = 4 \cdot \frac{n}{32} = \frac{n}{8} \text{ Takte}, \qquad \text{Faktor } 32.$$

Fassung	Takte	Faktor
Original	$4n$	1
vier Teilsummen	n	4
AVX, eine Summe	$4n/8 = n/2$	8
AVX, vier Teilsummen	$n/8$	**32**

In der Praxis begrenzt die Speicherbandbreite das Ergebnis. Bei einem großen Feld kann der Prozessor nicht mehr als etwa 32 Byte je Takt lesen, und dann ist bei Faktor 8 Schluss.

Schritt 4 – Die Assoziativitätsfrage.
Alle Fassungen außer der ersten addieren in anderer Reihenfolge. Bei Gleitkommazahlen liefert das ein anderes Ergebnis.

Fassung	Ergebnis	Fehler
exakt	1,000000000	—
sequentiell	0,999998	$2 \cdot 10^{-6}$
vier Teilsummen	1,0000003	$3 \cdot 10^{-7}$

(Beispiel: eine Million Zahlen der Größenordnung 10^{-6})

Bemerkenswert ist, dass die umgeordnete Fassung hier genauer *ist. Der Grund liegt darin, dass bei der sequentiellen Summation die Teilsumme schnell groß wird und kleine Beiträge verschluckt (Band 0, Kapitel 3).*

Die Umordnung ist also nicht per se schlechter, sondern nur anders. Wer eine bestimmte Genauigkeit braucht, benutzt ein Verfahren mit Fehlerkompensation, etwa die Summation nach KAHAN, und verzichtet auf einen Teil der Geschwindigkeit.

(b) Warum AVX-512 den Takt senkt.
Die Beobachtung. Auf Intel-Prozessoren der Jahre 2016 bis 2021 senkte die Benutzung breiter Vektorbefehle die Taktfrequenz aller Kerne.

Befehlsart	Takt	Absenkung
skalar	3,5 GHz	—
AVX2 (256 Bit)	3,2 GHz	9 %
AVX-512, leicht	2,9 GHz	17 %
AVX-512, schwer	2,2 GHz	**37** %

(Skylake-SP, alle Kerne aktiv; „schwer“ meint Gleitkommamultiplikationen)

Der Grund.

Ein 512 Bit breites Rechenwerk schaltet in jedem Takt achtmal so viele Transistoren wie ein skalares. Die Verlustleistung würde die thermische Grenze überschreiten, also senkt der Prozessor den Takt.

Die Rechnung, ob es sich lohnt.

$$\text{Gewinn} = \frac{\text{Breite} \times \text{Takt}_{\text{neu}}}{1 \times \text{Takt}_{\text{alt}}}.$$

Fall	Breite	Taktfaktor	Gewinn
AVX2	8	0,91	7,3
AVX-512	16	0,63	**10,1**

Es lohnt sich also, solange man durchgehend vektorisiert. Unangenehm wird es, wenn ein Programm nur gelegentlich einen AVX-512-Befehl ausführt, etwa in einer Bibliotheksfunktion. Dann sinkt der Takt für alles Übrige mit.

Das führte zu einer merkwürdigen Empfehlung: Manche Projekte schalteten AVX-512 bewusst ab, weil einzelne Aufrufe die Gesamtleistung senkten. Die Speicherfunktion `memcpy` war ein bekannter Fall.

Seit etwa 2021 haben neuere Fertigungsverfahren das Problem entschärft. Bei den Serverprozessoren ab 2023 ist die Absenkung deutlich geringer.

(c) Vektoren gegen Kerne.

Der Vergleich.

	Vektorbefehle	mehrere Kerne
Overhead	null	Fadenerzeugung, Synchronisation
Granularität	einzelne Schleife	ab etwa 10^4 Operationen
Speicherbandbreite	geteilt	geteilt
Programmierung	Übersetzer oder Intrinsics	Fäden, OpenMP
Fehleranfälligkeit	gering	Wettläufe, Verklemmungen

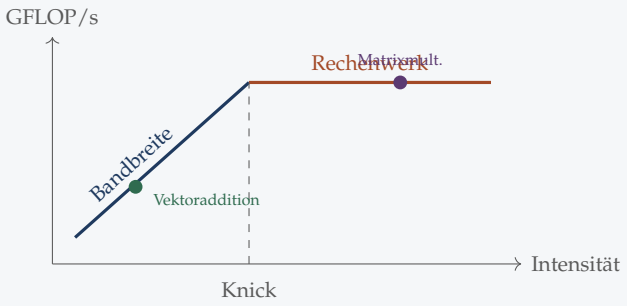
Vektorbefehle sind die billigere Parallelität. Sie kosten keinen Verwaltungsaufwand und können keine Wettläufe erzeugen. Man sollte sie deshalb zuerst ausschöpfen.

Die Rechnung an einem Beispiel.
Ein Feld von 10^8 floats soll elementweise verarbeitet werden. Maschine: 8 Kerne, AVX2, 50 GB/s Speicherbandbreite.

Fassung	Zeit	begrenzt durch
skalar, 1 Kern	0,40 s	Rechenwerk
AVX2, 1 Kern	0,055 s	Rechenwerk
AVX2, 8 Kerne	0,016 s	**Speicherbandbreite**
theoretisch, 8 Kerne	0,007 s	Rechenwerk

Von acht Kernen bringen nur dreieinhalb etwas, weil der Speicher nicht schneller liefert. Diese Grenze trifft man in der Praxis sehr häufig, und weitere Kerne helfen dann gar nicht mehr.

Das Dachlinienmodell.
SAMUEL WILLIAMS und Mitarbeiter schlugen 2009 eine einfache Darstellung dafür vor. Man trägt die erreichbare Rechenleistung über der **arithmetischen Intensität** auf, also über dem Verhältnis von Rechenoperationen zu geladenen Bytes.



Aufgabe	Intensität	begrenzt durch
$c[i] = a[i] + b[i]$	0,08	Bandbreite
Matrix-Vektor	0,25	Bandbreite
Matrixmultiplikation	$\approx n/12$	**Rechenwerk**
Stencil (5-Punkt)	0,5	Bandbreite

Das Modell sagt in einem Blick, ob sich Optimierung am Rechenwerk überhaupt lohnt. Liegt der Punkt links vom Knick, so hilft nur, weniger Daten zu bewegen.

Die dritte Zeile erklärt, warum die Matrixmultiplikation als einzige der vier Aufgaben die volle Rechenleistung erreichen kann. Sie führt $2n^3$ Operationen auf $3n^2$ Daten aus, und dieses Verhältnis wächst mit n. Hennessy und Patterson[8] behandeln Vektorarchitekturen ausführlich; Fog[6] dokumentiert die tatsächlichen Kosten aller Vektorbefehle; Asanović und Mitautoren[2] ordnen die Datenparallelität in die Landschaft ein.

11 Mehrere Kerne

Die Wende von 2005

Bis 2004 wurden Prozessoren schneller, indem sie höher takteten. Danach nicht mehr.

Jahr	Takt	Kerne	Leistung/Kern
1995	100 MHz	1	1
2000	1,0 GHz	1	8
2005	3,8 GHz	1–2	22
2015	4,0 GHz	4–8	45
2026	5,5 GHz	16–128	85

Zwischen 2005 und 2026 stieg der Takt um 45 Prozent und die Kernzahl um mehr als das Fünfzigfache. Die Last, damit umzugehen, wanderte von den Chipentwerfern zu den Programmierern.

Was Amdahls Gesetz dazu sagt.
Aus Band 0 (Kapitel 15) kennen wir die Formel

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}.$$

p	$S(8)$	$S(64)$	$S(1000)$	$S(\infty)$
0,90	4,7	7,8	9,9	10
0,95	6,0	13,9	19,6	20
0,99	7,5	39,3	91	100
0,999	7,9	60,2	500	1000

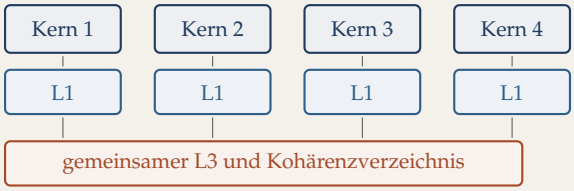
MARK HILL und MICHAEL MARTY haben 2008 gezeigt, was daraus für den Chipentwurf folgt. Bei fester Chipfläche kann man viele kleine oder wenige große Kerne bauen, und die richtige Wahl hängt vom parallelen Anteil ab.

Bei p nahe 1	viele kleine Kerne	alles läuft parallel
Bei kleinem p	wenige große Kerne	der serielle Teil zählt
Gemischt	**asymmetrisch**	ein großer, viele kleine

Die dritte Zeile war 2008 eine Empfehlung und ist seit 2020 Praxis. ARMs „big.LITTLE“ und Intels Aufteilung in Leistungs- und Effizienzkerne folgen genau dieser Überlegung.

Cachekohärenz

Jeder Kern hat eigene L1- und meist eigene L2-Caches. Damit kann dieselbe Speicherstelle in mehreren Caches liegen, und die Kopien müssen übereinstimmen.



Das MESI-Protokoll.
Jede Cachezeile trägt einen von vier Zuständen.

Zustand	Bedeutung	Erlaubt
Modified	nur hier, verändert	lesen, schreiben
Exclusive	nur hier, unverändert	lesen, schreiben (wird dann M)
Shared	mehrfach vorhanden	**nur lesen**
Invalid	ungültig	nichts

Will ein Kern in eine Zeile schreiben, die im Zustand S ist, so muss er die Kopien der anderen für ungültig erklären. Erst danach darf er schreiben.

Genau das erzeugt das falsche Teilen aus Kapitel 9. Zwei Kerne, die unabhängige Variablen in derselben Zeile beschreiben, schicken die Zeile ständig hin und her.

Vorgang	Takte	Bemerkung
L1-Treffer, eigener Kern	4	
Zeile im Zustand S holen	≈ 40	vom L3
Zeile von anderem Kern (M)	≈ 100–200	**Übertragung nötig**
über Sockelgrenze	≈ 300–500	bei Mehrsockelsystemen

Speichermodelle

Kohärenz sagt, dass alle Kerne für *eine* Speicherstelle dieselbe Folge von Werten sehen. Sie sagt nichts darüber, in welcher Reihenfolge Zugriffe auf *verschiedene* Stellen sichtbar werden.

Kern 1	Kern 2
`x = 1;`	`y = 1;`
`r1 = y;`	`r2 = x;`

Bei jeder denkbaren Verschränkung der vier Anweisungen müsste mindestens eine der beiden Zuweisungen vor dem entsprechenden Lesevorgang liegen. Das Ergebnis $r1 = r2 = 0$ sollte also unmöglich sein.

> *Auf realen Maschinen tritt es auf. Der Grund sind Schreibpuffer: Ein Schreibvorgang wird zwischengespeichert, und der folgende Lesevorgang darf ihn überholen.*

Modell	Umordnung erlaubt	Architektur
sequentiell konsistent	keine	keine reale
TSO	Schreiben vor Lesen	**x86-64**
schwach	fast alles	**ARM, RISC-V, POWER**

x86 gibt vergleichsweise starke Garantien, ARM sehr schwache. Deshalb läuft nebenläufiger Code, der auf x86 jahrelang funktioniert hat, auf ARM manchmal falsch, und zwar selten und schwer reproduzierbar.

Man erzwingt Ordnung mit **Speicherbarrieren**. In C und C++ geschieht das über `std::atomic` mit einer ausdrücklichen Ordnungsangabe.

Ordnung	Kosten (x86)	Kosten (ARM)
`relaxed`	0	0
`acquire` / `release`	0	einige Takte
`seq_cst` (Standard)	≈ 20 Takte	einige Takte

Auf x86 sind `acquire` und `release` kostenlos, weil das Modell sie ohnehin garantiert. Nur die stärkste Ordnung braucht eine Barriere. Auf ARM ist es umgekehrt: Man zahlt für jede Ordnungsangabe, dafür sind die Kosten gleichmäßiger.

Aufgabe 11.1.

(a) Rechnen Sie die Kosten eines Zählers unter mehreren Kernen aus.
(b) Untersuchen Sie NUMA und seine Folgen.
(c) Analysieren Sie, warum Sperren so teuer sind, und was man stattdessen tut.

Lösung Schritt für Schritt.

(a) Ein gemeinsamer Zähler.

Der Fall. N Kerne erhöhen je 10^7-mal denselben Zähler.
Fassung 1 – Atomare Erhöhung.

```
std::atomic<long> z; ...  z.fetch_add(1);
```

Jede Erhöhung braucht die Zeile im Zustand M. Bei N Kernen wandert sie reihum.

N	**Takte je Erh.**	**Gesamtzeit**	**Skalierung**
1	20	0,06 s	1,0
2	80	0,47 s	**0,13**
4	150	1,8 s	0,03
8	280	6,5 s	0,01

Mehr Kerne machen das Programm hundertfach langsamer. Das ist keine schlechte Skalierung, sondern negative Skalierung, und sie kommt in der Praxis häufiger vor, als man denkt.

Fassung 2 – Lokale Zähler.

```
long lokal = 0; for (...)  lokal++;  z.fetch_add(lokal);
```

N	**Zeit**	**Skalierung**
1	0,010 s	1,0
2	0,005 s	2,0
4	0,0026 s	3,9
8	0,0014 s	7,1

Der Vergleich der beiden Tabellen bei acht Kernen ergibt einen Faktor von 4600. Die Regel dahinter ist einfach: gemeinsame Zustände so weit wie möglich vermeiden und Ergebnisse erst am Ende zusammenführen.

(b) NUMA.
Bei größeren Systemen hat nicht jeder Kern gleich schnellen Zugang zu jedem Speicherbereich. Der Speicher ist an die Prozessorsockel verteilt.



Zugriff	Latenz	Bandbreite
lokal	80 ns	200 GB/s
entfernt (1 Sockel)	140 ns	80 GB/s
entfernt (2 Sockel)	200 ns	50 GB/s

Die Regel für die Praxis.
Linux ordnet eine Speicherseite demjenigen Knoten zu, dessen Kern sie zuerst berührt. Das heißt **first touch**.

Wenn ein einzelner Faden ein großes Feld anlegt und mit Nullen füllt, landet es vollständig auf einem Knoten. Alle anderen Kerne greifen dann entfernt zu.

```
/* schlecht */  memset(a, 0,
n*sizeof(double));
#pragma omp parallel for
for (i=0;i<n;i++) a[i] = f(i);
```

```
/* gut:  Initialisierung ebenfalls
parallel */
#pragma omp parallel for
for (i=0;i<n;i++) a[i] = 0;
```

Initialisierung	Rechenzeit	relativ
seriell	2,8 s	1,00
parallel	1,5 s	0,54

Fast Faktor zwei, allein durch die Frage, welcher Faden die Nullen hineinschreibt. Der eigentliche Rechencode ist in beiden Fällen identisch.

(c) Sperren und ihre Alternativen.
Was eine Sperre kostet.

Fall	Takte	Bemerkung
unbestritten, gleicher Kern	20–40	nur die atomare Operation
bestritten, anderer Kern	200–500	Zeile muss wandern
Faden schläft ein	10^4–10^5	Betriebssystem

Die dritte Zeile ist der Grund, warum man kurze kritische Abschnitte mit einer Warteschleife absichert statt mit einer echten Sperre. Wer nur zwanzig Takte wartet, sollte nicht ins Betriebssystem gehen.

Vier Wege, Sperren zu vermeiden.

Aufteilen	Jeder Faden arbeitet auf eigenen Daten; am Ende zusammenführen
Nur lesen	Unveränderliche Daten brauchen keine Sperre
Sperrfrei	Atomare Operationen mit Vergleichen-und-Tauschen
Nachrichten	Statt gemeinsamem Speicher tauschen die Fäden Nachrichten aus

Die erste Zeile ist fast immer die beste Antwort und wird zu selten gewählt. Die dritte ist verlockend und schwierig.

Sperrfreie Programmierung ist schwerer, als sie aussieht

Ein sperrfreier Kellerspeicher sieht so aus:

```
do { alt = kopf; neu->next = alt; }
while
(!kopf.compare_exchange_weak(alt,
neu));
```

Beim Entnehmen entsteht ein Problem, das als **ABA-Problem** bekannt ist. Ein Faden liest den Kopf, wird unterbrochen, und in der Zwischenzeit wird das Element entnommen, ein anderes eingefügt und dasselbe Element wieder eingefügt. Der Vergleich stimmt, obwohl sich die Struktur geändert hat.

> *Hinzu kommt die Frage, wann man einen entnommenen Knoten freigeben darf. Ein anderer Faden könnte noch einen Zeiger darauf halten. Die üblichen Lösungen heißen Gefahrzeiger oder epochenbasierte Rückgewinnung und sind beide nicht einfach.*

MAURICE HERLIHY und NIR SHAVIT schreiben in ihrem Lehrbuch[10], sperrfreie Datenstrukturen seien nur dann zu empfehlen, wenn man sie aus einer geprüften Bibliothek nimmt. Selbst geschriebene enthalten fast immer Fehler, die erst unter Last und auf schwach geordneten Architekturen auftreten.

> *Ein bekanntes Beispiel: Der Java-Standardbibliothek fiel 2004 auf, dass eine ihrer nebenläufigen Klassen auf mehreren Prozessorarchitekturen falsch arbeitete. Der Fehler steckte im Speichermodell, nicht im Code.*

Sorin, Hill und Wood[22] behandeln Kohärenz und Konsistenz auf sechzig Seiten sehr klar; Adve und Gharachorloo[1] sind die klassische Einführung in Speichermodelle; Herlihy und Shavit[10] sowie McKenney[17] behandeln die Programmierseite.

12 Grafikprozessoren und Spezialbausteine

Ein anderer Entwurfspunkt

Ein Prozessorkern gibt den größten Teil seiner Fläche für Dinge aus, die nicht rechnen: Sprungvorhersage, Umordnung, Caches, Registerumbenennung. Alles davon dient dazu, aus einem sequentiellen Programm etwas Parallelität herauszuholen.

Wenn die Parallelität schon im Problem steckt, ist dieser Aufwand verschwendet. Dann baut man lieber viele einfache Rechenwerke.

	CPU-Kern	**GPU**
Kerne	8–128	**10^4** (einfache)
Takt	5 GHz	1,5–2,5 GHz
Sprungvorhersage	aufwendig	**keine**
Umordnung	ja	**nein**
Cache je Kern	32 KB	wenige KB
Speicherbandbreite	100 GB/s	1–8 TB/s
Latenzbehandlung	Caches, Umordnung	**Umschalten auf andere Fäden**

Die letzte Zeile ist die entscheidende Umkehrung. Eine CPU versucht, Wartezeit zu vermeiden. Eine GPU nimmt sie hin und füllt sie mit anderer Arbeit.

Eine GPU hält Tausende von Fäden gleichzeitig bereit. Wartet einer auf den Speicher, so rechnet ein anderer weiter. Der Umschaltvorgang kostet nichts, weil alle Fäden ihre Register gleichzeitig behalten.

Wie eine GPU rechnet

Die Fäden werden in Gruppen von 32 zusammengefasst, bei Nvidia **Warp** genannt, bei AMD **Wavefront** mit 64. Alle Fäden einer Gruppe führen *denselben* Befehl aus.

> *Das ist im Kern ein Vektorbefehl, nur anders aufgeschrieben. Der Programmierer schreibt Code für einen Faden, und die Hardware führt ihn für 32 gleichzeitig aus.*

Was bei Verzweigungen geschieht.
Wenn die Fäden einer Gruppe bei einer Verzweigung verschiedene Wege nehmen, muss die Maschine beide Wege nacheinander ausführen und jeweils die nicht betroffenen Fäden abschalten.



Fall	Wirkungsgrad	Bemerkung
alle 32 gleich	100 %	Idealfall
zweigeteilt	50 %	beide Wege nacheinander
alle verschieden	≈ 3 %	32 Durchläufe

> *Man nennt das **Divergenz**, und sie ist der häufigste Grund für enttäuschende GPU-Leistung. Ein Algorithmus mit vielen datenabhängigen Verzweigungen läuft auf einer GPU schlecht, auch wenn er hochgradig parallel ist.*

Zusammenhängende Speicherzugriffe.
Wenn die 32 Fäden einer Gruppe auf 32 aufeinanderfolgende Speicherstellen zugreifen, fasst die Hardware das zu wenigen Speicherzugriffen zusammen. Greifen sie verstreut zu, werden es 32 einzelne.

Muster	Transaktionen	Bandbreite
zusammenhängend	4	100 %
Schrittweite 2	8	50 %
zufällig	32	≈ 12 %

Das ist dieselbe Überlegung wie bei den Cachezeilen aus Kapitel 7, nur mit anderen Zahlen und schärferen Folgen.

Spezialbausteine

Seit etwa 2015 baut man zunehmend Schaltungen für einzelne Aufgaben statt allgemeiner Rechenwerke.

Baustein	Aufgabe	Gewinn (Energie)
Videokodierer	H.265, AV1	100–1000-fach
Tensoreinheit	Matrixmultiplikation	30–80-fach
Kryptobefehle	AES, SHA	5–10-fach
Signalprozessor	Filter, FFT	10–50-fach

Die Gewinne sind bei der Energie größer als bei der Geschwindigkeit. Ein Spezialbaustein spart vor allem den Aufwand, den ein allgemeiner Prozessor für Dekodierung, Umordnung und Registerverwaltung treibt.

Wie Google eine eigene Recheneinheit baute

Um 2013 rechnete Google aus, was passieren würde, wenn alle Nutzer die Spracherkennung drei Minuten täglich benutzten. Das Ergebnis: Man müsste die Zahl der Rechenzentren verdoppeln.
Statt dessen entwarf man einen eigenen Baustein. Er hieß **Tensor Processing Unit**, war im Wesentlichen ein großes systolisches Feld für Matrixmultiplikationen mit 8-Bit-Zahlen, und ging 2015 in Betrieb.

	CPU	GPU	TPU v1
Durchsatz	1	14	**29**
je Watt	1	2,3	**34**

Bemerkenswert ist der Unterschied zwischen den beiden Zeilen. Beim reinen Durchsatz war die TPU doppelt so schnell wie eine GPU, bei der Energieeffizienz fünfzehnmal so gut. Das ist der eigentliche Grund für Spezialbausteine.

Der Entwurf verzichtete auf fast alles, was ein Prozessor sonst hat. Keine Sprungvorhersage, keine Umordnung, keine Caches im üblichen Sinn, kein Mehrfachfaden. Statt dessen 65 536 Multiplizierer und ein großer, vom Programm ausdrücklich verwalteter Zwischenspeicher.

NORMAN JOUPPI *und Mitautoren beschrieben das Ergebnis 2017 in einem Aufsatz, der zu den meistzitierten der Rechnerarchitektur wurde*[13].

Der Aufsatz enthält auch eine Feststellung, die für dieses Kapitel wichtiger ist als die Zahlen: Die TPU war zum Zeitpunkt der Veröffentlichung bereits durch die Speicherbandbreite begrenzt, nicht durch die Rechenwerke. Man hatte so viele Multiplizierer gebaut, dass der Speicher nicht nachkam.

Aufgabe 12.1.

(a) Bestimmen Sie, wann sich eine GPU lohnt und wann nicht.
(b) Untersuchen Sie die Kosten der Datenübertragung zwischen CPU und GPU.
(c) Diskutieren Sie, warum sich Spezialbausteine durchsetzen und was das kostet.

Lösung Schritt für Schritt.

(a) Wann sich eine GPU lohnt.

Die Bedingungen.

Viel Parallelität	Mindestens Zehntausende unabhängige Arbeitseinheiten
Gleichförmigkeit	Alle Einheiten führen im Wesentlichen denselben Code aus
Regelmäßige Zugriffe	Benachbarte Fäden lesen benachbarte Daten
Hohe Intensität	Genug Rechnung je übertragenem Byte

Vier Beispiele.

Aufgabe	**Gewinn**	**Grund**
Matrixmultiplikation	50–100	alle vier Bedingungen
Bildfilter	20–50	
Sortieren	3–8	Zugriffe unregelmäßig
Graphdurch-musterung	1–3	Divergenz, wahlfreie Zugriffe
Übersetzerbau	< 1	**gar nicht parallel**

Die letzten beiden Zeilen sind wichtig. Es gibt viele Aufgaben, bei denen eine GPU nichts bringt oder schadet, und die verbreitete Annahme, alles ließe sich beschleunigen, ist falsch.

Die Rechnung für die Graphdurchmusterung.
Eine Breitensuche greift über die Nachbarschaftslisten wahlfrei auf den Speicher zu, und die Zahl der Nachbarn schwankt stark.

Größe	Wert	Folge
Divergenz	hoch	Wirkungsgrad $\approx 20\,\%$
Zugriffe zusammenhängend	selten	Bandbreite $\approx 15\,\%$
effektiv nutzbar	$\approx 3\,\%$	von der Spitzenleistung

Bei einer GPU mit fünfzigfacher Spitzenleistung bleibt davon Faktor 1,5. Der Aufwand für die Portierung lohnt sich meist nicht.

(b) Die Übertragung.
Die Zahlen.

Verbindung	Bandbreite	Latenz
GPU-Speicher intern	3 TB/s	300 ns
PCIe 5.0 (16 Bahnen)	64 GB/s	≈ 2 µs
NVLink	900 GB/s	≈ 1 µs
CPU-Hauptspeicher	100 GB/s	80 ns

Die zweite Zeile ist bei üblichen Systemen der Engpass. Der GPU-Speicher ist fünfzigmal schneller als die Verbindung dorthin.

Die Rechnung für eine Vektoraddition.
$c[i] = a[i] + b[i]$ für $n = 10^8$ Elemente zu 4 Byte.

Schritt	Daten	Zeit
Hinüberkopieren	800 MB	12,5 ms
Rechnen	—	0,4 ms
Zurückkopieren	400 MB	6,3 ms
gesamt		**19,2 ms**
auf der CPU allein		12,0 ms

Die GPU ist hier langsamer als die CPU, obwohl sie dreißigmal schneller rechnet. Die Rechnung dauert zwei Prozent der Gesamtzeit; alles Übrige ist Transport.

Die Bedingung, unter der es sich lohnt.

$$\frac{\text{Rechenzeit auf der CPU}}{\text{Übertragungszeit}} > 1 + \frac{1}{\text{Beschleunigung}}.$$

Praktisch heißt das: Die Daten müssen auf der GPU oft genug angefasst werden. Bei einer Matrixmultiplikation mit $2n^3$ Operationen auf $3n^2$ Daten wächst dieses Verhältnis mit n, und ab etwa $n = 500$ lohnt sich die Übertragung.

Die praktische Konsequenz lautet: Daten so lange wie möglich auf der GPU lassen und ganze Rechenketten dort ausführen, statt für jeden Schritt hin und her zu kopieren.

Bei Apples M-Prozessoren und ähnlichen Entwürfen entfällt das Problem, weil CPU und GPU denselben Speicher benutzen. Man spricht von **einheitlichem Speicher**, und es ist einer der wesentlichen Vorteile dieser Bauart.

(c) Spezialbausteine und ihr Preis.
Warum sie sich durchsetzen.
Kapitel 13 wird zeigen, dass die Energie je Operation die begrenzende Größe geworden ist. Ein Spezialbaustein spart genau dort.

Wofür Energie draufgeht	**Anteil (CPU)**	**bei Spezialbaustein**
eigentliche Rechnung	$\approx 5\,\%$	$\approx 60\,\%$
Befehle holen und dekodieren	$\approx 25\,\%$	entfällt
Register und Umordnung	$\approx 30\,\%$	entfällt
Caches	$\approx 25\,\%$	stark reduziert
Sonstiges	$\approx 15\,\%$	

Ein allgemeiner Prozessor gibt etwa zwanzig Mal so viel Energie für die Verwaltung einer Operation aus wie für die Operation selbst. Genau diesen Aufschlag spart man ein.

Was es kostet.

Entwicklungszeit	Zwei bis vier Jahre von der Idee bis zum Chip
Unbeweglichkeit	Ändert sich der Algorithmus, ist der Baustein wertlos
Programmierung	Jeder Baustein braucht eigene Werkzeuge
Fläche	Ein ungenutzter Baustein kostet trotzdem Platz

Die zweite Zeile ist das eigentliche Risiko. Ein Videokodierer für H.264 wurde durch H.265 entwertet, und Bausteine für bestimmte neuronale Netzarchitekturen sind schnell veraltet.

Die Rückkehr eines alten Problems

Ein Chip mit zwanzig verschiedenen Spezialbausteinen ist im Grunde zwanzig verschiedene Rechner. Für jeden braucht es einen Übersetzer, eine Bibliothek und ein Programmiermodell.

> *Das ist genau die Lage vor 1964, als jeder Rechner sein eigenes Gerät war. Die stabile Schnittstelle, die die System/360 eingeführt hat, geht auf der Ebene der Beschleuniger wieder verloren.*

Versuche	OpenCL, SYCL, oneAPI, MLIR und andere Zwischenschichten
Stand	Keine hat sich allgemein durchgesetzt; die herstellereigenen Werkzeuge sind meist schneller

Nvidias CUDA hat seine Stellung wesentlich dadurch behauptet, dass es eine stabile Schnittstelle über viele Chipgenerationen hinweg bietet. Der technische Vorsprung der Hardware wird oft überschätzt, der Vorsprung des Werkzeugkastens unterschätzt.

Kirk und Hwu[14] sind das Standardlehrbuch für GPU-Programmierung; Jouppi und Mitautoren[13] beschreiben die TPU vollständig; Hennessy und Patterson[9] begründen in ihrer Turing-Rede, warum sie die Zukunft in Spezialbausteinen sehen.

Part V

Wo es hakt

13 Energie als Grenze

Warum 2005 etwas endete

Die Verlustleistung eines CMOS-Schaltkreises setzt sich aus zwei Anteilen zusammen.

$$P = \underbrace{\alpha\, C\, V^2 f}_{\text{Schalten}} + \underbrace{V\, I_{\text{leck}}}_{\text{Leckstrom}} .$$

α	Anteil der Transistoren, die je Takt schalten
C	Kapazität, die dabei umgeladen wird
V	Betriebsspannung
f	Taktfrequenz

Dennards Skalierung.
ROBERT DENNARD beschrieb 1974, was geschieht, wenn man alle Abmessungen um den Faktor k verkleinert.

Größe	**Faktor**	**Folge**
Länge, Breite	$1/k$	k^2-mal so viele Transistoren
Kapazität C	$1/k$	
Spannung V	$1/k$	
Frequenz f	k	schneller
Leistung je Transistor	$\mathbf{1/k^2}$	
Leistung je Fläche	**1**	**unverändert**

Die letzte Zeile ist das Bemerkenswerte. Solange man die Spannung mitverkleinerte, konnte man doppelt so viele Transistoren bei höherem Takt unterbringen, ohne dass der Chip heißer wurde. Dreißig Jahre lang funktionierte das.

Warum es aufhörte.
Die Spannung lässt sich nicht beliebig senken. Unterhalb von etwa einem Volt wird der Leckstrom dominant, weil die Schwellspannung des Transistors nicht mitsinken kann, ohne dass er sich nicht mehr abschalten lässt.

Jahr	Struktur	Spannung	Leckanteil
1995	350 nm	3,3 V	$< 1\,\%$
2001	130 nm	1,5 V	$\approx 10\,\%$
2005	65 nm	1,1 V	$\approx 30\,\%$
2026	2 nm	0,65 V	$\approx 40\,\%$

Seit 2005 sinkt die Spannung nur noch langsam. Damit steigt die Leistung je Fläche bei jeder Verkleinerung, und das setzt sowohl der Taktfrequenz als auch der Zahl gleichzeitig aktiver Transistoren eine Grenze.

Dunkles Silizium

HADI ESMAEILZADEH und Mitautoren rechneten 2011 aus, was daraus folgt. Wenn die Transistordichte weiter steigt, die Leistung je Transistor aber nicht mehr entsprechend sinkt, so lässt sich nur noch ein Teil des Chips gleichzeitig betreiben. Den Rest muss man abgeschaltet lassen.

Knoten	nutzbar	dunkel
45 nm	100 %	0 %
32 nm	≈ 80 %	20 %
22 nm	≈ 50 %	50 %
8 nm	≈ 25 %	**75** %

Ihre Vorhersage lautete, dass die Leistungssteigerung durch Mehrkernigkeit bei etwa dem Achtfachen enden würde statt bei dem Dreißigfachen, das die Transistorzahl erlaubt hätte.

Die Vorhersage traf im Wesentlichen zu. Sie erklärt auch, warum Spezialbausteine attraktiv wurden. Wenn ohnehin drei Viertel des Chips abgeschaltet sein müssen, kann man dort auch etwas hinbauen, das nur selten gebraucht wird, dann aber sehr effizient arbeitet.

Vorher	Fläche war knapp, Energie reichlich. Man baute allgemeine Rechenwerke und lastete sie aus
Seit etwa 2010	Energie ist knapp, Fläche reichlich. Man baut viele Spezialbausteine und schaltet die meisten ab

Diese Umkehrung ist die wichtigste Entwicklung der letzten fünfzehn Jahre in der Rechnerarchitektur, und sie erklärt, warum ein heutiger Mobilprozessor aus Dutzenden verschiedener Bausteine besteht.

Wo die Energie tatsächlich hingeht

Die folgenden Zahlen stammen aus Messungen an 45-nm-Technik und werden seit MARK HOROWITZ regelmäßig zitiert. Die absoluten Werte sind heute kleiner, die Verhältnisse ähnlich.

Operation	Energie	relativ
Ganzzahladdition, 8 Bit	0,03 pJ	1
Ganzzahladdition, 32 Bit	0,1 pJ	3
Gleitkommaaddition, 32 Bit	0,9 pJ	30
Gleitkommamultipl., 32 Bit	3,7 pJ	123
Register lesen	0,1 pJ	3
L1-Cache (8 KB)	10 pJ	333
L3-Cache (1 MB)	100 pJ	3300
DRAM	**1300 pJ**	**43 000**

Ein Zugriff auf den Hauptspeicher kostet so viel Energie wie über vierzigtausend Ganzzahladditionen. Das ist die wichtigste Zahl dieses Kapitels.

Daraus folgt eine Entwurfsregel, die der klassischen Sichtweise widerspricht: Es lohnt sich, mehrfach zu rechnen, um einen Speicherzugriff zu sparen.

Früher	Zwischenergebnisse speichern und wiederverwenden
Heute	Zwischenergebnisse oft neu berechnen, wenn sie nicht mehr im Cache stehen

*Bei Grafikprogrammen und beim Training neuronaler Netze ist das gängige Praxis. Man nennt es dort **Rematerialisierung** oder **gradient checkpointing** und spart erhebliche Mengen Speicher, indem man Teile der Rechnung wiederholt.*

Aufgabe 13.1.

(a) Rechnen Sie aus, wie sich Taktabsenkung und Kernzahl auf die Leistungsaufnahme auswirken.

(b) Bestimmen Sie den Energiebedarf einer Matrixmultiplikation und wo er hingeht.

(c) Untersuchen Sie, was Programmierer zum Energieverbrauch beitragen können.

Lösung Schritt für Schritt.

(a) Takt gegen Kerne.

Die Ausgangslage. Ein Kern bei f und V leiste P und erbringe die Rechenleistung 1.

Fall 1 – Takt verdoppeln.

Um doppelt so schnell zu takten, braucht man ungefähr die anderthalbfache Spannung.

$$P_{\text{neu}} = C\,(1{,}5V)^2\,(2f) = 4{,}5\,C\,V^2 f = 4{,}5\,P.$$

Doppelte Leistung für die viereinhalbfache Verlustleistung.

Fall 2 – Zwei Kerne bei gleichem Takt.

$$P_{\text{neu}} = 2\,P.$$

Doppelte Leistung für die doppelte Verlustleistung, sofern das Programm parallel läuft.

Fall 3 – Zwei Kerne bei $0{,}8f$ und $0{,}8V$.

$$P_{\text{neu}} = 2\,C\,(0{,}8V)^2(0{,}8f) = 2 \cdot 0{,}512\,P = 1{,}02\,P.$$

$$\text{Rechenleistung} = 2 \cdot 0{,}8 = 1{,}6.$$

Fall	Leistung	Verbrauch	je Watt
Ausgangslage	1,0	1,0	1,00
doppelter Takt	2,0	4,5	0,44
zwei Kerne	2,0	2,0	1,00
zwei Kerne, gedrosselt	1,6	1,0	**1,60**

Die letzte Zeile ist der Grund für die Wende von 2005. Zwei gedrosselte Kerne liefern bei gleichem Verbrauch sechzig Prozent mehr Rechenleistung als ein hochgetakteter. Vorausgesetzt, das Programm nutzt beide.

Die Kehrseite.
Diese Rechnung gilt nur bei $p = 1$ in Amdahls Formel. Bei $p = 0{,}8$ leisten zwei gedrosselte Kerne

$$0{,}8 \cdot \frac{1}{0{,}2 + \frac{0{,}8}{2}} = 0{,}8 \cdot 1{,}67 = 1{,}33,$$

also weniger als der doppelte Takt mit 2,0. Für seriellen Code ist die Wende ein Rückschritt gewesen.

(b) Energie einer Matrixmultiplikation.
Der Fall. $n = 1024$, doppelte Genauigkeit, naive Fassung.
Schritt 1 – Die Rechenoperationen.

$$2n^3 = 2{,}1 \cdot 10^9 \text{ Gleitkommaoperationen.}$$

$$E_{\text{Rechnung}} = 2{,}1 \cdot 10^9 \cdot 5 \text{ pJ} = 10{,}7 \text{ mJ}.$$

(5 pJ als grober Wert für eine kombinierte Multiplikation-Addition in doppelter Genauigkeit)
Schritt 2 – Die Speicherzugriffe, naiv.
Aus Kapitel 8: etwa $1{,}2 \cdot 10^9$ Fehlgriffe auf den Hauptspeicher, jeder holt 64 Byte.

$$E_{\text{Speicher}} = 1{,}2 \cdot 10^9 \cdot 1300 \text{ pJ} = 1560 \text{ mJ}.$$

Anteil	Energie	Anteil
Rechnung	10,7 mJ	0,7 %
Speicher	1560 mJ	**99,3** %

Über neunundneunzig Prozent der Energie gehen für den Datentransport drauf, nicht für die Rechnung. Bei der naiven Fassung ist die Multiplikation praktisch umsonst.

Schritt 3 – Mit Blockbildung.
Die Fehlgriffe sinken auf $1{,}3 \cdot 10^7$.

$$E_{\text{Speicher}} = 1{,}3 \cdot 10^7 \cdot 1300\ \text{pJ} = 17\ \text{mJ}.$$

Fassung	Rechnung	Speicher	gesamt
naiv	10,7 mJ	1560 mJ	1571 mJ
geblockt	10,7 mJ	17 mJ	**28 mJ**

$$\text{Ersparnis} = \frac{1571}{28} = 56\text{-fach}.$$

Die Blockbildung aus Kapitel 8 spart also nicht nur Zeit, sondern in ähnlichem Maße auch Energie. Beides hat dieselbe Ursache, nämlich weniger Datenbewegung.

(c) Was Programmierer beitragen können.
Die Rangfolge.

Maßnahme	Wirkung	Aufwand
Besserer Algorithmus	**bis** 1000**-fach**	unterschiedlich
Datenbewegung senken	10–50-fach	mittel
Vektorisieren	2–8-fach	mittel
Kürzere Zahlentypen	2–4-fach	gering
Sprache wechseln	2–50-fach	hoch

Zur vierten Zeile.
Eine Gleitkommazahl mit 32 Bit kostet etwa ein Viertel der Energie einer mit 64 Bit, weil sowohl die Rechnung als auch der Transport günstiger sind. Bei Anwendungen, die keine hohe Genauigkeit brauchen, ist das ein einfacher Gewinn.

Typ	Bits	Energie	Bandbreite
`double`	64	1,00	1,00
`float`	32	0,26	0,50
`bfloat16`	16	0,07	0,25
`int8`	8	0,01	0,125

Die dritte Zeile hat das maschinelle Lernen geprägt. Das Format `bfloat16` behält den Exponentenbereich von `float` und opfert Mantissenbits, weil sich das für Training als tragfähig erwiesen hat.

Zur fünften Zeile.
Eine Untersuchung von PEREIRA und Mitautoren (2017) verglich Laufzeit und Energieverbrauch von 27 Programmiersprachen an denselben Aufgaben.

Sprache	Energie	Zeit	Speicher
C	1,00	1,00	1,17
Rust	1,03	1,04	1,54
C++	1,34	1,56	1,34
Java	1,98	1,89	6,01
Go	3,23	2,83	1,05
JavaScript	4,45	6,52	4,59
Python	**75,88**	**71,90**	2,80

Man sollte diese Tabelle mit Vorsicht lesen. Sie misst rechenintensive Aufgaben in reiner Sprachimplementierung. Python-Programme, die ihre Arbeit an NumPy oder eine Datenbank abgeben, verhalten sich völlig anders.

Für die Praxis ist die Auskunft trotzdem nützlich: Wo tatsächlich in der Sprache gerechnet wird, ist der Unterschied groß. Wo die Sprache nur koordiniert, spielt er keine Rolle.

Die unangenehmste Zahl

Ein Rechenzentrum verbraucht heute etwa so viel Energie für die Kühlung wie für die Rechner. Der Kennwert dafür heißt PUE, und gute Anlagen erreichen 1,1 bis 1,2.

> *Jedes eingesparte Joule im Chip spart also etwa* 1,15 *Joule an der Steckdose. Und da die Energie überwiegend in die Datenbewegung geht, ist ein cachebewusst geschriebenes Programm auch ein energiesparendes.*

Der weltweite Stromverbrauch von Rechenzentren lag 2024 bei ungefähr 1,5 Prozent des gesamten Stromverbrauchs, mit deutlich steigender Tendenz durch das Training großer Modelle. Genaue Zahlen sind schwer zu bekommen und in der Literatur umstritten.

Esmaeilzadeh und Mitautoren[5] haben das dunkle Silizium ausgerechnet; Hennessy und Patterson[9] ziehen daraus die Folgerungen für den Chipentwurf; Hill und Marty[11] behandeln die Mehrkernseite.

14 Wenn die Abstraktion undicht wird

Was der Befehlssatz verschweigt

Kapitel 2 hat festgehalten, was ein Befehlssatz zusichert und was nicht. Die Zeit gehört zu dem, was er nicht zusichert.

> *Solange man Programme nur auf Korrektheit prüft, ist das folgenlos. Sobald man prüfen will, ob ein Programm etwas verrät, wird es entscheidend.*

Ein Angreifer, der die Ausführungszeit misst, sieht Dinge, die im Befehlssatz nicht vorkommen: welche Daten im Cache liegen, welche Sprünge vorhergesagt wurden, welche Rechenwerke belegt waren.

Kanal	beobachtbar	verrät
Cache	Zugriffszeit	welche Adressen berührt wurden
Sprungvorhersage	Zeit nach Verzweigung	welchen Weg der Code nahm
Rechenwerke	Durchsatz	welche Operationen liefen
Energie	Stromaufnahme	Schaltaktivität

Solche Angriffe sind seit den 1990er Jahren bekannt. PAUL KOCHER zeigte 1996, dass sich aus der Laufzeit einer RSA-Entschlüsselung der private Schlüssel gewinnen lässt.

> *Neu an Spectre und Meltdown war, dass sie nicht nur die Zeit* beobachten, *sondern die Maschine dazu bringen, Dinge zu tun, die im Programm gar nicht stehen.*

Meltdown

Der Angriff in vier Zeilen.

```
char *kern = 0xFFFF800000000000;   /* Kernspeicher */
char puffer[256*4096];
char geheim = *kern;      löst eine Ausnahme aus
char x = puffer[geheim * 4096];     wird trotzdem ausgeführt
```

Warum das funktionierte.
Die dritte Zeile ist unzulässig, und der Prozessor bemerkt das. Aber er bemerkt es *spät*, nämlich erst beim Rückordnen (Kapitel 6).

1. Der Ladebefehl wird spekulativ ausgeführt und liefert den Wert, weil die Rechteprüfung parallel läuft und noch nicht fertig ist.
2. Der vierte Befehl benutzt diesen Wert als Index und lädt eine Cachezeile.
3. Der Rückordnungspuffer stellt die Rechteverletzung fest und verwirft beide Befehle.
4. **Die geladene Cachezeile bleibt.**

*Der Angreifer misst danach, welche der 256 möglichen Zeilen schnell zugänglich ist. Daraus liest er das geheime Byte ab. Das Verfahren heißt **Flush and Reload** und stammt aus der Seitenkanalforschung.*

Zeile	Zugriffszeit	Schluss
puffer$[0 \cdot 4096]$	250 Takte	nicht im Cache
…	250 Takte	
puffer$[83 \cdot 4096]$	**45 Takte**	**geheim** $= 83$
…	250 Takte	

Die Ausleserate lag bei etwa 500 KB je Sekunde. Damit lässt sich der gesamte Kernspeicher in überschaubarer Zeit auslesen, einschließlich aller Passwörter und Schlüssel, die dort liegen.

Die Lücke betraf praktisch alle Intel-Prozessoren seit 1995 und einige ARM-Kerne. AMD war nicht betroffen, weil dort die Rechteprüfung früher erfolgt.

Die Abhilfe.
Man trennt die Seitentabellen für Kern und Anwendung vollständig, sodass die Kernadressen im Benutzerbetrieb gar nicht abgebildet sind. Das Verfahren heißt KPTI und kostet bei jedem Systemaufruf einen Wechsel der Seitentabelle.

Last	Verlust	Bemerkung
rechenintensiv	$< 1\,\%$	kaum Systemaufrufe
Dateizugriffe	5–15 %	
Datenbank	10–25 %	viele Aufrufe
Netzdienste	15–30 %	

Seit etwa 2019 lösen neuere Prozessoren das Problem in Hardware, indem sie die Rechteprüfung vorziehen. Der Verlust entfällt dann.

Spectre

Meltdown war ein Fehler in der Ausführungsreihenfolge und ließ sich beheben. Spectre nutzt die Sprungvorhersage selbst aus und ist deshalb grundsätzlicher.
Variante 1: Grenzprüfung umgehen.

```
if (i < feldlaenge) {
  char wert = feld[i];
  char x = puffer[wert * 4096]; }
```

Dieser Code ist korrekt. Er prüft die Grenze, bevor er zugreift.

1. Der Angreifer ruft die Funktion viele Male mit gültigen Indizes auf. Der Vorhersager lernt, dass die Bedingung wahr ist.
2. Dann ruft er sie mit einem sehr großen i auf.
3. Der Vorhersager rät „wahr“, und die Maschine liest spekulativ außerhalb des Feldes.
4. Die Grenzprüfung schlägt fehl, alles wird verworfen. **Die Cachezeile bleibt.**

Das Programm ist fehlerfrei. Der Angriff nutzt aus, dass die Maschine Code ausführt, den das Programm nie ausgeführt hätte, und dass diese Ausführung Spuren hinterlässt.

Variante 2: Den Vorhersager vergiften.
Der Sprungzielpuffer ist zwischen Prozessen nicht immer getrennt. Ein Angreifer kann Einträge so setzen, dass ein indirekter Sprung im Opferprozess spekulativ an eine von ihm gewählte Stelle springt.

*Dort sucht er sich ein Codestück, das ein Geheimnis lädt und in den Cache abbildet. Man nennt solche Stücke **Gadgets**, in Anlehnung an eine ältere Angriffsform.*

Warum sich das nicht sauber beheben lässt.

Spekulation abschalten	Kostet Faktor 3 bis 5 an Leistung. Nicht hinnehmbar
Cache zurücksetzen	Man müsste bei jeder verworfenen Spekulation den Cachezustand wiederherstellen. Sehr aufwendig
Einzelne Stellen absichern	Möglich, aber man muss alle finden

Der dritte Weg wird gegangen. An kritischen Stellen fügt man Befehle ein, die die Spekulation anhalten, etwa `lfence` auf x86.

Maßnahme	**Kosten**	**Wirkung**
`lfence` an kritischen Stellen	lokal hoch	wirksam, wenn vollständig
Retpoline	5–20 %	gegen Variante 2
Übersetzerhärtung	2–10 %	automatisch, unvollständig
Vorhersager trennen	gering	seit 2019 in Hardware

Stand August 2026 gilt Spectre als eingedämmt, nicht als behoben. Seit 2018 erschienen regelmäßig neue Varianten, und die Fachwelt geht davon aus, dass weitere folgen werden.

Der 3. Januar 2018

Die Lücken wurden im Sommer 2017 unabhängig von mehreren Gruppen gefunden. JANN HORN von Googles Project Zero meldete sie am 1. Juni 2017 an Intel, ARM und AMD. Fast gleichzeitig fanden Gruppen in Graz, in Adelaide, bei Cyberus und bei Rambus dasselbe.
Man vereinbarte eine gemeinsame Veröffentlichung am 9. Januar 2018, um den Herstellern Zeit für Gegenmaßnahmen zu geben.

> *Der Plan zerbrach, weil aufmerksame Beobachter aus den Änderungen am Linux-Kern schlossen, was vorging. Ende Dezember fielen ungewöhnlich umfangreiche Umbauten an der Seitentabellenverwaltung auf, die als dringend markiert und bis in ältere Versionen zurückportiert wurden.*

Am 2. Januar 2018 veröffentlichte *The Register* einen Artikel darüber. Am 3. Januar gaben die Forschergruppen ihre Ergebnisse vorzeitig frei.

Die beiden Lücken bekamen Namen und Logos, was damals neu war und seither Schule gemacht hat.

Die Reaktion war ungewöhnlich. Innerhalb von Tagen erschienen Aktualisierungen für alle verbreiteten Betriebssysteme, Browser reduzierten die Auflösung ihrer Zeitmessfunktionen, und Cloudanbieter starteten ihre Systeme neu.

> *Bemerkenswert an der Sache ist, dass niemand einen Fehler gemacht hatte. Die spekulative Ausführung war seit 1995 dokumentiert, richtig implementiert und funktionierte wie vorgesehen. Der Fehler lag in der Annahme, dass verworfene Arbeit keine Spuren hinterlässt.*

PAUL KOCHER, einer der Entdecker, formulierte es so: Die Rechnerarchitektur habe vierzig Jahre lang Leistung gegen Sicherheit eingetauscht, ohne dass jemand den Tausch bemerkt hätte.

Aufgabe 14.1.

(a) Analysieren Sie den Cache-Seitenkanal quantitativ.
(b) Untersuchen Sie, was konstantzeitige Programmierung bedeutet.
(c) Diskutieren Sie, was aus dem Vorfall für den Entwurf folgt.

Lösung Schritt für Schritt.

(a) Der Seitenkanal.
Das Grundverfahren.

1. Der Angreifer verdrängt alle Zeilen seines Puffers aus dem Cache (`clflush`).
2. Er löst die spekulative Ausführung aus.
3. Er misst die Zugriffszeit auf jede der 256 Zeilen.

Warum der Abstand 4096 Byte beträgt.
Bei kleinerem Abstand würde das Vorabholen der Hardware benachbarte Zeilen mitladen und die Messung verfälschen. Eine Seitengröße Abstand verhindert das zuverlässig.

> *Der Puffer ist deshalb* $256 \cdot 4096 = 1$ *MB groß, obwohl er nur 256 unterscheidbare Zustände tragen muss.*

Die Messgenauigkeit.

Fall	Takte	Streuung
im L1	45	± 5
im Hauptspeicher	250	± 40

Die Verteilungen überlappen kaum, also genügt eine einzige Messung meist. Bei Störungen wiederholt man und nimmt den Median.
Die Ausleserate.

Schritt	Zeit	Bemerkung
Puffer leeren	$256 \cdot 100$ Takte	
Spekulation auslösen	≈ 200 Takte	
256 Zeilen messen	$256 \cdot 250$ Takte	meist Fehlgriffe
je Byte	$\approx 9 \cdot 10^4$ Takte	
Rate bei 3 GHz	≈ 33 KB/s	einfache Fassung

Mit Optimierungen, etwa dem gleichzeitigen Auslesen mehrerer Bits, erreichten die Entdecker etwa 500 KB je Sekunde. Bei dieser Rate braucht das Auslesen von 8 GB Kernspeicher etwa fünf Stunden.
Die naheliegende Gegenmaßnahme und ihre Grenze.
Browser reduzierten die Auflösung ihrer Zeitfunktionen von Nanosekunden auf Millisekunden und fügten Rauschen hinzu.

Das half kurzfristig. Man kann sich aber eine eigene Uhr bauen, indem man einen Faden in einer Schleife einen gemeinsamen Zähler erhöhen lässt. Die Auflösung dieser Uhr ist so gut wie die Speicherlatenz.

Deshalb schränkten Browser zusätzlich den gemeinsamen Speicher zwischen Fäden ein. Das brach eine Reihe von Anwendungen und wurde erst später mit zusätzlichen Sicherheitsanforderungen wieder freigegeben.

(b) Konstantzeitige Programmierung.
Die Regel. Weder die Ausführungszeit noch das Zugriffsmuster darf von geheimen Daten abhängen.

Verboten	Verzweigung, deren Bedingung vom Geheimnis abhängt
Verboten	Feldindex, der vom Geheimnis abhängt
Verboten	Division oder andere datenabhängige Befehle
Erlaubt	Arithmetik und Logik auf ganzen Registern

Ein Beispiel: Vergleich zweier Schlüssel.

```
/* unsicher:  bricht beim ersten
Unterschied ab */
for (i=0;i<n;i++) if (a[i]!=b[i])
return 0;
return 1;
```

```
/* sicher:  läuft immer durch */
unsigned d = 0;
for (i=0;i<n;i++) d |= a[i] ^ b[i];
return (d == 0);
```

Die erste Fassung verrät durch ihre Laufzeit, wie viele Bytes übereinstimmen. Ein Angreifer kann damit einen Schlüssel Byte für Byte erraten, statt alle Möglichkeiten durchzuprobieren.

Aus 2^{128} Versuchen werden $16 \cdot 256 = 4096$. Genau dieser Fehler steckte 2009 in mehreren Bibliotheken zur Prüfung von Nachrichtenauthentifizierungscodes.

Das Problem mit dem Übersetzer.
Die sichere Fassung ist nur sicher, solange der Übersetzer sie nicht umschreibt. Er darf die Schleife abbrechen, sobald $d \neq 0$ ist, weil das Ergebnis feststeht.

Übliche Abhilfe	Die kritischen Teile in Assembler schreiben
Neuere Ansätze	Sprachen mit einem Typ für geheime Werte, etwa in `HACL*` oder `Jasmin`
Prüfwerkzeuge	`ctgrind`, `dudect` messen, ob die Zeit datenabhängig ist

Die zweite Zeile ist der aussichtsreichste Weg. Wenn die Sprache weiß, welche Werte geheim sind, kann der Übersetzer Umformungen unterlassen, die die Zeit beeinflussen.

(c) Was daraus für den Entwurf folgt.
Erstens: Schnittstellen müssen über Zeit reden.
Kapitel 2 hat die neuen Befehlssatzerweiterungen erwähnt, die datenunabhängige Zeiten zusichern. Das ist ein grundsätzlicher Wandel.

> *Sechzig Jahre lang war es ein Vorzug, dass der Befehlssatz nichts über Zeit sagt. Er konnte dadurch stabil bleiben, während sich die Implementierung völlig änderte. Diese Freiheit ist jetzt teilweise aufgegeben.*

Zweitens: Isolation braucht mehr als Adressräume.

Bisher geteilt	Caches, Sprungvorhersager, TLB, Rechenwerke
Neue Praxis	Trennung nach Prozess, Rücksetzen beim Wechsel, getrennte Vorhersagestrukturen

Bei Cloudanbietern ging man weiter und verzichtet auf gemeinsame Nutzung physischer Kerne durch verschiedene Kunden. Das kostet etwa die Hälfte der Kapazität, die durch gleichzeitiges Mehrfädenrechnen gewonnen wurde.
Drittens: Der Preis der Leistung wurde sichtbar.

Mechanismus	Gewinn	Sicherheitskosten
Caches	50-fach	Seitenkanal
Spekulation	3–5-fach	Spectre
Mehrfädenrechnen	1,3-fach	geteilte Ressourcen
Umordnung	2-fach	Meltdown

Jede Zeile beschreibt einen Mechanismus, der Leistung bringt, indem er Zustand über Grenzen hinweg teilt oder erhält. Und genau daraus entsteht der Kanal.

Der Zusammenhang mit Band 0

In Band 0 (Kapitel 12) steht ein Gesetz von JOEL SPOLSKY: Alle nichttrivialen Abstraktionen sind in gewissem Maße durchlässig.
Dort ging es um Leistung. Ein Programm, das die Abstraktion glaubt, läuft langsam. Das ist ärgerlich und behebbar.

> *Spectre zeigt die schärfere Fassung. Ein Programm, das die Abstraktion glaubt, ist* unsicher. *Und das ist nicht durch Messen und Nachbessern zu beheben, sondern nur dadurch, dass man die Schicht darunter versteht.*

Genau deshalb steht dieses Kapitel in einem Band über Rechnerarchitektur und nicht in einem über Sicherheit.

Kocher und Mitautoren[15] sowie Lipp und Mitautoren[16] sind die Originalarbeiten und beide gut lesbar; Hennessy und Patterson[9] behandeln die Folgen für den Entwurf; Fog[6] dokumentiert, welche Prozessoren welche Gegenmaßnahmen enthalten.

Part VI

Aktueller Stand

15 Wo die Architektur heute steht

Was sich in zwanzig Jahren geändert hat

Größe	2005	2026	Faktor
Taktfrequenz	3,8 GHz	5,5 GHz	1,4
Kerne	1–2	16–128	≈ 50
IPC	≈ 1,5	≈ 3	2
Vektorbreite	128 Bit	512 Bit	4
L3-Cache	2 MB	96 MB	48
Speicherbandbreite	8 GB/s	100 GB/s	12
Speicherlatenz	≈ 70 ns	≈ 65 ns	**1,1**

Die letzte Zeile ist die interessanteste. Die Speicherlatenz hat sich in zwanzig Jahren praktisch nicht verbessert. Alles andere wurde schneller, das Warten auf den Hauptspeicher nicht.

Deshalb wachsen die Caches so stark, deshalb wird das Befehlsfenster größer, deshalb gibt es Vorabholer. Der überwiegende Teil des Aufwands in einem heutigen Prozessorkern dient dazu, diese eine Zahl zu verstecken.

Vier Entwicklungen

Erstens: Chiplets.
Große Chips lassen sich nicht mehr wirtschaftlich fertigen, weil die Ausbeute mit der Fläche sinkt. Man baut deshalb mehrere kleinere Chips und verbindet sie im Gehäuse.

Ansatz	Ausbeute	Bemerkung
ein großer Chip	≈ 30 %	bei 600 mm^2
acht Chiplets	≈ 85 %	je 75 mm^2

AMD führte das 2019 im großen Maßstab ein, Intel folgte 2023. Der Preis ist eine höhere Latenz zwischen den Chiplets, die sich im Speicherzugriff bemerkbar macht.
Zweitens: Speicher näher heranholen.

Technik	Bandbreite	Einsatz
DDR5	100 GB/s	Arbeitsplatzrechner
HBM3	3 TB/s	Beschleuniger
Cache auf dem Chip gestapelt	—	AMD 3D V-Cache

Die dritte Zeile ist ein bemerkenswerter Fall. AMD stapelt einen zusätzlichen L3-Cache senkrecht auf den Rechenchip und verbindet beide durch Durchkontaktierungen. Bei Spielen bringt das bis zu 30 Prozent, bei numerischen Anwendungen kaum etwas.
Drittens: Asymmetrische Kerne.

	Leistungskern	Effizienzkern	Verhältnis
Fläche	1,0	0,25	4
Leistung	1,0	0,55	1,8
Verbrauch	1,0	0,12	8

Vier Effizienzkerne brauchen so viel Fläche wie ein Leistungskern und liefern gemeinsam mehr als das Doppelte an Durchsatz, bei der Hälfte des Verbrauchs. Für parallele Lasten ist das die bessere Wahl, für serielle nicht.

Die Schwierigkeit liegt beim Betriebssystem, das entscheiden muss, welcher Faden wohin gehört. Das ging anfangs regelmäßig schief, insbesondere unter Windows bei Intels ersten asymmetrischen Prozessoren von 2021.
Viertens: Rückkehr des Befehlssatzes als Streitfrage.

Architektur	Anteil 2026	Entwicklung
x86-64	fallend	Arbeitsplatz, Server
ARM64	**stark steigend**	mobil, Arbeitsplatz, Server
RISC-V	steigend	eingebettet, zunehmend Server

Apples Wechsel zu ARM 2020 hat gezeigt, dass ein Architekturwechsel im Massenmarkt möglich ist. Amazon, Google und Microsoft entwerfen eigene ARM-Serverprozessoren.

Was offen ist

Frage	Status	Bemerkung
Speicherlatenz senken	ungelöst	seit 25 Jahren stagnierend
Spectre grundsätzlich beheben	ungelöst	folgt aus Spekulation
Parallele Programmierung	teilweise	für Fachleute lösbar
Einheitliches Programmiermodell für Beschleuniger	ungelöst	viele Versuche
Weiter als 2 nm	offen	Tunneleffekt

Zur ersten Zeile.
Die Latenz von DRAM wird durch Physik bestimmt: Ein Kondensator muss geladen, eine lange Leitung getrieben, ein Signal verstärkt werden. Seit den 1990er Jahren hat sich die Bandbreite verhundertfacht und die Latenz kaum verbessert.

> *Man kann mehr Daten gleichzeitig holen, aber nicht schneller mit dem Holen beginnen. Deshalb helfen Vorabholen und viele gleichzeitige Anfragen, und deshalb hilft nichts bei einer Kette abhängiger Zeigerzugriffe.*

Zur letzten Zeile.
Unterhalb von etwa 5 nm Kanallänge tunneln Elektronen durch die Sperrschicht, und der Transistor lässt sich nicht mehr sauber abschalten. Die Bezeichnungen „3 nm“ und „2 nm“ sind Generationsnamen und entsprechen keiner gemessenen Länge; die tatsächliche Gatelänge liegt bei etwa 16 nm (Band 0, Kapitel 15).

Gate-all-around	Seit 2022 in Fertigung; der Kanal wird vom Gate vollständig umschlossen
Rückseitige Stromversorgung	Ab 2025; Versorgungsleitungen auf der Chiprückseite
Neue Materialien	Forschung; Kohlenstoffnanoröhren, zweidimensionale Halbleiter

Die Fachwelt geht davon aus, dass die klassische Verkleinerung Anfang der 2030er Jahre endet. Was danach kommt, ist offen.

Zum Weiterlesen

Tanenbaum[23] ordnet den Rechner konsequent als Schichtenmodell, wie es Band 0 aufgegriffen hat. Wulf und McKee[27] sind auf vier Seiten immer noch die beste Darstellung des Grundproblems dieses Bandes. Williams, Waterman und Patterson[26] haben das Dachlinienmodell aus Kapitel 10 eingeführt.

Schlussgedanke: drei Einsichten

Erstens: Die Zeit geht in die Datenbewegung, nicht in die Rechnung.

Vorgang	Takte	Energie
Gleitkommamultiplikation	4	4 pJ
Hauptspeicherzugriff	250	1300 pJ
Verhältnis	**60**	**325**

Wer ein Programm schneller machen will, sollte zuerst fragen, wie viele Bytes es bewegt, und erst danach, wie viele Operationen es ausführt. Diese Reihenfolge ist gegenüber der klassischen Algorithmenanalyse umgekehrt.

Zweitens: Die Fiktion der sequentiellen Ausführung ist eine Leistung, kein Naturzustand.

> *Zwischen dem, was ein Programm zu tun scheint, und dem, was der Prozessor tut, liegt sehr viel Maschinerie. Sie funktioniert gut genug, dass man sie meist ignorieren kann.*

Sie gelingt	bei Korrektheit, fast immer
Sie gelingt teilweise	bei Laufzeit; man muss die Schicht darunter kennen
Sie misslingt	bei Sicherheit; Kapitel 14

Drittens: Messen schlägt Nachdenken.

> *Praktisch jede Vermutung darüber, wo ein Programm seine Zeit verbringt, ist falsch. Das gilt für Anfänger und für Fachleute, und die vorliegenden Untersuchungen zeigen, dass es auch für die Autoren von Leistungsmessungen gilt.*

Dieser Band gibt Modelle an die Hand, mit denen man Messwerte deuten kann. Er ersetzt das Messen nicht. Wer aus den Kapiteln über Caches und Pipelines ableitet, wo sein Programm langsam sein müsste, und danach optimiert, ohne zu prüfen, hat den Band missverstanden.

Die richtige Reihenfolge lautet: messen, verstehen, ändern, wieder messen.

Wir haben vierzig Jahre lang Leistung gegen Sicherheit eingetauscht, ohne zu bemerken, dass wir tauschen.

— sinngemäß PAUL KOCHER, 2018,
einer der Entdecker von Spectre.

Part VII

Anhang

A Übersicht und Zahlen

A.1 Die Zahlen, die man kennen sollte

Latenzen

Vorgang	Takte	Zeit	maßstäblich
Register	0–1	0,2 ns	1 s
L1-Cache	4	1 ns	4 s
L2-Cache	14	3 ns	15 s
L3-Cache	50	12 ns	1 min
Hauptspeicher	250	65 ns	4 min
Kohärenz, anderer Kern	150	40 ns	3 min
NUMA, anderer Sockel	600	150 ns	10 min
SSD, Lesezugriff	$4 \cdot 10^5$	100 µs	5 Tage
Festplatte	$4 \cdot 10^7$	10 ms	1,5 Jahre
Netz, Rechenzentrum	$2 \cdot 10^6$	500 µs	1 Monat
Netz, interkontinental	$6 \cdot 10^8$	150 ms	24 Jahre

(Größenordnungen für 2026, Takt 4 GHz; die rechte Spalte skaliert einen Registerzugriff auf eine Sekunde)

Befehlszeiten

Befehl	Latenz	Durchsatz	Energie
Ganzzahladdition	1	4/Takt	0,1 pJ
Ganzzahlmultiplikation	3	1/Takt	≈ 3 pJ
Ganzzahldivision	20–40	1/6 Takte	hoch
Gleitkommaaddition	4	2/Takt	0,9 pJ
Gleitkommamultipl.	4	2/Takt	3,7 pJ
FMA (kombiniert)	4	2/Takt	≈ 4 pJ
Gleitkommadivision	14	1/4 Takte	
Quadratwurzel	18	1/6 Takte	
Bedingter Sprung	1	2/Takt	
Fehlvorhersage	15–20	—	

(x86-Kerne der Jahre um 2025; die genauen Werte je Modell stehen bei Fog[6])

Bemerkenswert ist der Unterschied zwischen Latenz und Durchsatz. Bei unabhängigen Operationen zählt die rechte Spalte, bei einer Abhängigkeitskette die linke.

Kenngrößen und ihre Zielwerte

Größe	gut	bei schlechtem Wert prüfen
IPC	> 2	was blockiert
L1-Fehlgriffrate	$< 5\,\%$	Zugriffsreihenfolge
L3-Fehlgriffrate	$< 1\,\%$	Arbeitsmenge, Blockbildung
Sprungfehlerrate	$< 2\,\%$	datenabhängige Bedingungen
TLB-Fehlgriffrate	$< 0{,}1\,\%$	große Seiten erwägen
Kohärenzverkehr	gering	falsches Teilen

Nützliche Formeln.

$$t_{\text{mittel}} = t_{\text{Treffer}} + p_{\text{Fehl}} \cdot t_{\text{Strafe}}$$

$$\mathrm{CPI} = \mathrm{CPI}_{\text{ideal}} + \sum_i p_i \cdot s_i$$

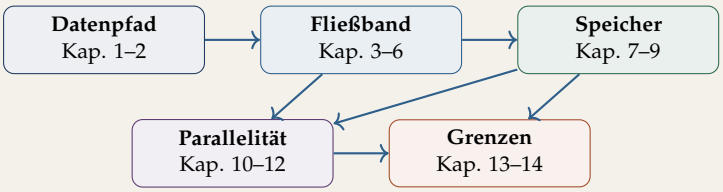
$$S = \frac{1}{(1-p) + \frac{p}{N}} \qquad \text{(Amdahl)}$$

$$T_{\text{Takt}} = \frac{T}{k} + r, \qquad S_{\text{Fließband}} = \frac{k}{1 + \frac{kr}{T}}$$

$$\text{arithm. Intensität} = \frac{\text{Operationen}}{\text{bewegte Bytes}}$$

A.2 Die Landkarte des Bandes

Aufbau und Querverbindungen



Informatik-Band 0	Gatter, Datenpfad, Abstraktionsschichten, Gleitkomma
Informatik-Band 1	$\mathcal{O}$-Notation, Datenstrukturen, warum die Analyse lügt
Informatik-Band 5	Betriebssysteme, Fäden, Synchronisation
Informatik-Band 6	Übersetzer, Befehlsanordnung, Optimierung
Informatik-Band 8	Kryptographie, Seitenkanäle
Informatik-Band 13	Lernende Systeme, Tensoreinheiten

A.3 Glossar

Die Begriffe dieses Bandes

Assoziativität	Zahl der Cacheplätze, die für eine Adresse in Frage kommen
Cachezeile	Kleinste Einheit, die geladen wird; meist 64 Byte
Chiplet	Teilchip, der im Gehäuse mit anderen verbunden wird
CPI	Takte je Befehl; Kehrwert des IPC
Divergenz	Auf einer GPU: Fäden einer Gruppe nehmen verschiedene Wege
Dunkles Silizium	Chipteile, die aus Energiegründen abgeschaltet bleiben müssen
Falsches Teilen	Zwei Kerne beschreiben verschiedene Variablen derselben Cachezeile
Fließband	Überlappende Ausführung mehrerer Befehle in verschiedenen Stufen
Kohärenz	Alle Kerne sehen für eine Speicherstelle dieselbe Wertefolge
Konsistenz	Regeln über die Reihenfolge von Zugriffen auf verschiedene Stellen
Lokalität	Zeitlich: bald wieder; räumlich: nebenan
Mikrooperation	Interne RISC-artige Zerlegung eines CISC-Befehls
NUMA	Zugriffszeit hängt davon ab, an welchem Sockel der Speicher hängt
Rückordnungspuffer	Sorgt dafür, dass Wirkungen in Programmreihenfolge sichtbar werden
Registerumbenennung	Trennung von Registernamen und physischer Speicherstelle
Seitenkanal	Beobachtbare Nebenwirkung, aus der sich Daten erschließen lassen
Spekulation	Ausführung vor der Gewissheit, dass sie nötig ist
TLB	Zwischenspeicher für Adressübersetzungen
Vorabholen	Daten laden, bevor sie angefordert werden
Warp	Gruppe von 32 GPU-Fäden, die denselben Befehl ausführen

A.4 Zeittafel

Von der Atlas bis heute

1945	von Neumann	gespeichertes Programm
1961	**Atlas**	**virtueller Speicher, Seitenwechsel**
1964	**IBM System/360**	**Architektur als Schnittstelle**
1965	IBM 360/85	erster Cache
1965	Moore	Verdopplungsgesetz
1967	**Tomasulo**	**Umordnung, Registerumbenennung**
1967	Amdahl	Grenze der Parallelisierung
1974	Dennard	Skalierungsgesetz
1976	Cray-1	Vektorrechner
1980	**Patterson, Ditzel**	**RISC**
1985	Smith, Pleszkun	präzise Ausnahmen
1989	Hill	Einteilung der Cachefehlgriffe
1991	**Yeh, Patt**	**zweistufige Sprungvorhersage**
1993	Pentium	superskalar im Massenmarkt
1995	**Wulf, McKee**	**„Hitting the Memory Wall“**
1995	Pentium Pro	Umordnung im Massenmarkt
1996	Kocher	Zeitangriffe auf Kryptographie
1996	MMX	SIMD für PCs
1999	Frigo, Leiserson	cache-oblivious-Verfahren
2001	Itanium	VLIW scheitert im Markt
2001	POWER4	erster Zweikernchip
2005	—	**Taktrate stagniert; Wende zu Mehrkern**
2006	Seznec	TAGE-Sprungvorhersage
2007	CUDA	GPUs allgemein programmierbar
2008	Hill, Marty	Amdahl im Mehrkernzeitalter
2009	Mytkowicz u. a.	Messungen sind voreingenommen
2009	Williams u. a.	Dachlinienmodell
2011	**Esmaeilzadeh u. a.**	**dunkles Silizium**
2011	RISC-V	offener Befehlssatz
2015	Google	TPU im Einsatz
2018	**Spectre, Meltdown**	**Mikroarchitektur als Angriffsfläche**
2019	Hennessy, Patterson	„A New Golden Age“
2019	AMD	Chiplets im großen Maßstab
2020	Apple M1	ARM im Arbeitsplatzrechner
2022	TSMC, Samsung	Gate-all-around in Fertigung
2026	—	Speicher nah am Rechenwerk; Spezialbausteine

B Kommentierte Literatur

Achtundzwanzig Titel

Die Auswahl enthält acht Originalarbeiten und mehrere frei verfügbare Texte. Die Reihenfolge ist alphabetisch.

[1] Adve, S. V.; Gharachorloo, K.: *Shared Memory Consistency Models: A Tutorial.* IEEE Computer 29(12), 1996.

Die klassische Einführung in Speichermodelle, auf zwölf Seiten und ohne Formalismus. Wer verstehen will, warum nebenläufiger Code auf ARM anders läuft als auf x86, fängt hier an.

[2] Asanović, K. u. a.: *The Landscape of Parallel Computing Research: A View from Berkeley.* Technischer Bericht, 2006.

Eine Bestandsaufnahme kurz nach der Mehrkernwende, mit dreizehn wiederkehrenden Rechenmustern. Der Bericht hat die Forschungsrichtung eines Jahrzehnts geprägt und ist immer noch lesenswert.

[3] Bryant, R. E.; O'Hallaron, D. R.: *Computer Systems: A Programmer's Perspective.* 3. Aufl., Pearson 2015.

Der beste Text für die Verbindung zwischen Programm und Maschine. Die Kapitel über Speicherhierarchie und Optimierung sind die praxisnächsten, die ich kenne, und alles ist an lauffähigem Code gezeigt.

[4] Drepper, U.: *What Every Programmer Should Know About Memory.* 2007.

114 Seiten über Caches, DRAM, NUMA und Vorabholen, frei verfügbar. Die Zahlen sind veraltet, die Erklärungen nicht. Wer nur einen Text zum Thema Speicher liest, sollte diesen nehmen.

[5] Esmaeilzadeh, H. u. a.: *Dark Silicon and the End of Multicore Scaling.* ISCA 2011.

Die Rechnung, die zeigte, dass sich immer größere Teile eines Chips nicht mehr gleichzeitig betreiben lassen. Sie hat die Hinwendung zu Spezialbausteinen wesentlich begründet.

[6] Fog, A.: *Optimization Manuals.* Fortlaufend aktualisiert seit 1996.

Fünf Handbücher, frei verfügbar, mit gemessenen Latenzen und Durchsätzen jedes Befehls auf jedem gängigen Prozessor. Die Hersteller veröffentlichen diese Zahlen oft nicht; Fog misst sie selbst. Bei ernsthafter Optimierung unentbehrlich.

[7] Harris, S. L.; Harris, D. M.: *Digital Design and Computer Architecture.* 2. Aufl., Morgan Kaufmann 2021.

Baut den Prozessor von den Gattern aufwärts auf, in Ausgaben für ARM und RISC-V. Die natürliche Fortsetzung von Band 0 dieser Reihe.

[8] Hennessy, J. L.; Patterson, D. A.: *Computer Architecture: A Quantitative Approach.* 6. Aufl., Morgan Kaufmann 2017.

Das Standardwerk. Argumentiert konsequent mit Messungen statt mit Meinungen und deckt alles ab, was in diesem Band steht, in größerer Tiefe. Wenn Sie *ein* Buch zu diesem Band kaufen: dieses.

[9] Hennessy, J. L.; Patterson, D. A.: *A New Golden Age for Computer Architecture.* CACM 62(2), 2019.

Die Turing-Rede der beiden. Sie ziehen Bilanz aus fünfzig Jahren und begründen, warum sie die kommenden Jahre für die spannendsten seit den 1980er Jahren halten. Zwölf Seiten, frei verfügbar.

[10] Herlihy, M.; Shavit, N.: *The Art of Multiprocessor Programming.* 2. Aufl., Morgan Kaufmann 2020.

Das Standardwerk zur nebenläufigen Programmierung, von der Theorie bis zu sperrfreien Datenstrukturen. Ungewöhnlich ehrlich darüber, wie schwer das ist.

[11] Hill, M. D.; Marty, M. R.: *Amdahl's Law in the Multicore Era.* IEEE Computer 41(7), 2008.

Sechs Seiten, die Amdahls Formel auf die Frage anwenden, wie man eine gegebene Chipfläche auf Kerne verteilt. Das Ergebnis, nämlich asymmetrische Chips, ist seit 2020 Praxis.

[12] Jacob, B.; Ng, S.; Wang, D.: *Memory Systems: Cache, DRAM, Disk.* Morgan Kaufmann 2007.

Tausend Seiten über Speicher auf allen Ebenen, bis hinunter zu Zeitdiagrammen der DRAM-Bausteine. Zum Nachschlagen, wenn man wissen will, warum eine Latenz genau so groß ist.

[13] Jouppi, N. P. u. a.: *In-Datacenter Performance Analysis of a Tensor Processing Unit.* ISCA 2017.

Die Beschreibung von Googles TPU, mit ungewöhnlich offenen Messwerten. Enthält auch die Feststellung, dass der Baustein schon bei Erscheinen durch die Speicherbandbreite begrenzt war.

[14] Kirk, D. B.; Hwu, W.-m. W.: *Programming Massively Parallel Processors.* 4. Aufl., Morgan Kaufmann 2022.

Das Standardlehrbuch für GPU-Programmierung, von den Grundlagen bis zu fortgeschrittenen Mustern. Erklärt Divergenz und Speicherzusammenfassung besonders klar.

[15] Kocher, P. u. a.: *Spectre Attacks: Exploiting Speculative Execution.* IEEE S&P 2019.

Die Originalarbeit. Erstaunlich gut lesbar, mit vollständigem Beispielcode. Wer den Angriff verstehen will, sollte nicht die Sekundärliteratur lesen, sondern diese sechzehn Seiten.

[16] Lipp, M. u. a.: *Meltdown: Reading Kernel Memory from User Space.* USENIX Security 2018.

Ebenso. Der Angriff ist einfacher als Spectre und die Darstellung entsprechend zugänglicher.

[17] McKenney, P. E.: *Is Parallel Programming Hard, And, If So, What Can You Do About It?* Fortlaufend, frei verfügbar.

Über sechshundert Seiten aus der Praxis des Linux-Kerns. McKenney hat RCU entwickelt und schreibt mit trockenem Humor über die Fallen, in die er selbst getappt ist.

[18] Mytkowicz, T. u. a.: *Producing Wrong Data Without Doing Anything Obviously Wrong!* ASPLOS 2009.

Zeigt, dass Umgebungsvariablen und Verknüpfungsreihenfolge die gemessene Laufzeit um bis zu einem Drittel verändern. Sollte lesen, wer Leistungsmessungen veröffentlicht, und wer welche liest.

[19] Patterson, D. A.; Hennessy, J. L.: *Computer Organization and Design.* RISC-V-Ausgabe, 2. Aufl., Morgan Kaufmann 2020.

Das Einführungswerk. Entwickelt Datenpfad, Fließband und Speicherhierarchie Schritt für Schritt, mit sorgfältig gestuften Aufgaben. Der richtige Einstieg vor [8].

[20] Seznec, A.: *Arbeiten zur TAGE-Sprungvorhersage.* Seit 2006, frei verfügbar.

Seznec hat seit 2004 praktisch jeden Vorhersagewettbewerb gewonnen. Seine Aufsätze sind technisch, aber sie enthalten die Verfahren, die tatsächlich in heutigen Prozessoren stecken.

[21] Smith, J. E.; Nair, R.: *Virtual Machines: Versatile Platforms for Systems and Processes.* Morgan Kaufmann 2005.

Behandelt Virtualisierung auf allen Ebenen, von der Binärübersetzung bis zu Hypervisoren. Erklärt auch, wie Apples Rosetta-Übersetzung im Prinzip arbeitet.

[22] Sorin, D. J.; Hill, M. D.; Wood, D. A.: *A Primer on Memory Consistency and Cache Coherence.* 2. Aufl., Morgan & Claypool 2020.

Etwa 200 Seiten, die Kohärenz und Konsistenz sauber trennen, was in der Literatur oft vermischt wird. Der beste Text zu diesem verwirrenden Thema.

[23] Tanenbaum, A. S.: *Structured Computer Organization.* 6. Aufl., Pearson 2012.

Entwickelt den Rechner konsequent als Schichtenmodell, von der Digitallogik bis zum Betriebssystem. Das Buch, das die Denkfigur aus Band 0, Kapitel 12 geprägt hat.

[24] Tomasulo, R. M.: *An Efficient Algorithm for Exploiting Multiple Arithmetic Units.* IBM Journal of Research and Development 11(1), 1967.

Zwölf Seiten, die dreißig Jahre lang niemanden interessierten und heute in jedem Hochleistungsprozessor stecken. Bemerkenswert klar geschrieben.

[25] Waterman, A.; Asanović, K. (Hrsg.): *The RISC-V Instruction Set Manual.* Fortlaufend, frei verfügbar.

Die offizielle Beschreibung. Ungewöhnlich gut lesbar für ein Normendokument, mit Begründungen für die Entwurfsentscheidungen in eigenen Abschnitten.

[26] Williams, S.; Waterman, A.; Patterson, D.: *Roofline: An Insightful Visual Performance Model for Multicore Architectures.* CACM 52(4), 2009.

Führt das Dachlinienmodell ein, mit dem sich in einem Bild ablesen lässt, ob eine Rechnung durch die Rechenwerke oder durch die Speicherbandbreite begrenzt ist. Eine der nützlichsten Denkfiguren des Gebiets, und in zehn Minuten verstanden.

[27] Wulf, W. A.; McKee, S. A.: *Hitting the Memory Wall: Implications of the Obvious.* Computer Architecture News 23(1), 1995.

Vier Seiten, die das Grundproblem dieses Bandes benennen. Die beiden rechneten aus, was jeder wusste, und zogen den Schluss, den niemand ziehen wollte. Sie schließen mit dem Satz, sie hofften sich zu irren.

[28] Yeh, T.-Y.; Patt, Y. N.: *Two-Level Adaptive Training Branch Prediction.* MICRO 1991.

Die Arbeit, mit der die moderne Sprungvorhersage begann. Die Idee, Sprunggeschichte und Sprungadresse gemeinsam als Index zu benutzen, steckt in jedem heutigen Vorhersager.

Wenn Sie nur drei Bücher kaufen

[19]	**Patterson, Hennessy**	zum Lernen – Schritt für Schritt aufgebaut
[3]	**Bryant, O'Hallaron**	zum Programmieren – die Verbindung zum Code
[8]	**Hennessy, Patterson**	zum Nachschlagen – alles Weitere

Frei verfügbar und an einem Wochenende zu lesen: [4] Drepper *über Speicher*, **[27] Wulf und McKee** *über das Grundproblem*, **[9] Hennessy und Patterson** *über die Zukunft*.

*Und wer messen will, braucht **[6] Fog** – die Zahlen stehen sonst nirgends.*